



Module Code & Module Title

CS6P05NI Project

Interim Report

25% Individual Coursework

Submission: Final Submission

AY 2025 2026

Academic Semester: Autumn Semester 2025

Credit: 30 Credit Year Long Module

Student Name: Viom Lal Shrestha

London Met ID: 23048612

College ID: NP01AI4A230041

Assignment Due Date: Tuesday, December 30, 2025

Assignment Submission Date: Tuesday, December 30, 2025

Internal Supervisor: Alish K.C

External Supervisor: Rozeen Nakarmi

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1. Introduction	1
1.1 Problem Scenario	2
1.2 Project as a solution.	4
1.3 Aims and Objectives	5
1.3.1 Aim	5
1.3.2 Objectives:	5
1.4 Report Structure	5
1.4.1 Background	5
1.4.2 Development Methodology.....	5
1.4.3 Development Till Date	5
1.4.4 Analysis of Progress	6
1.4.5 Future Work	6
2. Background and Literature Review	7
2.1 About the End User / Client	8
2.2 Understanding the Project	9
2.3 Review of Similar Projects	10
2.3.1 Whering - Your Digital Closet	10
2.3.2 Acloset - AI Fashion Assistant.....	11
2.3.3 Fits - Outfit Planner & Closet App	11
2.4 Comparison Table of Similar Projects.....	13
2.5 Comparative Analysis of Similar Projects	14
3. Development Methodology.....	15
3.1 Considered Methodologies	15
3.2 Selected Methodology: Iterative Incremental Methodology	15
3.3 Work Breakdown Structure	17
3.4 Gantt Chart	19

3.5 Resource Requirement.....	20
4. Development Till Date	20
4.1 Initial Survey	20
4.2 DFD	21
4.2.1 DFD Level 0	21
4.2.2 DFD Level 1	22
4.2.3 DFD Level 2: Manage Storage.....	23
4.3 System Architecture Diagram	24
4.4 Use Case Diagram	25
4.4.1 High Level Use Case Description.....	26
4.1.2 Expanded Use Case Description	27
4.5 ERD	28
4.6 SRS	28
4.7 Sequence Diagram	29
4.7.1 Sequence Diagram: Manage Account.....	29
4.7.2 Sequence Diagram: Upload Clothing Item	30
4.7.3 Sequence Diagram: Manage Storage	31
4.7.4 Sequence Diagram: Generate Outfit	32
4.7.5 Sequence Diagram: Add Non-Clothing Item	33
4.7.6 Sequence Diagram: View Admin Dashboard	34
4.8 Flowchart Diagram.....	35
4.8.1 Flowchart Diagram: Manage Account	35
4.8.2 Flowchart Diagram: Upload Clothing Item.....	36
4.8.3 Flowchart Diagram: Add Non-Clothing Item.....	37
4.8.4 Flowchart Diagram: Manage Storage.....	38
4.8.5 Flowchart Diagram: View Admin Dashboard.....	38
4.8.6 Flowchart Diagram: Generate Outfit.....	39

4.9 Wireframe Design	40
4.10 Development.....	40
4.10.1 Frontend Development.....	40
4.10.2 Backend Development	41
4.10.3 AI Development.....	41
4.10.4 Database Configuration and Model Creation	42
4.11 UI Design.....	42
4.12 Project Management and Version Control	42
5. Progress Analysis.....	43
5.1 Progress Review.....	43
5.1.1 Requirement Analysis and Planning	44
5.1.2 System and Architecture Design	44
5.1.3 Increment 1: User Authentication	44
5.1.4 Increment 2: Segmentation and Color Extraction.....	45
5.1.4 Increment 3: Classification (Initial)	45
5.2 Progress Table	46
5.3 Action Plan	49
6. Future Work	50
6.1 Backend Development.....	50
6.2 Frontend Development	51
6.3 AI Development	51
6.4 System Integration and Testing	52
6.5 Finalization and Documentation.....	52
6.6 Expected Future Work	52
7. Conclusion	53
8. References.....	54
9. Appendix	57

9.1 Software Requirement Specifications	57
9.1.1 Introduction	57
9.1.2 General Description	58
9.1.3 Functional Requirements	60
9.1.4 Non-Functional Requirements.....	62
9.1.5 Conclusion	62
9.2 Details of Considered methodology	63
9.2.1 Iterative Waterfall Methodology.....	63
9.2.2 Scrum Methodology	65
9.3.3 Spiral Methodology	66
9.4 Phases of Iterative Incremental Methodology	67
9.5 Gantt Chart Task Table	69
9.6 Resource Requirement Specifications	72
9.7 Expanded Use-Case Description	73
9.8 Online Survey Form.....	79
9.9 Wireframes	93
9.10 UI Design Screenshots	103
9.11 Development Evidence	116
9.11.1 Frontend Development.....	116
9.11.2 Backend Development	127
9.11.3 AI Development.....	132
9.11.4 Migration and Model Creation	134
9.12 Project Management and Versioning	138
9.12.1 Trello	138
9.12.2 GitHub	139

Table of Figures

Figure 1: Problem for Clothing Selection.....	1
Figure 2: Consumption Growth rate of clothing (Yehounme, 2017)	2
Figure 3: Wardrobe App Market (WiseGuy Reports, 2025).....	7
Figure 4: Whering website wardrobe screenshot	10
Figure 5: Acloset website screenshot.....	11
Figure 6: Fits App Screenshot.....	12
Figure 7: Iterative Incremental Model	15
Figure 8: WBS	17
Figure 9: WBS Part-1	18
Figure 10: WBS Part-2	18
Figure 11: Gantt Chart.....	19
Figure 12: DFD Level 0	21
Figure 13: DFD Level 1	22
Figure 14: DFD Level 2 - Manage Storage.....	23
Figure 15: System Architecture Diagram.....	24
Figure 16: Use Case Diagram.....	25
Figure 17: ERD	28
Figure 18: Manage Account Sequence Diagram.....	29
Figure 19: Upload Clothing Item Sequence Diagram	30
Figure 20: Manage Storage Sequence Diagram	31
Figure 21: Generate Outfit Sequence Diagram	32
Figure 22: Add Non-Clothing Item Sequence Diagram	33
Figure 23: View Admin Dashboard Sequence Diagram	34
Figure 24: Manage Account Flowchart Diagram	35
Figure 25: Upload Clothing Item Flowchart Diagram.....	36
Figure 26: Add Non-Clothing Item Flowchart Diagram	37
Figure 27: Manage Storage Flowchart Diagram.....	38
Figure 28: View Admin Dashboard Flowchart Diagram.....	38
Figure 29: Generate Outfit Flowchart Diagram.....	39
Figure 30: Django settings configuration	42
Figure 31: System Perspective Diagram	58
Figure 32: Iterative Waterfall (GeeksforGeeks, 2025)	63

Figure 33: Scrum Cycle (GeeksforGeeks, 2025).....	65
Figure 34: Spiral Methodology Cycle (GeeksforGeeks, 2025)	66
Figure 35:Survey Form 1.....	79
Figure 36:Survey Form 2.....	80
Figure 37:Survey Form 3.....	81
Figure 38:Survey Form 4.....	82
Figure 39:Survey Form 5.....	83
Figure 40: Survey Form Response 1.....	84
Figure 41:Survey Form Response 2	85
Figure 42:Survey Form Response 3	86
Figure 43:Survey Form Response 4	87
Figure 44:Survey Form Response 5	88
Figure 45:Survey Form Response 6	89
Figure 46:Survey Form Response 7	90
Figure 47:Survey Form Response 8	91
Figure 48: Survey Form Response 9.....	92
Figure 49: Loading Page	93
Figure 50: Login Page	94
Figure 51: Register Page	95
Figure 52: Homepage Wireframe	96
Figure 53: Profile Page Wireframe	97
Figure 54: Add Item Wireframe	98
Figure 55: Add Non-Clothing Item Wireframe	99
Figure 56: Upload Clothing Wireframe	100
Figure 57: Segmentation Screen Wireframe	101
Figure 58: Metadata display Wireframe.....	102
Figure 59: Loading Screen UI	103
Figure 60: Login Screen UI	104
Figure 61: Registration Page.....	105
Figure 62: Homepage.....	106
Figure 63: Profile Page 1.....	107
Figure 64: Profile Page 2.....	108
Figure 65: Add Item Page1	109
Figure 66: Add Item Page 2	110

Figure 67: Upload Clothing page.....	111
Figure 68: Processing Image	112
Figure 69: Confirm Segmentation Page	113
Figure 70: Clothing Metadata Page.....	114
Figure 71: Add Non-Clothing page	115
Figure 72: Frontend folder structure	116
Figure 73: Login registration code	117
Figure 74: Session management code	118
Figure 75: Session management code 2	119
Figure 76: Code to Fetch user information	120
Figure 77: Image segmentation backend	121
Figure 78: Delete segmented image code.....	122
Figure 79: metadata extraction code	122
Figure 80: Loading page code.....	123
Figure 81: Login page code.....	124
Figure 82: Registration page code	125
Figure 83: Profile page code	126
Figure 84: Backend folder structure	127
Figure 85: User authentication with jwt.....	127
Figure 86: Serializer code 1	128
Figure 87: Serializer code 2	129
Figure 88: Backend code	130
Figure 89: Segmentation backend	131
Figure 90: Colour extraction backend.....	132
Figure 91: Segmentation code	132
Figure 92: Code for colour extraction	133
Figure 93: Storage Unit Model	134
Figure 94: Clothing Item Model	135
Figure 95: Non-Clothing Item Model.....	135
Figure 96: Outfit Model.....	136
Figure 97: Successfully Migrating the database	136
Figure 98: Database view in PostgreSQL	137
Figure 99: Trello Board.....	138
Figure 100: GitHub Repository for the project	139

Table of Tables

Table 1: Feature Comparison of Similar Projects	13
Table 2: Backend Future Works	50
Table 3: Frontend future works	51
Table 4: AI Development future works	51
Table 5: System Integration future works	52
Table 6: Finalization work.....	52
Table 7: Reasoning for Iterative Waterfall	64
Table 8: Reasoning for Scrum.....	65
Table 9: Reasoning for Spiral Model	66
Table 10: Gantt Chart Task Table	69

Table of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CAGR	Compound Annual Growth Rate
DB	Database
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
FYP	Final Year Project
ML	Machine Learning
SRS	Software Requirements Specification
UI	User Interface
UX	User Experience
U ² -Net	U-Squared Net

Acknowledgement

I would like to thank all the help which I have received during this journey from the teachers and other members of Islington College. They have all been very helpful in providing me with a positive environment where I can grow.

I would like to extend my sincere gratitude to my internal supervisor Mr. Alish K.C. and external supervisor Mr. Rozen Nakarmi, who helped me with their valuable supervision and guidance while carrying out the development of my Closet Buddy project. The development of my project would not have gone so smoothly without their excellent supervision and industry expertise.

I would also like to thank my family and my friends for the continuous motivation, patience, and support they have given me throughout the duration of this project. Though not directly contributing towards the completion of the project, their encouragement, criticisms, and feedback on the design of the project and my rationale played an important role in shaping the rationale for this project.

This particular project would not have been possible without the help and support of those mentioned above as well as the College itself.

Abstract

Closet Buddy is an artificial intelligence (AI) powered virtual wardrobes management software that aims at assisting users to systematically store their clothing online and provide smart clothing suggestions. Several people experience problems with managing their wardrobes effectively, and this process is likely to result in inappropriate selections of outfits, inadequate use of garments, and waste of time in choosing clothes and spending time on making decisions on a daily basis. The proposed project is expected to resolve these problems through integrating mobile application development with artificial intelligence methodologies.

Users can add pictures of their clothes in the system, and this is automatically verified, segmented and what is extracted is the most important metadata, color, and type. These objects are then stored in a virtual storage space which can be configured to resemble real-life storage areas such as closets, drawers, and boxes. According to the existing clothing selections, the system is developed to produce outfit recommendations through a hybrid solution which incorporates rule-based thinking and AI based solutions. Personalization is supported with time, using user feedback in the form of ratings.

The Closet Buddy is created based on Flutter as a frontend, Django with a relational database as a back end and deep learning models can be concluded as image segmentation and color extraction. It is developed in an incremental form of development to provide systematic development and scalability. The work shows that artificial intelligence can be efficiently used in dealing with personal wardrobe and provides an effective, convenient, and scalable solution.

1. Introduction

In our chaotic everyday world where life is becoming fast paced, and we as humans are seeking new ways to make our way of life easy and more convenient, one of the most important aspects of our everyday life, being our daily wear is still not up to date to our steady fast lifestyle. We know that clothes make the first impression and are a way to express ourselves, but not everyone of us are born with a sense of style and most of us struggle with making our outfit in a hurry through unorganized closets and hit-and-trial on matching clothes.

Not only is trying to mix and match clothes to make an outfit difficult, but it is also difficult for people to remember where exactly they have placed a certain clothing item due to messy and unorganized storage habits. The mental load of daily dressing has also become a burden on our everyday life. Studies suggest that an average person spends about 10-17 minutes per day to choose an outfit, some can decide in seconds while some even take 30 minutes or more (Quora, 2015).



Figure 1: Problem for Clothing Selection

To solve this issue, 'Closet Buddy' an AI powered virtual closet and storage assistant, that can assist the users to manage and organize their storage spaces more effectively and creating daily outfit for them while also keeping track of their outfits and clothes has been proposed.

1.1 Problem Scenario

Many people struggle with their wardrobes and outfit coordination due to busy lifestyles, leading to time inefficiencies while dressing. Traditional wardrobe management is completely manual, unstructured and offers no intelligent solutions to visualize their wardrobe, track usage frequency, and receive intelligent outfit recommendations. This lack of a unified digital platform for wardrobe management and organization leads to significant time inefficiencies and “wardrobe panic”, a phenomenon where an individual feels like they have nothing to wear even though they have a ton of clothes sitting by uselessly, which according to a survey an average person experiences at least 36 times in a year (HERStyle, 2023).

The root cause of this issue is due to the lack of wardrobe visibility: research revealed that 61% of the people simply forget what they own, which results in unnecessary purchases and low reuse rate of already owned clothing items (Bang, 2022). In fact, research shows that on average a person wears only 20% of the clothes that they already own on a regular basis. Leaving the remaining 80% of your clothes in the wardrobe are used for a long period of time (The Carbon Literacy Trust, 2025).

Average Consumer Bought **60%** More Clothing in 2014 Than in 2000,
But Kept Each Garment Half as Long

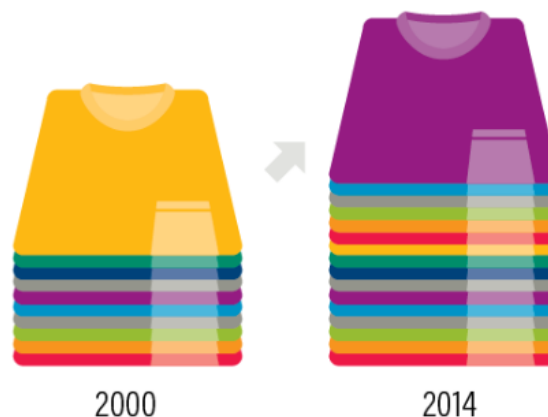


Figure 2: Consumption Growth rate of clothing (Yehounme, 2017)

These inefficiencies and problems have a major impact environmentally. From 2000 to 2014, the amount of clothing items bought by a person increased by 60% (McKinsey & Company, 2025). Not only this but clothing is only worn 7 to 10 times before being thrown away, a decline of 35% than 15 years ago (uniformmarket, 2025).

The fashion sector generates 92 million tonnes of textile waste annually, of which a significant portion ends up in landfills (United Nations Environment Programme (UNEP), 2025). Without proper tools and technology to help individuals organize, reuse, and utilize their clothes to the full extent of their life, the imbalance between production and utilization in the textile industry continues to grow and affects environmental and social sustainability.

1.2 Project as a solution.

The modern problems related to fashion consumption are caused by poor wardrobe management, inefficient organization and decision fatigue during selection of outfit. Although people own many garments, they still struggle to keep track of what they own, where they had stored the item, how often they are used, and how to combine them efficiently. To address both the personal and environmental challenges, the proposed project titled “Closet Buddy” aims to develop an AI-driven virtual wardrobe application that digitalizes a user’s storage spaces and manages the storage units to better help them organize, understand, and reuse their existing wardrobe to facilitate the users. The problems this project aims to solve are:

- **Wardrobe Visualization:** This project transforms physical storage spaces into their structured digital representation (Closets/Drawers/Boxes) in which the users can map their real-life storage spaces digitally because of which the problem of unorganized clothing spaces can be solved. This enables the user to get an accurate overview of their belongings and allows the user to locate, manage, and review clothing items more efficiently.
- **Automates clothing categorization:** A major issue for wardrobe digitalization is the manual effort required to categorize and manage clothing data, but this project solves this issue by automatically validating the uploaded image to ensure that the picture presents clothing items then applies segmentation for extracting clean cutouts and applies AI to extract meaningful meta data from the clothing. This helps to reduce user efforts and reduces data inconsistencies.
- **Generates Outfit:** Closet Buddy analyzes the digital wardrobe and uses both rule-based logic and AI driven methods to generate outfit suggestions, this helps to reduce decision fatigue and improve daily dressing efficiency.

Potential feature and problem this project can solve and implement:

- **Improves dressing sense:** Closet Buddy allows the user to make their own outfit from their wardrobe and receive a rating by AI. This helps the user to improve their dressing sense over time.

In conclusion, Closet Buddy serves as a comprehensive solution to various problems related to wardrobe disorganization, inefficient usage, decision fatigue, and unsustainable clothing habits by combining AI and modern technologies.

1.3 Aims and Objectives

1.3.1 Aim

To develop an AI-powered virtual storage and wardrobe assistant that authenticates and digitizes clothes into a well-structured storage component, recommends daily outfits based on user requirements, promotes sustainability, and integrates with weather Api to enhance user convenience and reduce fast-fashion-driven waste.

1.3.2 Objectives:

1. To provide the users with a user-friendly convenient way to manage and track their real-life storage spaces, specially targeted towards clothing management and keeping record of their outfits.
2. To integrate authentication, classification and outfit recommendation features with AI and provide the users with a service to generate outfits according to their requirements.

1.4 Report Structure

This section outlines the report structure.

1.4.1 Background

This section details information about the end user of this project as well as understanding the project on a deeper level. This section also includes research on similar on a distinct similar such projects related to wardrobe organization or outfits generation.

1.4.2 Development Methodology

In this section the methodology considered for this project will be discussed as well as the methodology which was selected for this project. This section will also include work breakdown structure and Gantt chart of the project

1.4.3 Development Till Date

In this section information related to the UML diagrams like use case diagram and data flow diagram will be presented. Wireframes and UI designs that have been made till now will also be presented in this section along with the Software Requirement Specification. This section will also include development status with evidence.

1.4.4 Analysis of Progress

This section documents the progress level of the project till the submission of interim report. This section is used to measure how far along the project is to completion and to see if the development progress is going on as intended.

1.4.5 Future Work

In the future works section, the remaining portion of the development is to be mentioned and discussed about. After acknowledging the development that has already been done the development that is still to be done is to be mentioned in this part of the report.

2. Background and Literature Review

The rapid growth and development of fast fashion and digital consumerism culture have significantly changed how people get and manage clothing. Although now clothes have become more affordable and more people can buy fashionable and good quality clothes at less cost it has also resulted in overcrowded closets, inefficient clothing management, and increased textile waste across the world. People own more clothes than they can actively use yet have trouble in selecting outfits and wardrobe organization in everyday life.

Wardrobe App Market (USD Bn)

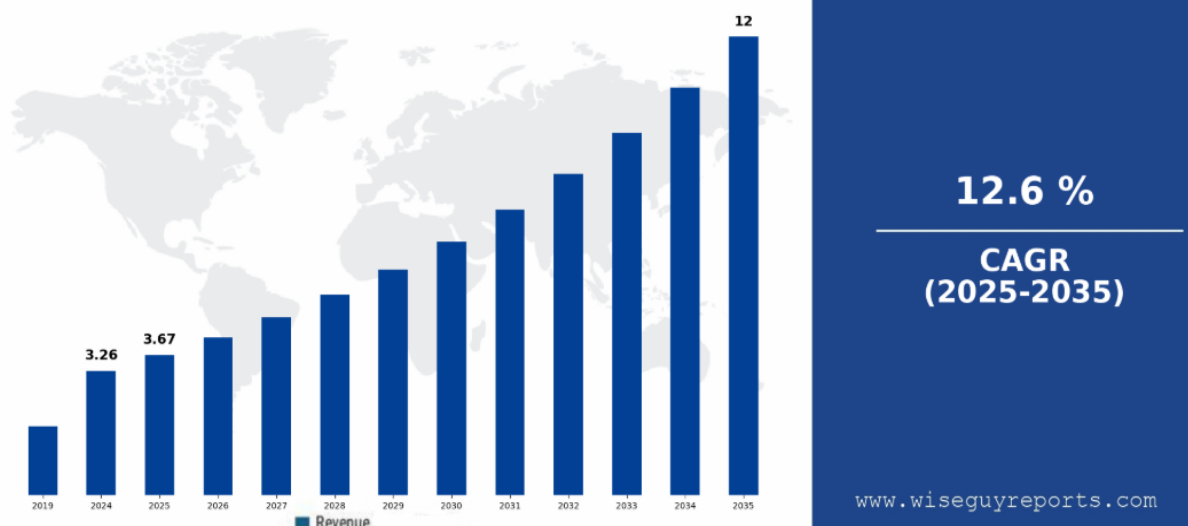


Figure 3: Wardrobe App Market (WiseGuy Reports, 2025)

Because of these issues the concept of “Virtual Wardrobes or “Digital Closets” are starting to become a practical necessity in modern times. The rise of “Fast Fashion” era characterized by high volume production and low-cost garments has led to a crisis of over consumption where people lose track of their possessions. “According to recent academic studies and industry reports it is said that from 2024 AD to 2025 AD the digital wardrobe market is expected to grow at a CAGR of about 12.6% reaching to an estimated value of over \$12 billion by 2025 2035 AD” (WiseGuy Reports, 2025). The reason for this growth is mainly due to a shift towards Sustainable Fashion and Circular Economy, in which focus is mainly given to maximum utilization of existing items rather than buying new ones.

2.1 About the End User / Client

The primary users of the Closet Buddy system are usually individuals who own multiple pieces of clothing items and have trouble organizing, tracking, and efficiently utilizing the items in their wardrobe. This demographic includes students, working professionals, or fashion-conscious individuals, usually Millennials and Gen Zs who are likely to have high volume of clothing but lack the time to manage it, making them the primary beneficiaries of this project. They face daily challenges with outfit selection due to various reasons like limited time, decision fatigue, or lack of awareness of their existing clothing. These users typically store their clothing across multiple physical spaces such as closets, drawers, or storage boxes often without any systematic structure for tracking item location or usage. As a result, many garments may be forgotten, rarely worn, or repurchased without need. The target users may also lack fashion knowledge and would greatly benefit from intelligent guidance on outfit coordination and styling.

The secondary target users of this project are people who advocate for minimalism and stand for sustainable fashion, such users focus on cost-per-wear and are mindful of their carbon footprint. This project serves as a tool for these users to ensure that every item in their limited clothing collection is being utilized effectively. This project is also suitable for users who focus on sustainable consumption but lack the tools to translate this awareness into daily behavior because Closet Buddy provides insights into clothing uses and promotes underutilized garments promoting sustainability to some extent.

The Closet Buddy application is designed to serve a broad spectrum of users and can be utilized across different levels of technical literacy and life stages. Another target user of this project is the user group for whom the primary objective is not style but inventory management. Large households or even smaller ones often struggle with lost items such as seasonal blankets, extra linens, or holiday decorations stored in high self-boxes, under bed spaces, or additional closets. Unlike traditional fashion apps, users can also store non-clothing items like sleeping bags, blankets, or sheets as non-clothing items in the storage spaces. By knowing exactly which box contains items like winter blankets or extra seats this helps the users to eliminate the physical labor of searching through multiple storage spaces, saving time and reducing stress.

2.2 Understanding the Project

The project “Closet Buddy” is designed as an AI powered virtual wardrobe and storage management system that helps to connect physical clothing storage with digital organization. This project aims to address functional and behavioral challenges associated with storage management by digitalizing not only clothing items but also non-clothing items and their respective storage environment.

At the core of this project the system enables users to upload images of their clothing items which are then automatically processed to verify if the uploaded image is a clothing item or not, then it removes the background using U²-Net to produce a clean cutout and extracts meaningful attributes like category (Top wear, Bottom wear, Foot wear), subcategory (Shirt, Pant, T-Shirt,...), usage type, and color. This information is stored within a structured database that mirrors the user’s real life storage layout, allowing clothing items to be mapped to specific storage spaces like closets, drawers or boxes. The users also have an option to store non-clothing items into the storage spaces, to store non-clothing items users can directly store them without uploading image or extracting metadata. This feature is included to simulate real life storage spaces so as to facilitate users to track items other than clothing that are also stored in these spaces.

Beyond wardrobe organization this project also supports outfit recommendation using the clothes available in the user's closet considering things like garment compatibility, usage patterns, factors such as weather and user. Getting user feedback on the outfit can further enhance personal personalization and enable the system to adapt recommendation over time.

By providing features such as automation, artificial intelligence and user-oriented design, Closet Buddy is not only a mere cataloging tool but is also an intelligent assistant that helps users to understand their wardrobe layout, visualize storage spaces, track clothing usage and outfit repetition, reduce overconsumption, and improve daily dressing. Therefore, this project shifts from traditional passive wardrobe listing system to an interactive virtual wardrobe solution.

2.3 Review of Similar Projects

In recent years a number of mobile and web applications have emerged to help users manage their wardrobe, plan outfits and make better use of their existing clothes. These projects have different functionalities from one to another, some apps focus on simple wardrobe cataloging and organization, while others fully focus on AI enhancement including styling assistants. Research on similar project helps to understand the standing of Closet Buddy project within the current market.

2.3.1 Whering - Your Digital Closet

Whering provides a digital closet combined with social and community components with user engagement. Users can upload clothing items from photos or from retailer derivatives as the app offers a chrome extension to clip items directly from shopping sites to build wardrobes. It allows users to collaborate on an outfit with their friends, making it interactive. It is a sustainability first app in the market and focuses on the social aspect of styling and reducing fashion waste.

Link: <https://app.whering.co.uk>

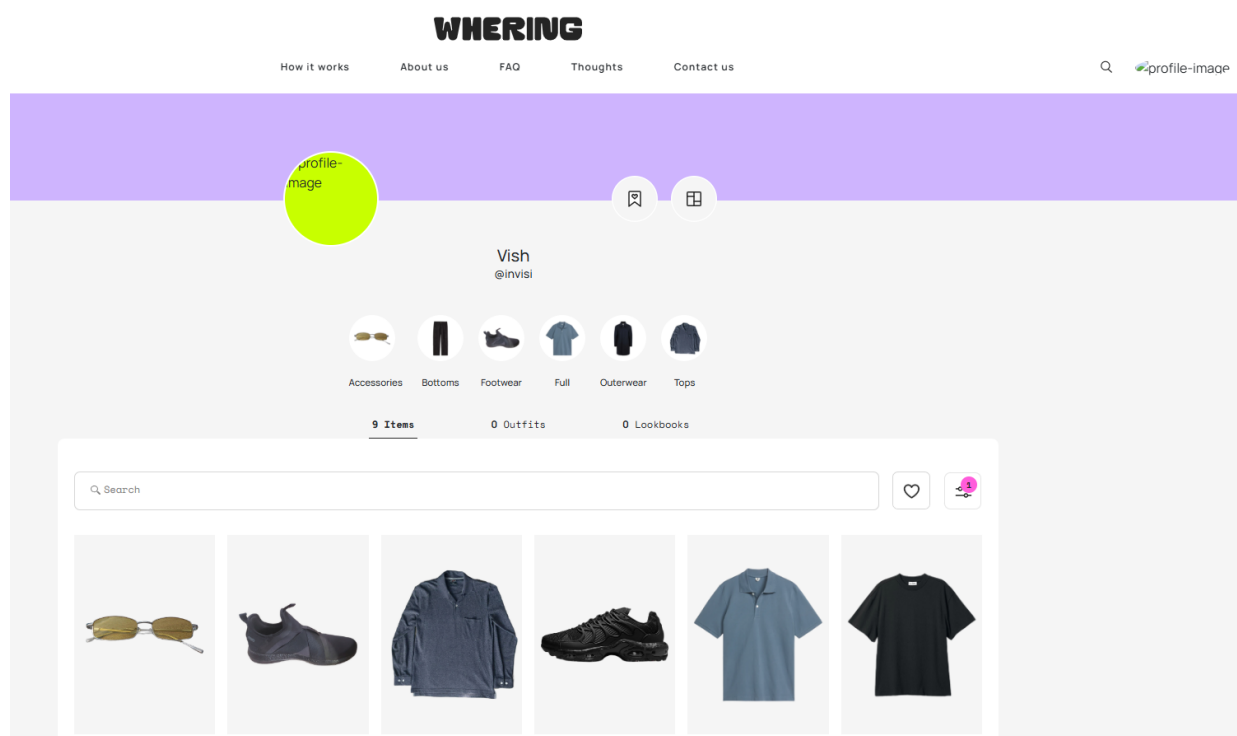


Figure 4: Whering website wardrobe screenshot

2.3.2 Acloset - AI Fashion Assistant

Acloset focuses on both wardrobe organization and analytics providing a comprehensive fashion ecosystem that blends organization with a social marketplace. It places itself as a data-driven tool for “Circular Economy”. It uses AI engines to generate output suggestions based on weather, occasion or even a user’s color analysis. It has a built-in resale market as well as a cost-per-wear tracker. Its shopping advisor analyzes the clothes in our wardrobe and recommends new purchases depending on it. Free accounts can only upload 100 items and require users to watch ads for specific actions.

Link: <https://www.acloset.app>

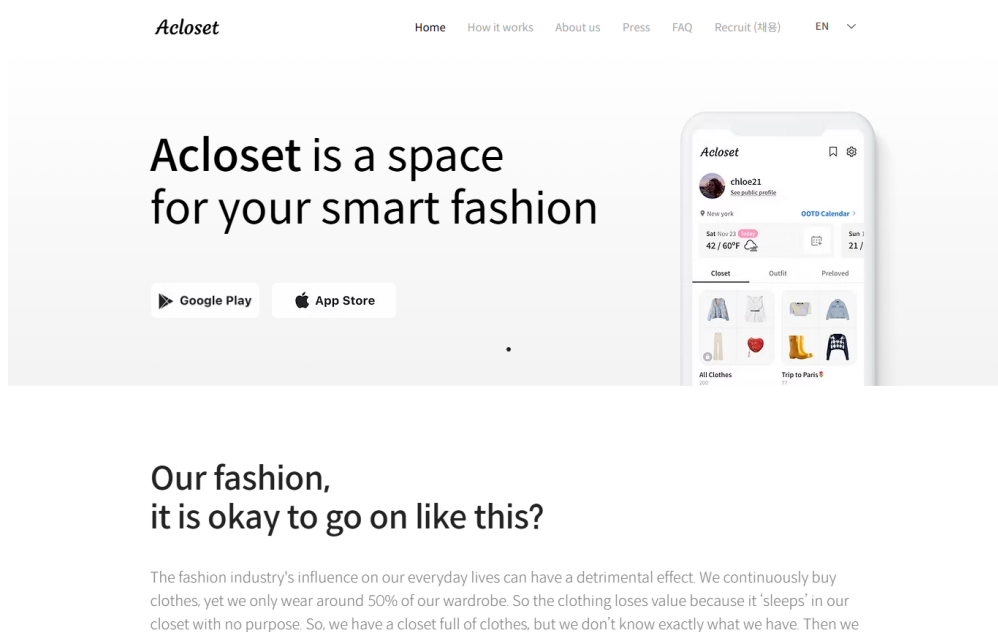


Figure 5: Acloset website screenshot

2.3.3 Fits - Outfit Planner & Closet App

Fits is a digital wardrobe and outfit planner planning application that helps the user to organize their clothes and plan for outfits. The main focus of this app is the ease of creating outfits and styling inspiration. This app uses deep learning for highly accurate background removal, and metadata tagging as well as AI Try on feature. It has a similar UI to Instagram where users can post their outfits of the day, view other user’s profile and outfits, give likes, or follow other users. Also, it has wardrobe analytics for the user and offers unlimited item uploads for free users too.

Link: <https://www.fits-app.com>

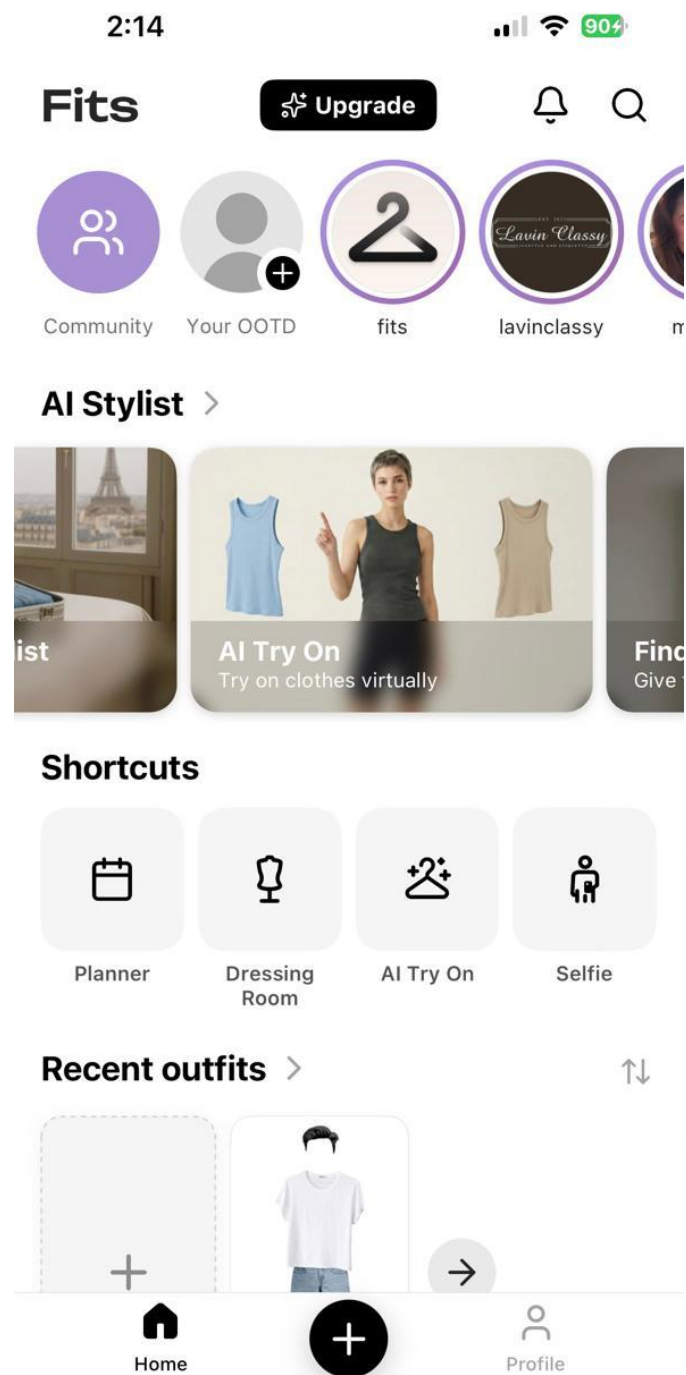


Figure 6: Fits App Screenshot

2.4 Comparison Table of Similar Projects

Similar digital wardrobe apps like this project vary widely in the features that they provide and how much they use automation, analytics, or AI. The table below compares the three similar systems above with the proposed system “Closet Buddy” based on different features:

Table 1: Feature Comparison of Similar Projects

Features	Fits	Acloset	Whering	Closet Buddy (Proposed)
Digital wardrobe	✓	✓	✓	✓
Clothing Authentication	X	X	X	✓
AI Background Removal	✓	✓	✓	✓
AI metadata extraction	✓	✓	Limited	✓
AI-powered Outfit Generation	✓	✓	✓	✓
Feedback & learning (ratings)	X	X	X	✓
Mapping to real storage spaces	X	X	X	✓
"Put Away" Logic	X	X	X	✓
Store Non-Clothing Items	X	X	X	✓
Mobile Support	✓	✓	✓	✓
Web Support	X	X	✓	X

2.5 Comparative Analysis of Similar Projects

Comparative analysis of similar projects with Closet Buddy helps to reveal strengths and limitations. Apps like Acloset and Whering treats wardrobe as a single flat file and do not account for where an item is physically located, Closet Buddy addresses this by mapping real storage spaces digitally. This makes it easier to find where exactly an item is located quickly.

Fits focus more on outfit creation and styling inspiration using AI. However, it lacks deeper wardrobe analytics and usage tracking, limiting its capacity for wardrobe and sustainability management. Acloset on the other hand provides more features for wardrobe management like usage statistics and weather-based suggestions but much of its functionality is limited and constrained behind the paywall. Whering has social features and promotes community and discovery through engagement with other users. This app is useful for people who enjoy fashion communities and idea sharing so it's emphasis more on community inspiration and less on AI driven recommendation and personalization.

Existing system may recommend a clothing item like a sweater in an outfit because it matches the weather or occasion even if it's stored away in the attic or difficult to reach places, Closet Buddy's "Put Away" logic solves this by automatically filtering out items stored away, ensuring the AI uses only the clothes currently reachable while recommending outfits. Most of these apps are strictly for fashion purposes only but Closet Buddy is a hybrid tool that also allows tracking of non-clothing items within the same storage spaces, providing a more holistic "home organizer" value than a traditional fashion app.

3. Development Methodology

The development of a system like Closet Buddy, which is an AI driven full stack system requires a methodology that supports experimentation, experimental feature development, and refinement. Given that it involves multiple components like image processing, machine learning models, back-end services, front end, etc.

3.1 Considered Methodologies

The following methodologies were considered for the project:

- I. Iterative Waterfall Methodology
- II. Scrum methodology
- III. Spiral methodology

The further details about these methodologies are discussed in the [appendix](#).

3.2 Selected Methodology: Iterative Incremental Methodology

The methodology selected for this project is Iterative Incremental Model. It was deemed suitable as the methodology of this project because this method helps to break down complex projects into smaller parts instead of figuring out everything at the beginning. **Invalid source specified..** It combines structured development with flexibility allowing the developers to focus on making each part working well before adding more, resulting in a better overall system.

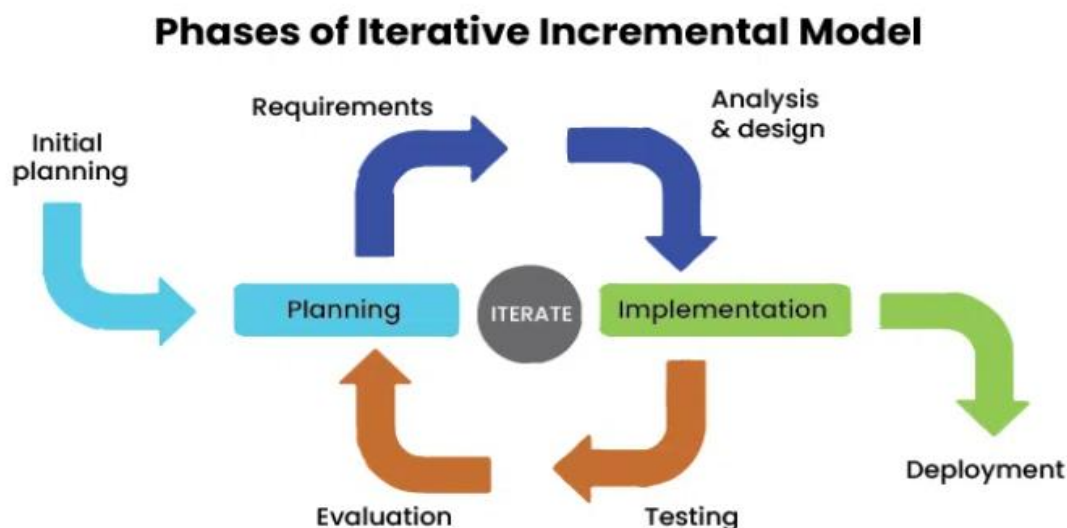


Figure 7: Iterative Incremental Model

The system will be built through small increments, where each increment adds functional components like authentication, segmentation, classification, closet storage, recommendations, etc. Each increment itself goes through iterations of planning, design, implementation, and testing, ensuring continuous refinement and reduced development risk. This model aligns perfectly well with the experimental and modular nature of the project especially in AI-driven components.

The methodology phases are provided in the appendix.

Reasons for choosing this methodology:

- As the project progresses, various requirements related to AI models, system design or recommendation logic might evolve with it and these evolving requirements are supported by this methodology. As the iterative approach allows these changes to be made without restarting the entire development process.
- Developing the project in increments enables early testing of both frontend and backend components helping to identify issues and problems in time, hence mitigating risk early on.
- The machine learning components such as clothing classification, color extraction, authentication, and metadata extraction could be developed and tested independently before integration, reducing overall system risk.
- This method allows works well with the academic timelines and documentation requirements for presenting development progress and outcomes to the supervisors.
- Allows for the development of the project to be carried out piece by piece and helps to configure and understand the system better along the way.

3.3 Work Breakdown Structure



Figure 8: WBS

Partitioning the figure for clarity, WBS Part-1:

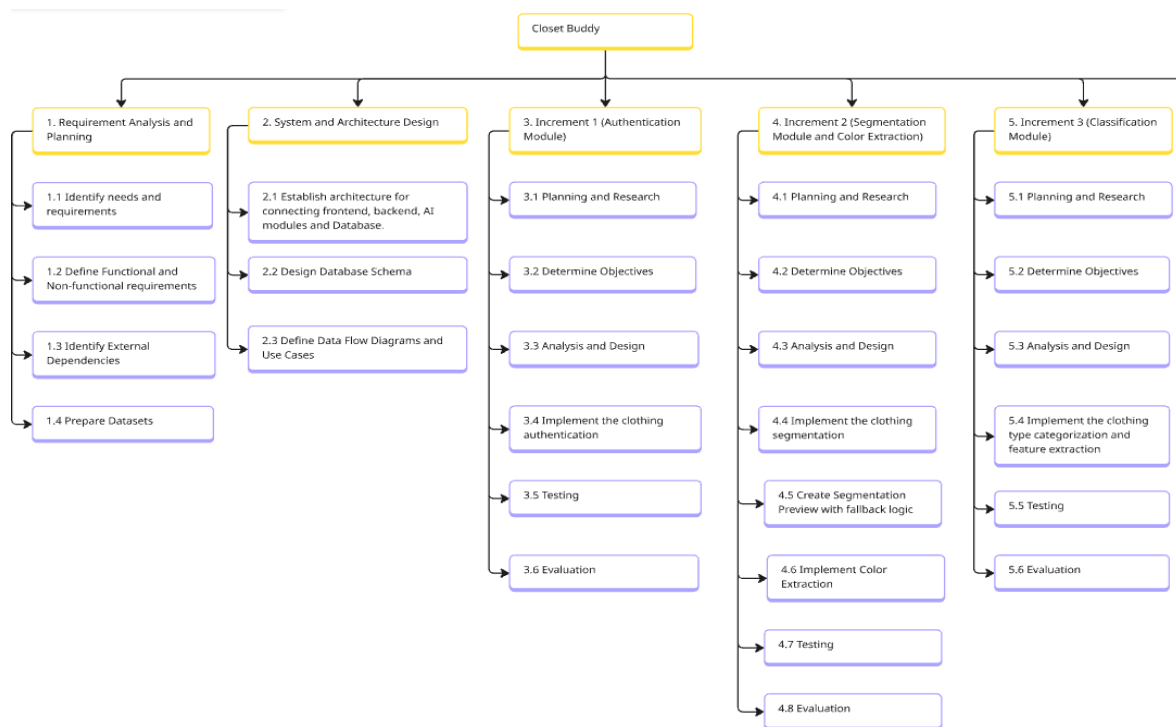


Figure 9: WBS Part-1

WBS Part-2:

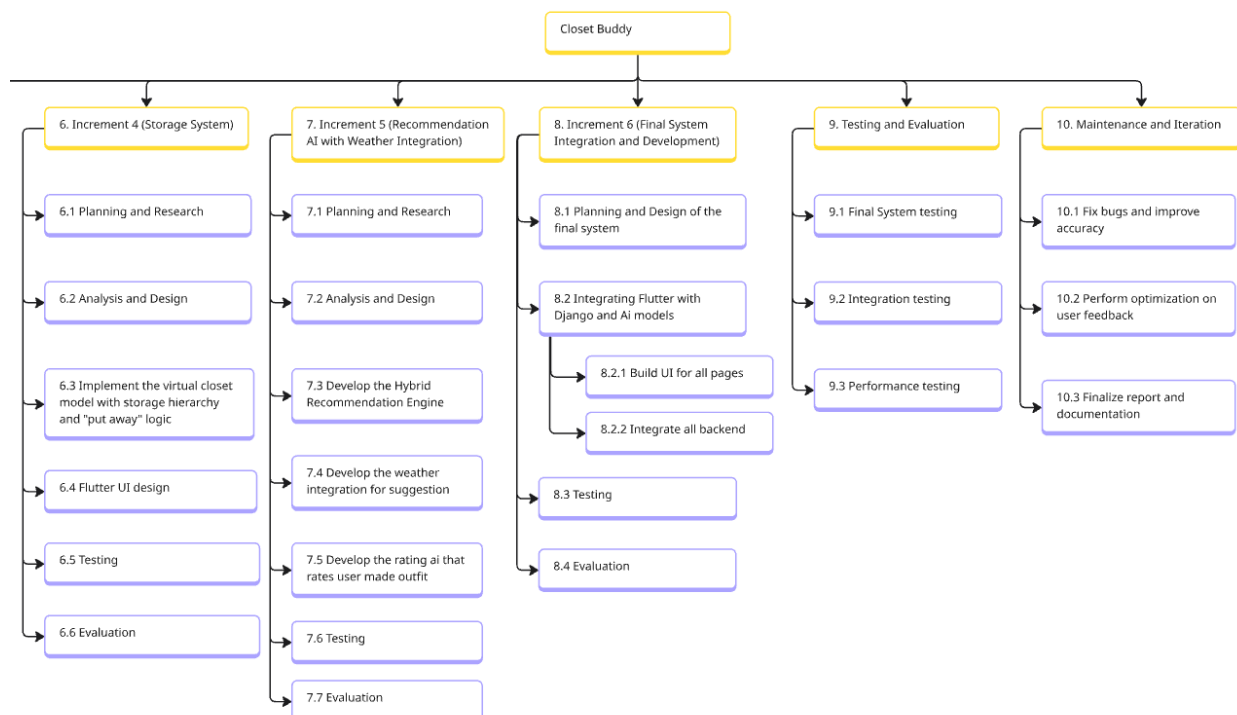


Figure 10: WBS Part-2

3.4 Gantt Chart

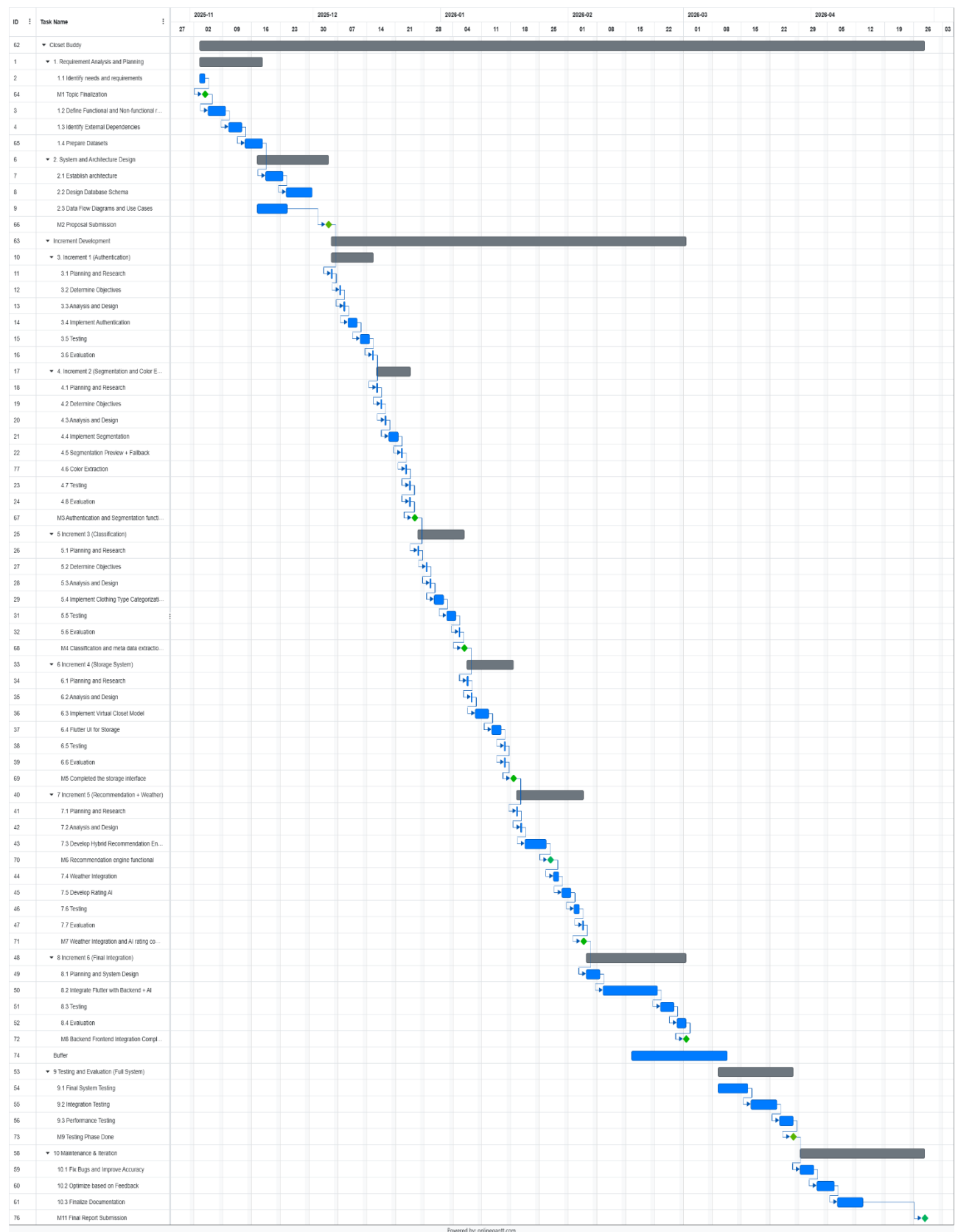


Figure 11: Gantt Chart

The details of the Gantt Chart are given [here](#).

3.5 Resource Requirement

The resource requirements for this project are specified in the [appendix](#).

4. Development Till Date

Till date steady, consistent, and significant progress has been made in the development of Closet Buddy system. The project started by collecting requirements to define AI features through surveys and meetings with supervisors for analysis of user needs. The database and entity relation diagram were designed to support data integrity. To set up the system and architectural blueprint various structural models and UML diagrams were developed including data flow diagrams, use case diagrams, sequence diagrams, and activity diagrams. Moving on to the technical side core functionalities of the system are already taking shape, secure user authentication has been established and AI features like color extraction and segmentation have also been implemented. For an initial blueprint of the design, wireframes and some initial UI have been developed, providing a visual framework.

4.1 Initial Survey

An initial survey was carried out to understand the user's clothing and storage habits as well as to analyze the requirements and justify project features. The initial survey provided insights into people's behavior and dressing habits and gather requirements for the project. The questions and results of this survey are provided in the [appendix](#).

4.2 DFD

DFD or Data Flow Diagram is a graphical tool used to visualize the flow of data within a system that shows data inputs, outputs, data stores, and different processes through which data is transformed (GeeksforGeeks, 2025). It provides a high-level overview of the system functionality and helps with system analysis.

4.2.1 DFD Level 0

The level 0 DFD is a contextual diagram in which the entire system is shown as a single process and it only interacts with the external entities without data stores, only the major inputs and outputs of the system are highlighted in this diagram (GeeksforGeeks, 2025).

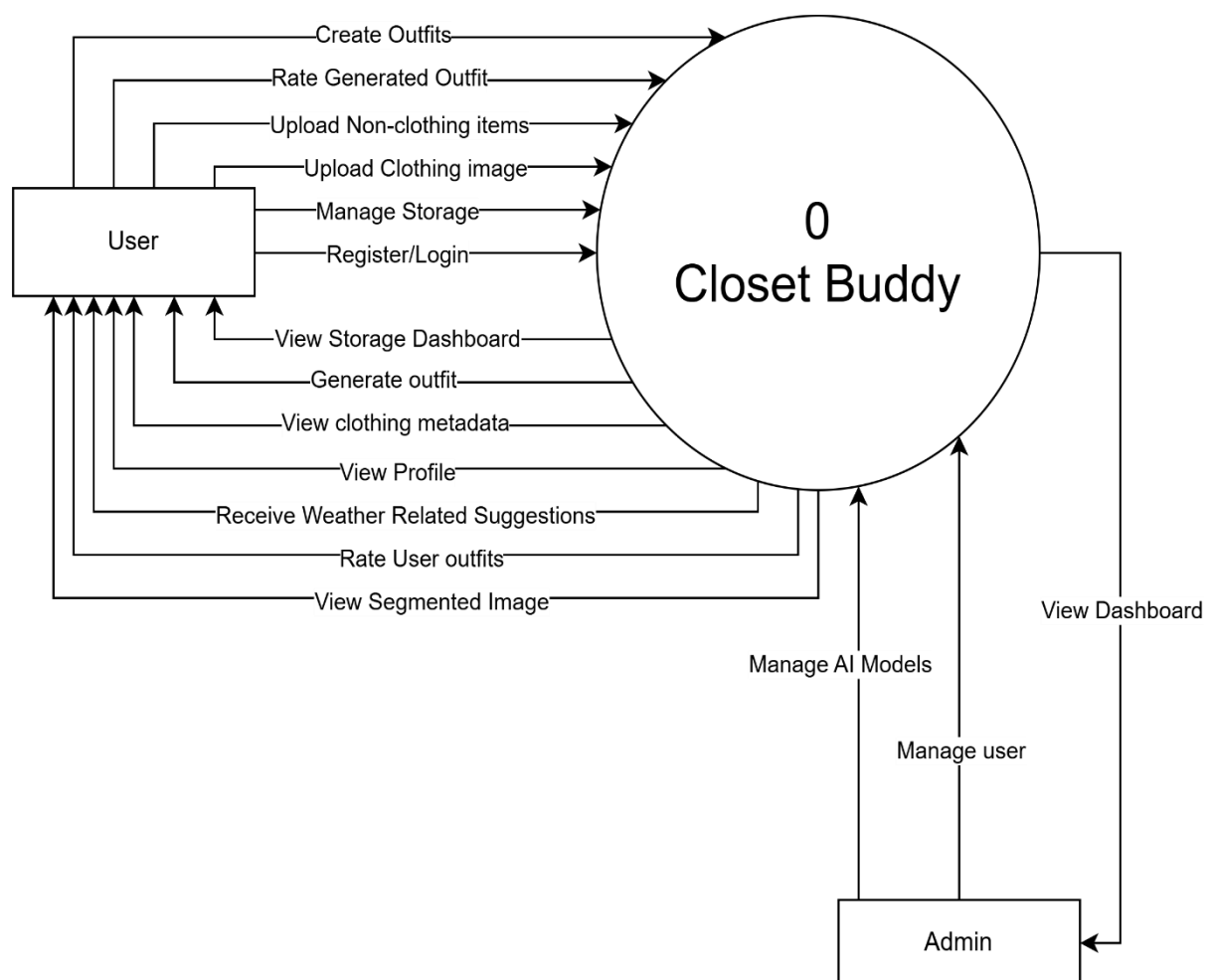


Figure 12: DFD Level 0

4.2.2 DFD Level 1

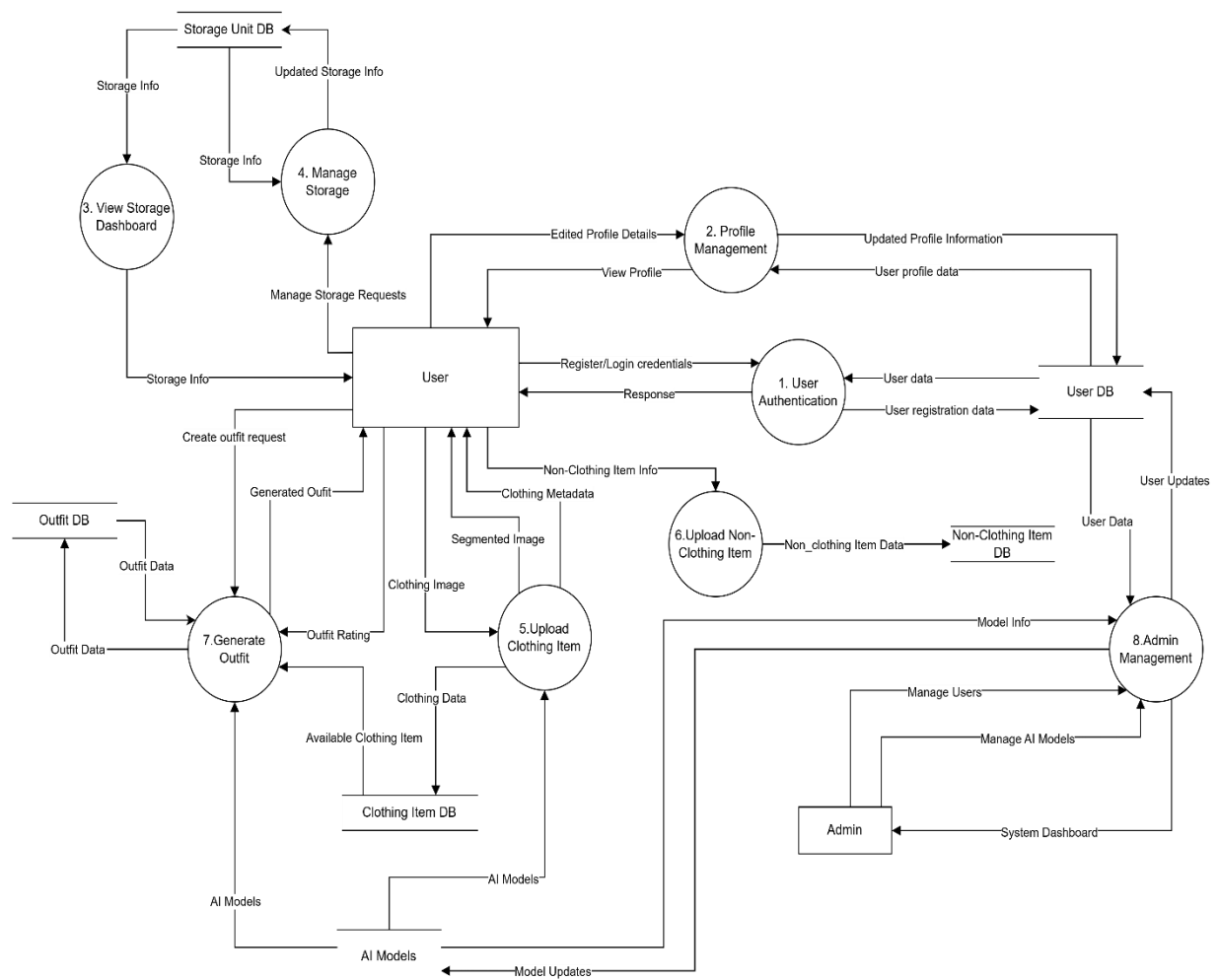
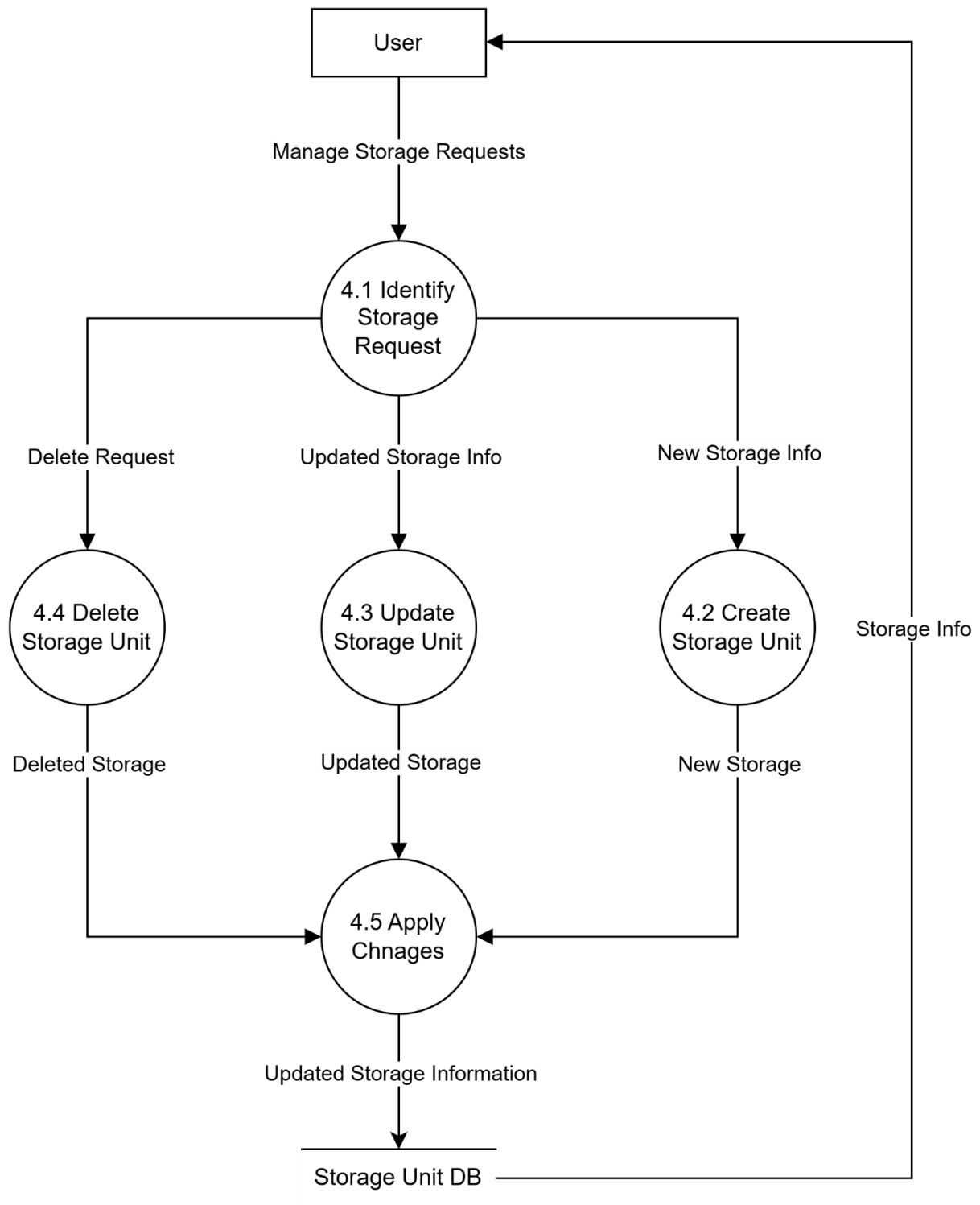


Figure 13: DFD Level 1

4.2.3 DFD Level 2: Manage Storage*Figure 14: DFD Level 2 - Manage Storage*

4.3 System Architecture Diagram

System architecture diagram is a visual blueprint of a software system that shows all of its core components like (data bases, modules, services) and how they connect, interact, and flow (MockFlow , 2025).

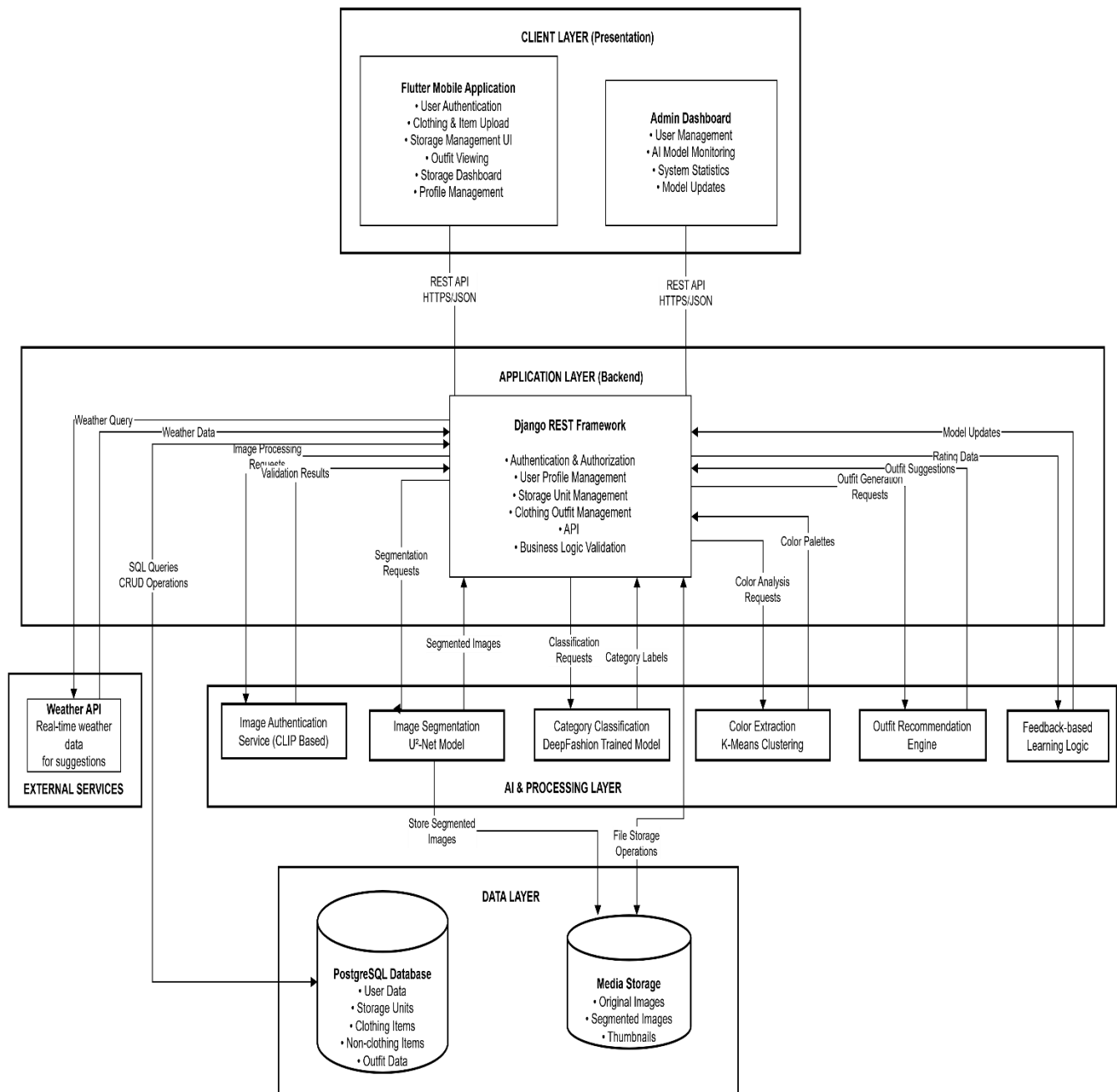


Figure 15: System Architecture Diagram

4.4 Use Case Diagram

Use case diagram is a visual representation of how a user interacts with the system, it shows user engagement with various system functionalities within the system boundary and provides a high-level overview of system's behavior (GeeksforGeeks, 2025).

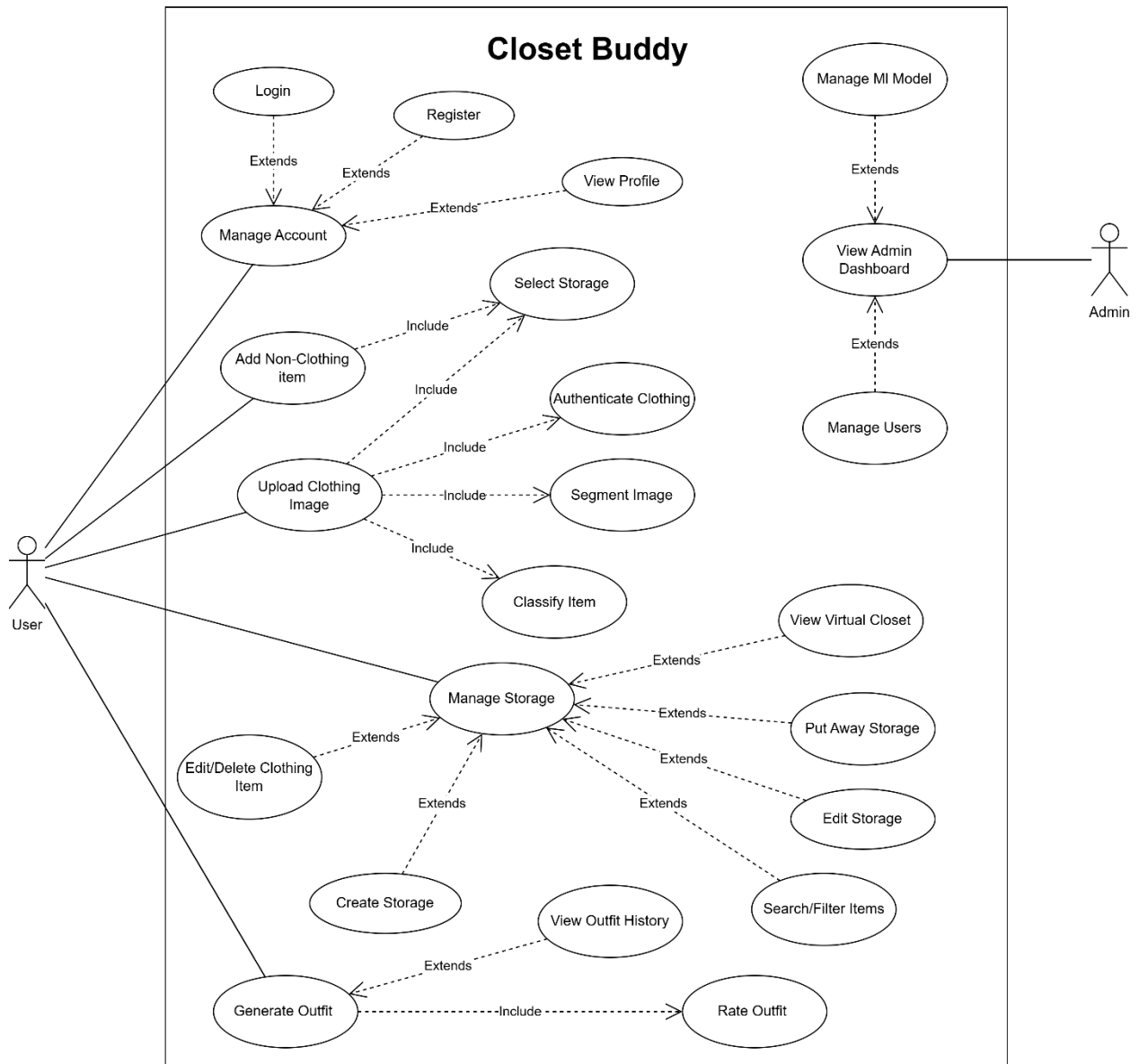


Figure 16: Use Case Diagram

4.4.1 High Level Use Case Description

High level use case descriptions provide a brief overview of the use case showing main objective and actors involved. The high-level descriptions are given:

I. Use Case: Manage Account

Actors: User

Description: Allows user to manage their information and access within the Closet Buddy system. This allows the user to register a new account or log into an existing account. After logging in, the user can view or update their profile information. Access to the system is given only after logging in and authentication is done using Django's authentication framework

II. Use Case: Upload Clothing Item

Actors: User

Description: The users can add a new clothing item to their virtual wardrobe by selecting a storage unit and uploading images. Open uploading an image the system automatically verifies if the image is a valid clothing item, segments the clothing item from the background to make a clean cut out, extracts relevant metadata such as category and color, and stores it in the selected virtual storage unit.

III. Use Case: Add Non-Clothing Item

Actors: User

Description: This use-case allows adding non-clothing items such as accessories or miscellaneous items into the virtual storage spaces. Users can select a virtual space and simply add the name and description for the non-clothing item to store it. This ensures that the user's physical storage spaces are digitally represented.

IV. Use Case: Manage Storage

Actors: User

Description: This use case allows the users to organize their virtual storage spaces and the items in it. Users can create, edit, and view virtual storage units such as closets, drawers, or boxes. The users can mark storage spaces as "put away" to exclude the clothing items from outfit generation and search and filtered items within the storage units.

V. Use Case: Generate Outfit

Actors: User

Description: Enables the system to generate outfit recommendations based on user's available clothing items and their requirements. The users can give ratings as feedback to the generated outfits and view their outfit history.

VI. Use Case: View Admin Dashboard

Actors: Admin

Description: Provides the system administrators with the tools to manage user accounts, monitor system activity and analytics, and manage AI models used for clothing recognition and recommendation. This ensures system stability and model maintenance.

4.1.2 Expanded Use Case Description

View expanded use case description in [appendix](#).

4.5 ERD

ERD or Entity Relation Diagram is a conceptual model of the system's database which presents the logical structure of the database with its entities, attributes, and relationship between them including cardinality and modality (Lucidchart, 2025).

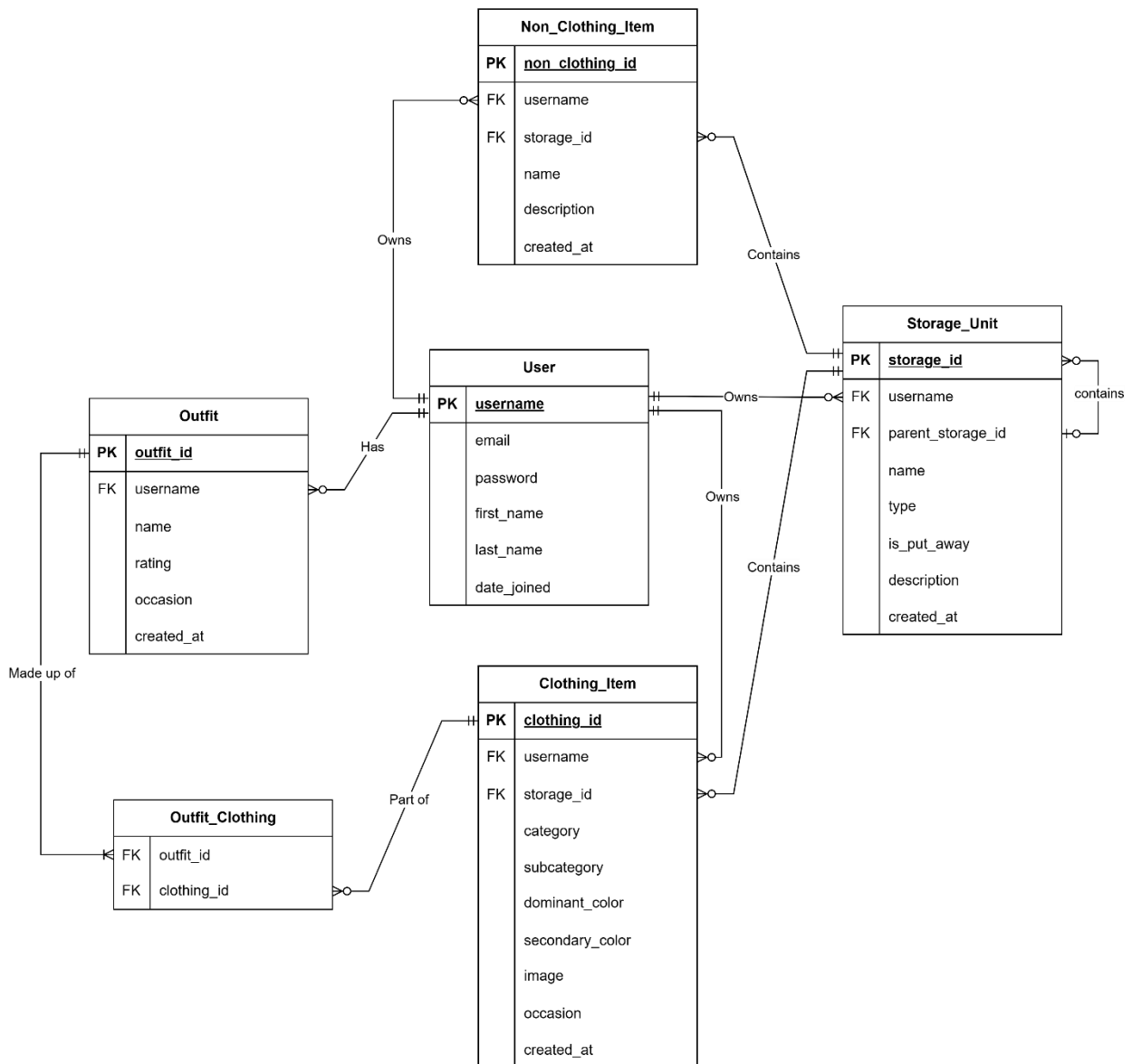


Figure 17: ERD

4.6 SRS

The SRS of the proposed project is provided in the [appendix](#).

4.7 Sequence Diagram

Sequence diagram is a diagram used to visualize the sequential order of interaction between objects in a system, it focuses on how objects interact and communicate with each other over time (GeeksforGeeks, 2025).

4.7.1 Sequence Diagram: Manage Account

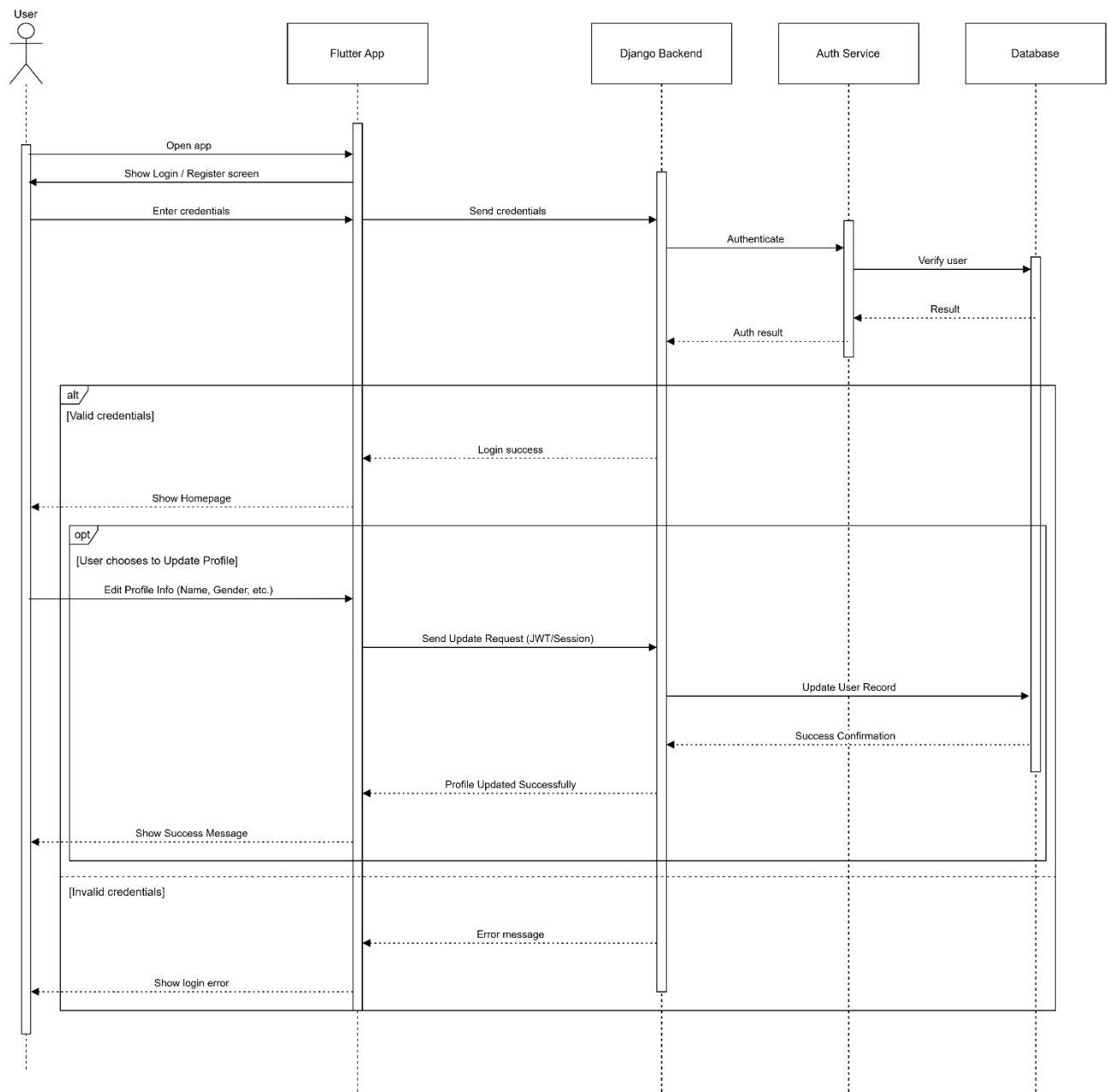


Figure 18: Manage Account Sequence Diagram

4.7.2 Sequence Diagram: Upload Clothing Item

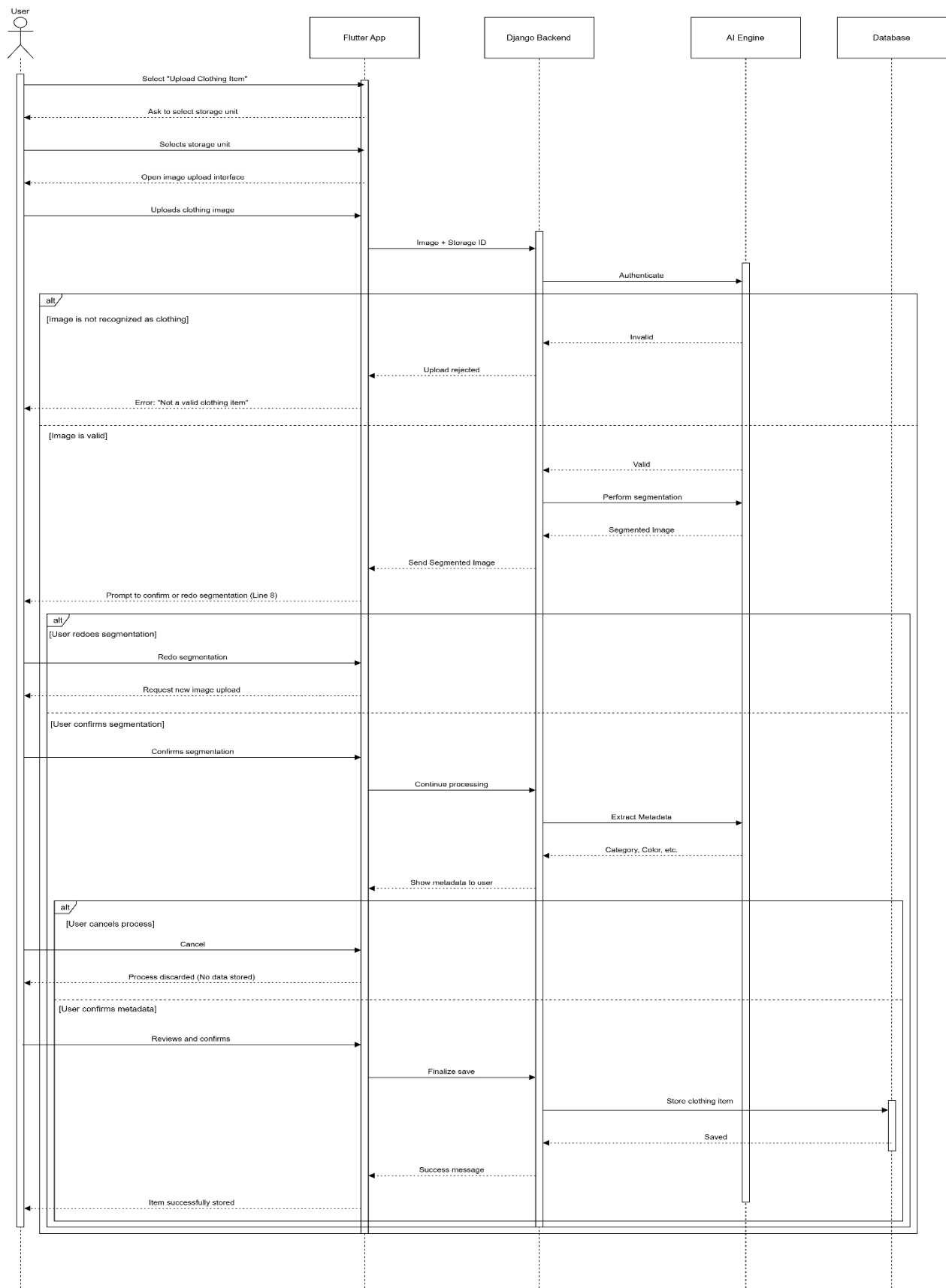


Figure 19: Upload Clothing Item Sequence Diagram

4.7.3 Sequence Diagram: Manage Storage

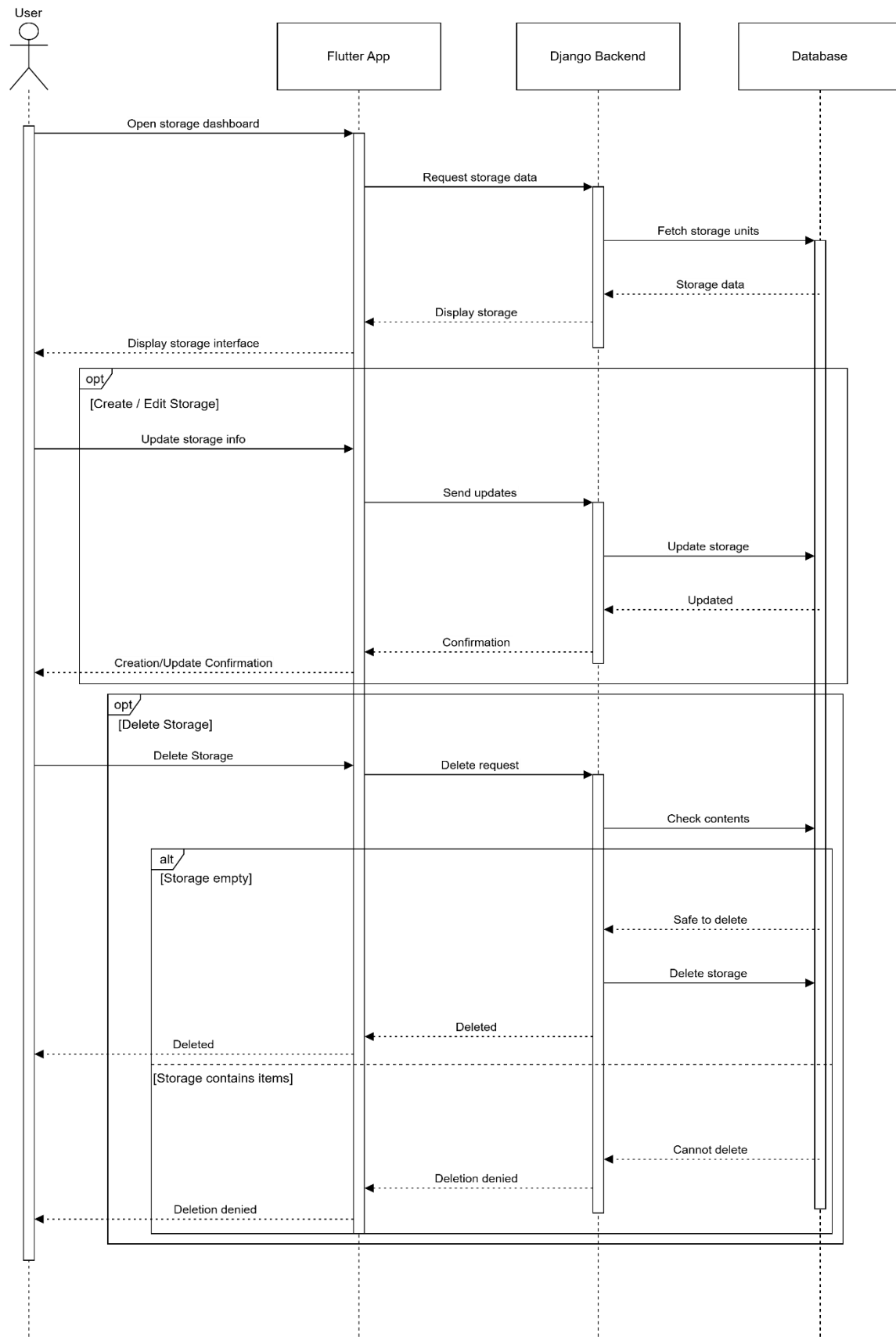


Figure 20: Manage Storage Sequence Diagram

4.7.4 Sequence Diagram: Generate Outfit

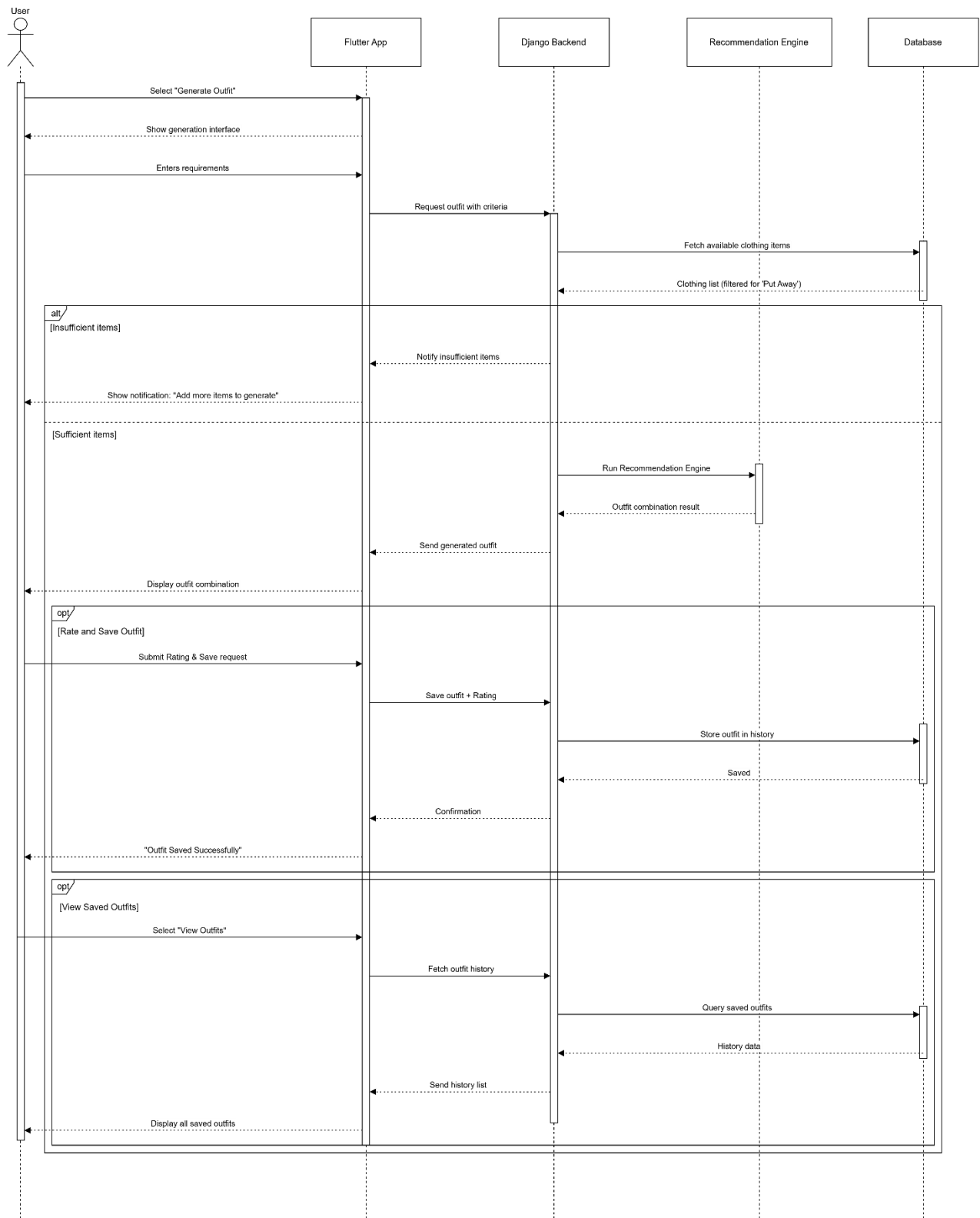


Figure 21: Generate Outfit Sequence Diagram

4.7.5 Sequence Diagram: Add Non-Clothing Item

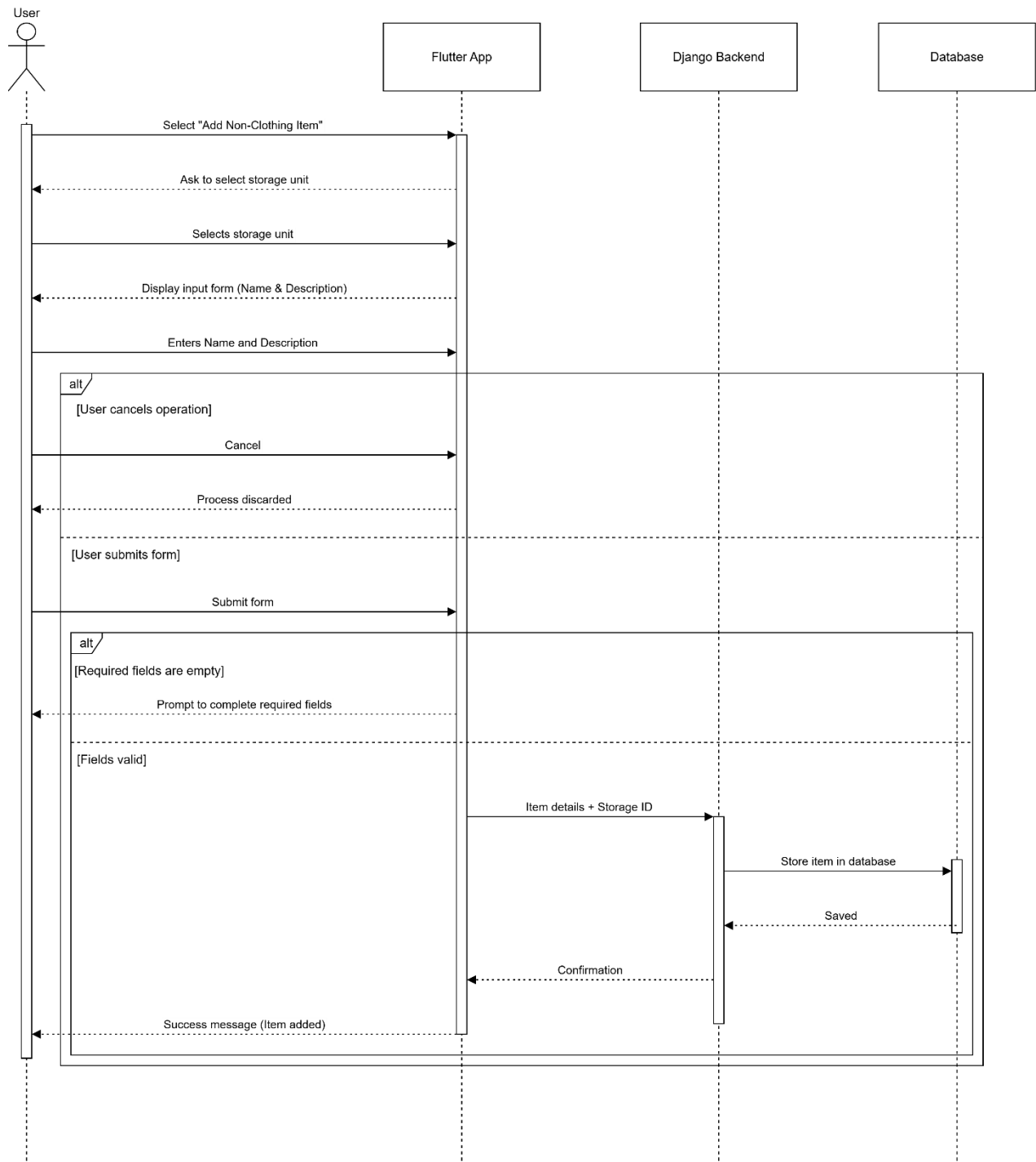


Figure 22: Add Non-Clothing Item Sequence Diagram

4.7.6 Sequence Diagram: View Admin Dashboard

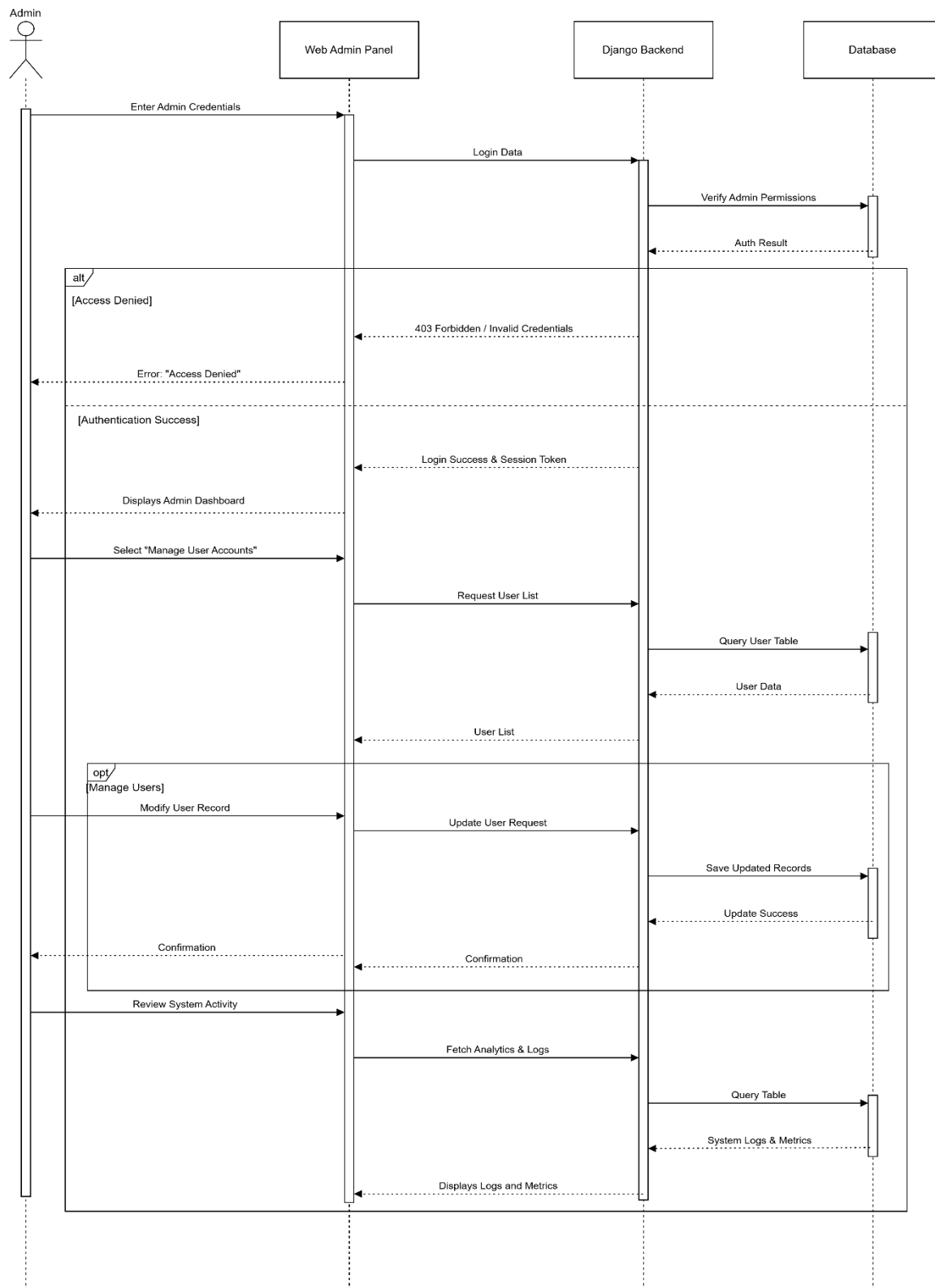


Figure 23: View Admin Dashboard Sequence Diagram

4.8 Flowchart Diagram

Flow chart is a diagrammatic representation of data, algorithms, processes or tasks giving a visual approach to understanding how the code works, it provides a step-by-step solution to a problem (GeeksforGeeks, 2025).

4.8.1 Flowchart Diagram: Manage Account

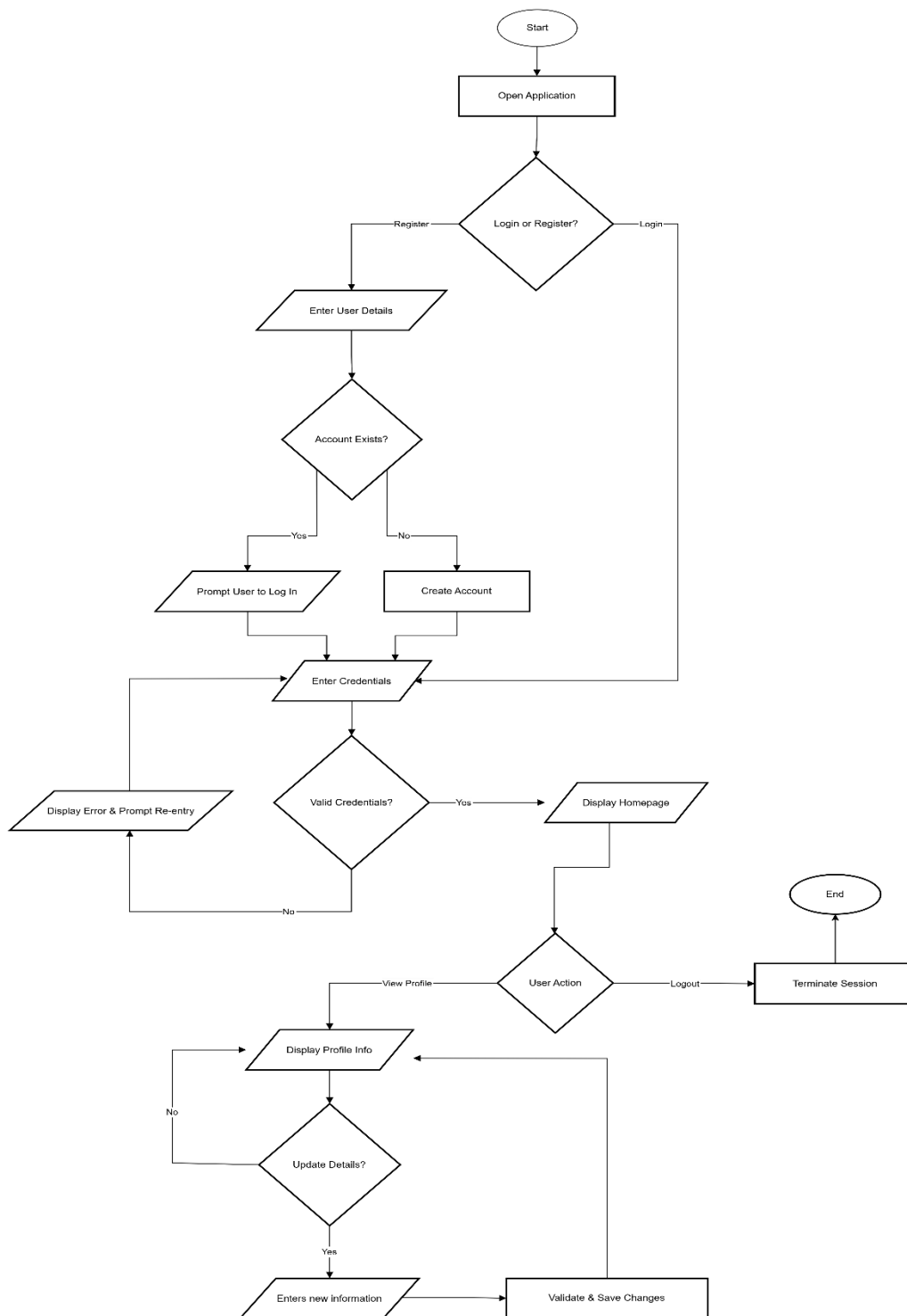


Figure 24: Manage Account Flowchart Diagram

4.8.2 Flowchart Diagram: Upload Clothing Item

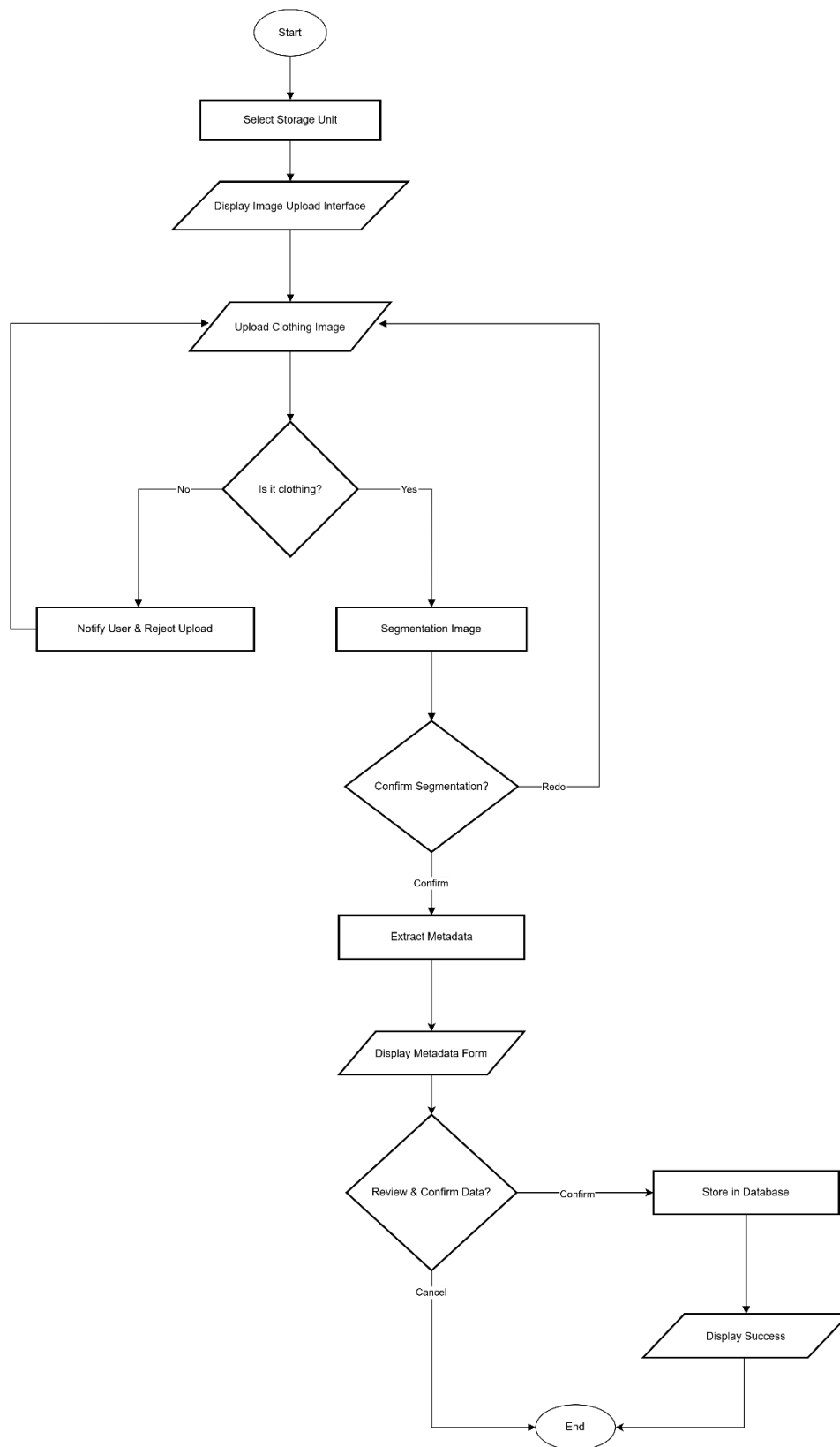


Figure 25: Upload Clothing Item Flowchart Diagram

4.8.3 Flowchart Diagram: Add Non-Clothing Item

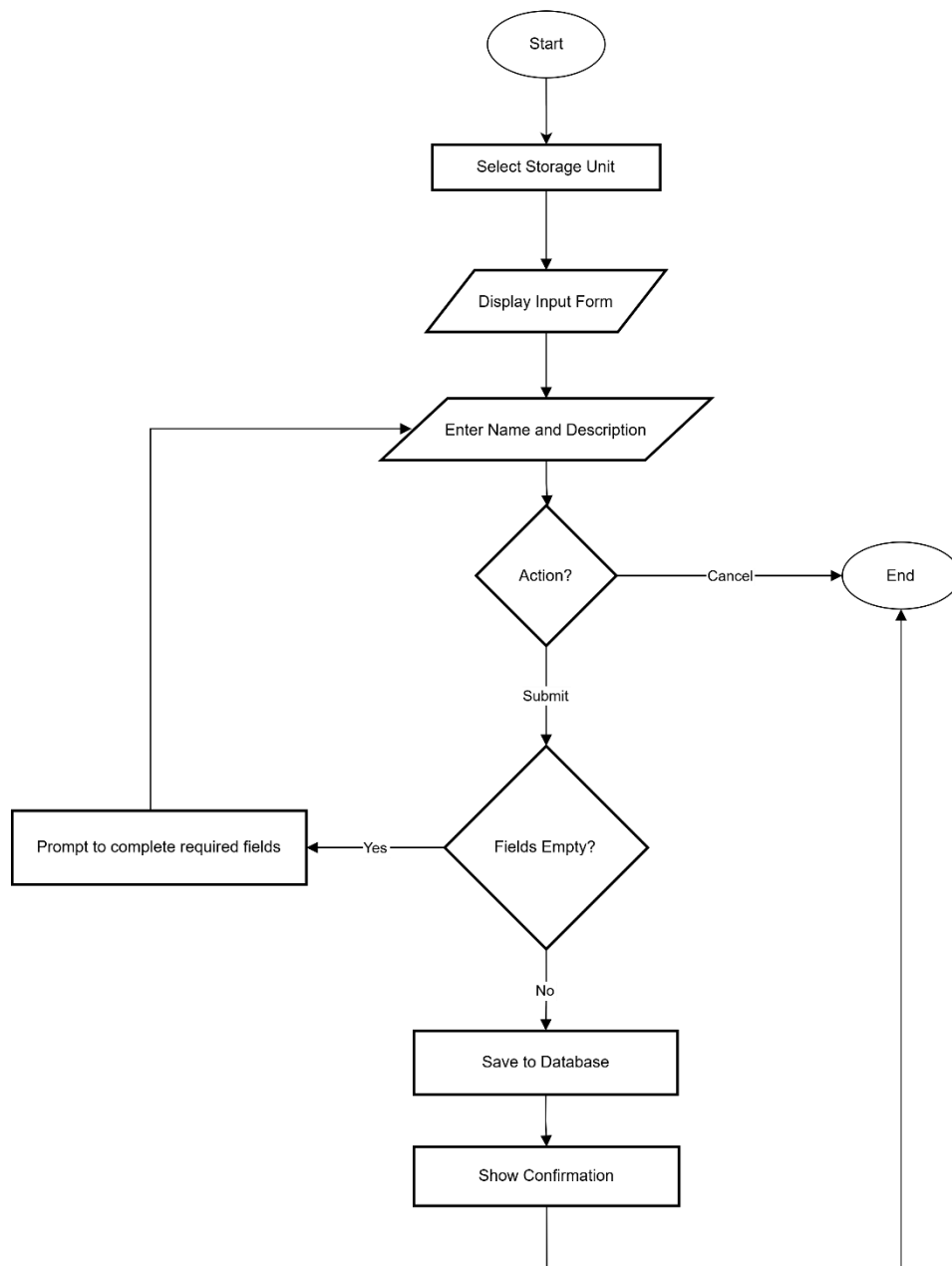


Figure 26: Add Non-Clothing Item Flowchart Diagram

4.8.4 Flowchart Diagram: Manage Storage

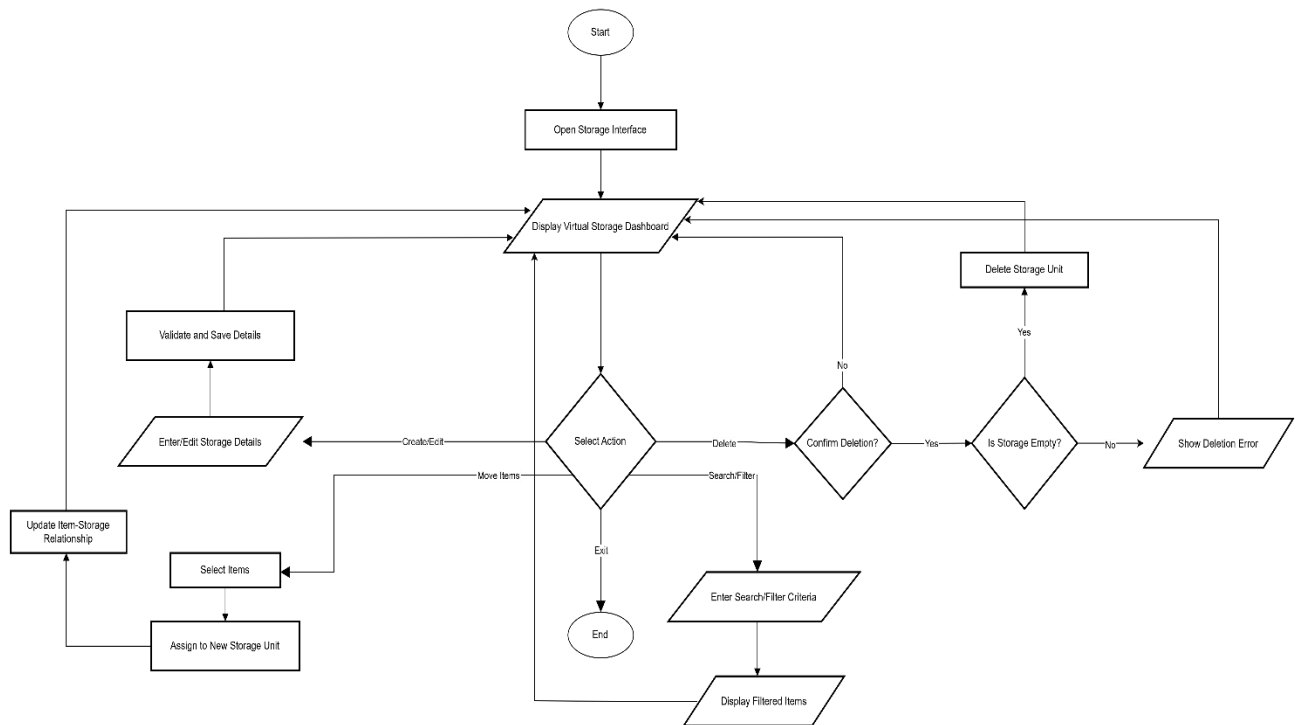


Figure 27: Manage Storage Flowchart Diagram

4.8.5 Flowchart Diagram: View Admin Dashboard

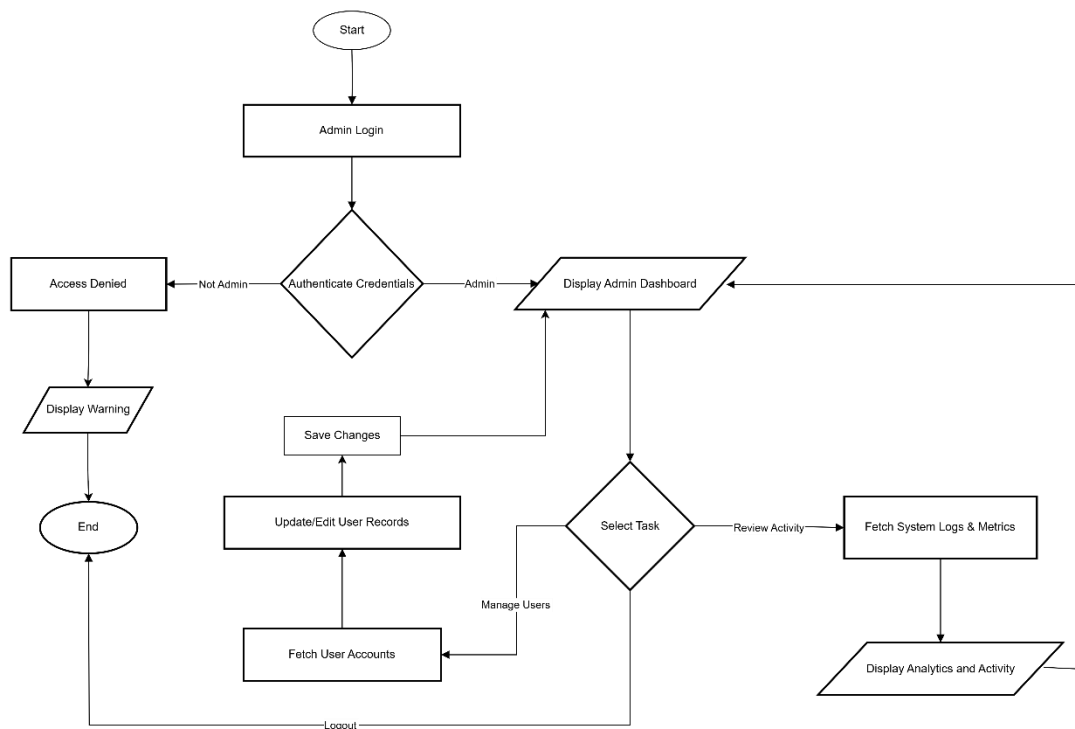


Figure 28: View Admin Dashboard Flowchart Diagram

4.8.6 Flowchart Diagram: Generate Outfit

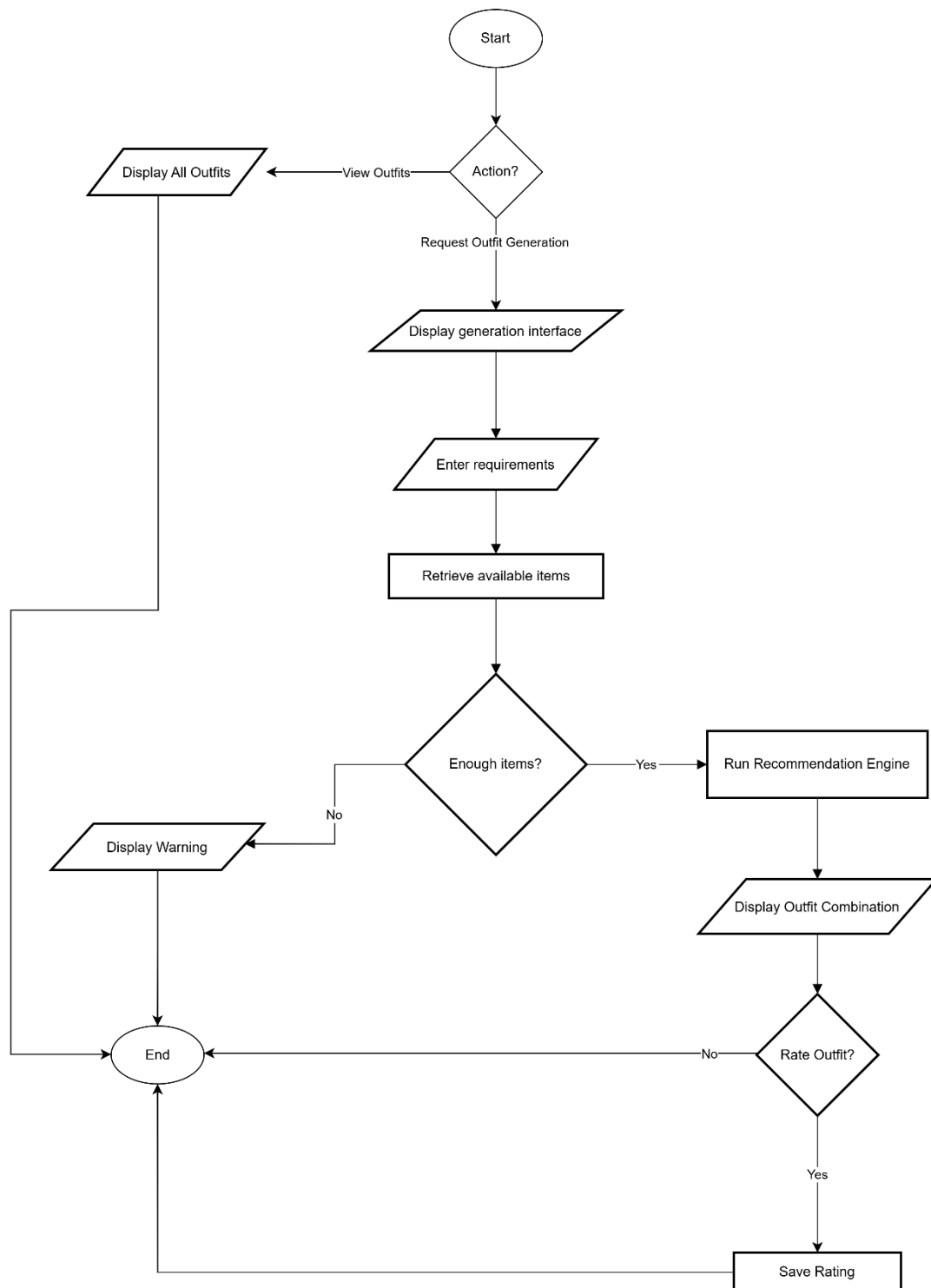


Figure 29: Generate Outfit Flowchart Diagram

4.9 Wireframe Design

Wireframe is a basic rough sketch for developing a website, app, or any interface, focusing solely on the structure, layout, and functionality, and not on visual designs or user appeal. The wireframes made till now for this project are shown in the appendix.

4.10 Development

In this section the development progressed till now will be presented. The work that has been done till now can be separated into 3 main parts front-end, back-end, and AI development. The focus was placed on establishing systems semi-core functionalities and major back-end services and some essential components required for processing clothing images. While the system is not fully developed major groundworks have already been laid.

4.10.1 Frontend Development

The frontend development was not the main objective for this phase so only the parts required for supporting the developed backend have been integrated, but some other pages are also developed without backend integration. The frontend development was done in Flutter entirely using Dart Programming language.

Development completed so far:

- The loading page was developed for the initial system startup.
- Login and registration pages were made to support user authentication and access to the system.
- Implemented the upload item page where user can select a storage space and decide whether to store clothing or non-clothing item in it.
- Developed the profile page where users can view their data and edit.

Evidence: appendix.

4.10.2 Backend Development

In the backend development, implemented some core services like user authentication and login as well as features like clothing segmentation and color extraction.

Development completed so far:

- Completed the user authentication feature including user registration and login
- User credentials are handled securely using Django's authentication framework
- Implemented session-based authentication
- Clothing image upload, image segmentation, and color extraction were also implemented.

Evidence: [appendix](#).

4.10.3 AI Development

AI development was the main focus for this phase, especially for image processing features. Two models were used and implemented for image segmentation and color extraction. The world's completed so far are

- Segmentation Model: an image segmentation model was implemented to separate the clothing items from background and produce a clean cut out. For this the rembg library's remove function was used which uses the U²-Net, a deep-learning based semantic segmentation model.
- Color Extraction: For extracting meta data from the clothing item, dominant color and secondary color were to be extracted and for this purpose unsupervised machine learning through K-means clustering was utilized to cluster the pixels.

Evidence: [appendix](#).

4.10.4 Database Configuration and Model Creation

This section will discuss the process involved in setting up the PostgreSQL database, creating models in Django, and visualizing the tables in pgAdmin.

Django configuration:

The database is connected in the backend through configuring settings.py file.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'closet_buddy',  
        'USER': 'postgres',  
        'PASSWORD': '2005',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

Figure 30: Django settings configuration

Creating Models in Django:

The models to create the structure of the database are created in the models.py file of backend app. After configuring the database and creating models in the backend, migrations are made to create those tables in PostgreSQL. The evidence can be seen in the [appendix](#).

4.11 UI Design

The screenshots of the UI designs are in the [appendix](#) section.

4.12 Project Management and Version Control

Trello was used for managing the project, and GitHub was used for version control. Further details are kept in [appendix](#).

5. Progress Analysis

This section discusses the analysis of progress that has been made for Closet Buddy project up to the current stage of development. Completed work is evaluated against the planned schedule for analysis. The project is developed following an iterative incremental methodology where the functionalities are developed and validated in successive increments and improved in iterations.

5.1 Progress Review

The project development officially started in November 2025 after getting approval from the supervisors on the project topic, requirements, scope, feasibility, and preliminary idea. The start of development was approved after the pre-proposal submission and developing a clear idea of the project features. For the development of the project, iterative incremental approach was selected where the project is divided into logical increments where each increment delivers a functional part of the project and is improved through feedback iteratively overtime. Following this approach, allows validation of its features and components such as user authentication, segmentation, and feature extraction before integrating into the final product.

In this phase, more emphasis was given for requirement analysis and gathering, understanding the scope of the project, system design, architecture planning, and resource selection for establishing a good foundation for development. After completing the initial steps, moving on to incremental development of features Increment 1 focused on implementing user authentication and Increment 2 focused on clothing segmentation and color extraction through AI. Until now these two increments have been successfully completed and the progress of the project development can be considered to be a little over **40%** completed. The steps and increments completed are discussed:

5.1.1 Requirement Analysis and Planning

From November 2, 2025 to November 16, 2025

- Identified the needs, requirements, and necessary features for the project.
- Finalized the project topic and basic workflow with scope.
- Defined the functional and non-functional requirements.
- Identified the different external dependencies that may be required.
- Data set to be used in the project for AI model training was also found.
- Got approval for the project development methodology.

5.1.2 System and Architecture Design

From November 16, 2025 to December 3, 2025

- Established the system architecture and flow.
- Designed the database schema and ERD.
- Configured the PostgreSQL database and created models in Django.
- The data flow diagrams and use cases were made.
- Some basic wireframes were also designed.
- The proposal for this project was submitted to supervisors.

5.1.3 Increment 1: User Authentication

From December 4, 2025 to December 17, 2025

- Set up the Django backend and Flutter frontend project structure and dependencies.
- Conducted research on Django's default user authentication by JWT.
- Determined the objectives for user authentication and registration.
- Designed the login and registration pages.
- Implemented user registration, login, and authentication using Django's built in authentication framework.
- Separated roles for users and admins.
- Implemented session management and password hashing mechanism.

5.1.4 Increment 2: Segmentation and Color Extraction

From December 17, 2025 to December 30, 2025

- Conducted research on image segmentation by U²-Net and color extraction through K-Means Clustering.
- Established objectives for segmentation and color extraction.
- Designed the basic workflow for segmentation and color extraction.
- Implemented segmentation using rembg library of Python.
- Implemented the segmentation preview with the user choice confirmation.
- Implemented color extraction to segmented image using K-Means clustering.
- Connected the models to the backend with a simple frontend interface.
- Started on the interim report.

5.1.4 Increment 3: Classification (Initial)

From January 1, 2026

- Conducted basic research for implementing classification models through finetuning and inference.
- Determined the objectives.

5.2 Progress Table

ID	Task Name	Expected Completion	Actual Completion	Status	Progress (%)
1	Requirement Analysis and Planning	Nov 16, 2025	Nov 16, 2025	Complete	100
1.1	Identify needs and problem scope	Nov 04, 2025	Nov 03, 2025	Complete	100
1.2	Define functional requirements	Nov 09, 2025	Nov 08, 2025	Complete	100
1.3	Define non-functional requirements	Nov 09, 2025	Nov 09, 2025	Complete	100
1.4	Identify external dependencies	Nov 13, 2025	Nov 12, 2025	Complete	100
1.5	Dataset identification and preparation	Nov 16, 2025	Nov 14, 2025	Complete	100
2	System and Architecture Design	Dec 03, 2025	Dec 03, 2025	Complete	100
2.1	Define overall system architecture	Nov 20, 2025	Nov 19, 2025	Complete	100
2.2	Design database schema	Nov 26, 2025	Nov 22, 2025	Complete	100
2.3	Create DFDs (Level 0–2)	Nov 22, 2025	Nov 23, 2025	Complete	100
2.4	Develop use case and UML diagrams	Nov 22, 2025	Nov 28, 2025	Complete	100
2.5	Proposal Submission	Dec 3, 2025	Dec 3, 2025	Complete	100
3	Increment 1 – Authentication	Dec 19, 2025	Dec 17, 2025	Complete	100
3.1	Authentication design and planning	Dec 09, 2025	Dec 05, 2025	Complete	100
3.2	Implement user registration	Dec 13, 2025	Dec 10, 2025	Complete	100

3.3	Implement user login and session handling	Dec 13, 2025	Dec 12, 2025	Complete	100
3.4	Authentication testing and validation	Dec 17, 2025	Dec 15, 2025	Complete	100
4	Increment 2 – Segmentation and Color Extraction	Dec 30, 2025	Dec 30, 2025	Complete	100
4.1	Clothing image authentication	Dec 20, 2025	Dec 24, 2025	Complete	100
4.2	Implement clothing segmentation model	Dec 22, 2025	Dec 20, 2025	Complete	100
4.3	Segmentation preview and redo fallback	Dec 23, 2025	Dec 22, 2025	Complete	100
4.4	Implement color extraction (K-means)	Dec 25, 2025	Dec 24, 2025	Complete	100
4.5	Testing and evaluation	Dec 29, 2025	Dec 28, 2025	Complete	100
4.6	Interim Report Submission	Dec 30, 2025	Dec 30, 2025	Complete	100
5	Increment 3 – Classification	Jan 15, 2026	-	In Progress	25
5.1	Research and model selection	Jan 02, 2026	Dec 28, 2025	Complete	100
5.2	Implement clothing category classification	Jan 09, 2026	-	Incomplete	0
5.3	Metadata extraction integration	Jan 14, 2026	-	Incomplete	0
5.4	Testing and evaluation	Jan 15, 2026	-	Incomplete	0
6	Increment 4 – Storage System	Jan 29, 2026	-	In Progress	25
6.1	Storage system design	Jan 19, 2026	Dec 20, 2025	Complete	100
6.2	Implement virtual storage models	Jan 22, 2026	-	In Progress	75

6.3	Flutter UI for storage management	Jan 27, 2026	-	Incomplete	0
6.4	Storage system testing	Jan 28, 2026	-	Incomplete	0
7	Increment 5 – Recommendation and Weather	Feb 19, 2026	-	Incomplete	0
7.1	Design recommendation logic	Feb 02, 2026	-	Incomplete	0
7.2	Implement hybrid recommendation engine	Feb 09, 2026	-	Incomplete	0
7.3	Integrate weather-based tips	Feb 12, 2026	-	Incomplete	0
7.4	Implement outfit rating logic	Feb 16, 2026	-	Incomplete	0
7.5	Testing and evaluation	Feb 19, 2026	-	Incomplete	0
8	Increment 6 – Final Integration	Mar 20, 2026	-	In Progress	15
8.1	System integration planning	Feb 24, 2026	-	Incomplete	0
8.2	Integrate Flutter frontend with backend	Mar 12, 2026	-	In Progress	35
8.3	Integrate AI services	Mar 12, 2026	-	Incomplete	0
8.4	Integration testing and evaluation	Mar 20, 2026	-	Incomplete	0
9	Testing and Evaluation (Full System)	Mar 31, 2026	-	Incomplete	0
9.1	System testing	Mar 17, 2026	-	Incomplete	0
9.2	Integration testing	Mar 25, 2026	-	Incomplete	0
9.3	Performance and reliability testing	Mar 31, 2026	-	Incomplete	0

10	Maintenance and Iteration	Apr 21, 2026	-	Incomplete	0
10.1	Bug fixing and refinement	Apr 06, 2026	-	Incomplete	0
10.2	Optimization based on feedback	Apr 10, 2026	-	Incomplete	0
10.3	Final documentation and report submission	Apr 21, 2026	-	Incomplete	0

5.3 Action Plan

Moving forward from this phase the main priority would be to complete clothing metadata extraction through AI, managing the wardrobe interface, and developing the hybrid AI for outfit recommendation in that order. Till now a little over 40% of the project development has been completed, including Clothing segmentation all our extraction and user authentication. The remaining work will focus on core functionalities incrementally after which a system wide integration, testing and final evaluation will be done. Finally, the full system integration including frontend, backend, and AI components will be done to make a full functioning system.

6. Future Work

Although some of core components have been completed still a large portion of the system is yet to be implemented. The future work Where is the song what's left and its planned timeline.

6.1 Backend Development

The backend currently supports user authentication, clothing segmentation, and color extraction only. The future development will focus on implementing remaining business logic, storage management, and recommendation flows.

Table 2: Backend Future Works

Task	Planned Timeline	Description
Clothing category classification	January 1, 2025 – January 15, 2025	Training a custom deep fashion-based AI model to classify clothing category and subcategory.
Virtual storage interface and management logic	January 15, 2025 – January 29, 2025	Implementing CRUD operations for storage units
Non-clothing item support	January 1, 2025 – January 10, 2025	Backend logic for storing and managing non-clothing items in a storage space
Outfit recommendation logic	February 1, 2025 – February 19, 2025	Implementing a rule based and AI driven outfits recommendation logic using available clothing metadata.
Ratings are Outfit History	February 20, 2025 – February 28, 2025	Store the generated outfits and track user rating.

6.2 Frontend Development

A lot of the frontend development has already been completed including loading, login, restriction, home, adding clothing, adding non-clothing, adding items, and segmentation screen have already been made. The remaining work on the development focuses on completing the remaining interfaces and UI refinement.

Table 3: Frontend future works

Task	Planned Timeline	Description
UI development	January 1, 2026 – February, 2026	Designing and implementing the remaining phases for the project
UI refinement	January 2026 – February, 2026	Refining the pages which have already been made.

6.3 AI Development

In this phase only the AI models for image segmentation and color extraction have been made. Remaining work for AI development will include clothing metadata extraction from classification, occasion classification, and outfit recommendation AI.

Table 4: AI Development future works

Task	Planned Timeline	Description
Developing clothing category classification model	January 1, 2026 – January 15, 2026	Fine tuning and validating a clothing category classifier using deep fashion 50-category data set.
Occasion classification model integration	January 2, 2026 – January 15, 2026	Integrating the pretrained usage classification model to extract clothing occasion.
Hybrid outfit recommendation logic	January 30, 2026 – February 20, 2026	Combining rule-based logic with AI for suggesting outfit generation.
Refining the clothing authentication AI	January 1, 2026 – January 10, 2026	Improving the CLIP-based clothing authenticator

6.4 System Integration and Testing

Once the individual features and modules are completed, system level integration and testing are to be carried out to ensure system functionality.

Table 5: System Integration future works

Task	Planned Timeline	Description
Frontend and Backend Integration	March 01, 2026 – March 20, 2026	Connecting all frontend interfaces with their respective backend.
Functional and Integration Testing	March 10, 2026 – March 30, 2026	Testing features and integration of the system.
Performance Refinement	April 1, 2026 – April 7, 2026	Improve the overall user experience and system performance.

6.5 Finalization and Documentation

This final phase will focus on preparing the system and documentation for submission and evaluation.

Table 6: Finalization work

Task	Planned Timeline	Description
Review and Adjustments	After final integration	Final system verification of functionalities and some refinements.
Report completion	Alongside system review and refinements.	Completing the final project report and documentation.

6.6 Expected Future Work

After completing all the proposed system features and functionalities, if there's still time for development the following features can be added to the system:

- Developing a more advanced deep learning model for clothing classification.
- Adding a calendar API for calendar-based outfit planning.
- Deploying the system in cloud as a full-fledged mobile application.
- Expanding the recommendation logic with user feedback and weather conditions.
- Enabling the users to create their own outfits and receive AI rating.

7. Conclusion

This interim report details and describes the design and development process involved in the making of Closet Buddy application, which is an AI-powered virtual wardrobe and storage management system focused on helping the users to organize their storage spaces and generate outfits. The system tries to address common problems like disorganization of wardrobe, outfit selection difficulty, and poor use of the available clothing by integrating the mobile applications development with the artificial intelligence.

For the development of this project a structured iterative incremental development approach was followed. The core functionalities completed in this development phase are user authentication, clothing image upload, image segmentation, and color extraction for creating the foundation of the project development. The system was designed by integrating a scalable backend in Django, user friendly Flutter-based frontend, and AI models for segmentation and color extraction. Following this architecture approach ensures that the project is flexible, maintainable, and can be extended in the future if need be, to integrate more functionalities.

In this report the background research on the end users, understanding what the project is about and what it does, and research on similar projects were carried out. Various diagrams like system architecture diagram, flow charts, sequence diagrams, use case diagrams, data flow diagrams, ERD, work breakdown structure, Gantt chart, were made to describe the architecture and design of the system. Segmentation and classification as well as rule-based and AI-enhanced logic of outfit suggestions show the evidence of the potential machine learning usage in the daily lifestyle management. Also, the virtual storage system can introduce the user to a virtually mirrored physical wardrobe, enhancing its accessibility and convenience.

The proposed solution is feasible and effective, though there are still a number of more advanced features that need to be developed, which are tested by the current implementation. Overall, the given project can be said to have achieved its main goals and offers a solid base to a smart and user-friendly wardrobe management system, illustrating the opportunities of AI-based solutions in individual fashion and life-related use.

8. References

Bang, H. & S. J., 2022. Who Uses Virtual Wardrobes? Investigating the Role of Consumer Traits in the Intention to Adopt Virtual Wardrobes. January, p. 16.

GeeksforGeeks, 2025. *Activity Diagrams - Unified Modeling Language (UML)*. [Online]

Available at: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-activity-diagrams/>

[Accessed 27 12 2025].

GeeksforGeeks, 2025. *Iterative Waterfall Model - Software Engineering*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering/software-engineering-iterative-waterfall-model/>

[Accessed 25 12 2025].

GeeksforGeeks, 2025. *Sequence Diagrams - Unified Modeling Language (UML)*. [Online]

Available at: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-sequence-diagrams/>

[Accessed 27 12 2025].

GeeksforGeeks, 2025. *Use Case Diagram - Unified Modeling Language (UML)*. [Online]

Available at: <https://www.geeksforgeeks.org/system-design/use-case-diagram/>

[Accessed 26 12 2025].

GeeksforGeeks, 2025. *What is a Flowchart and its Types?*. [Online]

Available at: <https://www.geeksforgeeks.org/computer-science-fundamentals/what-is-a-flowchart-and-its-types/>

[Accessed 27 12 2025].

GeeksforGeeks, 2025. *What is DFD (Data Flow Diagram)*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering/what-is-dfddata-flow-diagram/>

[Accessed 26 12 2025].

GeeksforGeeks, 2025. *What is GitHub?*. [Online]
Available at: <https://www.geeksforgeeks.org/git/what-is-github/https://www.geeksforgeeks.org/git/what-is-github/>
[Accessed 29 12 2025].

GeeksforGeeks, 2025. *What is Scrum in Software Development?*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering/scrum-software-development/>
[Accessed 25 12 2025].

GeeksforGeeks, 2025. *What is Scrum? Understanding the Agile Framework for Project Management*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering/what-is-scrum/>
[Accessed 25 12 2025].

GeeksforGeeks, 2025. *What is Spiral Model in Software Engineering?*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering/software-engineering-spiral-model/>
[Accessed 25 12 2025].

HERStyle, 2023. *The REAL Reason You Have a Closet Full of Clothes and Nothing to Wear*. [Online]
Available at: <https://herstylellc.com/2023/01/17/the-real-reason-you-have-a-closet-full-of-clothes-and-nothing-to-wear/>
[Accessed November 2025].

Lucidchart, 2025. *What is an Entity Relationship Diagram (ERD)?*. [Online]
Available at: <https://www.lucidchart.com/pages/er-diagrams>
[Accessed 27 12 2025].

McKinsey & Company, 2025. *mckinsey*. [Online]
Available at: <https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-fast-fashion>
[Accessed 22 November 2025].

MockFlow , 2025. *Architecture Diagram*. [Online]
Available at: <https://mockflow.com/glossary/architecture-diagram#:~:text=An%20Architecture%20Diagram%20is%20a,on%20the%20purpose>

%20and%20audience.

[Accessed 26 12 2025].

Quora, 2015. *How long does it normally take to choose the outfit in the morning? Is dressing up nicely important?*. [Online]

Available at: <https://www.quora.com/How-long-does-it-normally-take-to-choose-the-outfit-in-the-morning-Is-dressing-up-nicely-important#:~:text=Some%20people%20spend%20%E2%80%9310%20minutes%20deciding%20what,sitting%20down%20for%20most%20of%20the%20day?>

[Accessed 25 November 2025].

The Carbon Literacy Trust, 2025. *carbonliteracy*. [Online]

Available at: <https://carbonliteracy.com/fast-fashions-carbon-footprint/>

[Accessed 22 Nov 2025].

Trello, 2025. *What is Trello: Learn Features, Uses & More*. [Online]

Available at: <https://trello.com/tour>

[Accessed 29 12 2025].

uniformmarket, 2025. *uniformmarket*. [Online]

Available at: <https://www.uniformmarket.com/statistics/fast-fashion-statistics>

[Accessed 22 Novemner 2025].

United Nations Environment Programme (UNEP), 2025. *unep*. [Online]

Available at: <https://www.unep.org/technical-highlight/sustainable-fashion-take-centre-stage-zero-waste-day>

[Accessed 22 Nov 2025].

WiseGuy Reports, 2025. *Global Wardrobe App Market Research Report*, s.l.: WiseGuy Reports.

Yehounme, D. D. a. G., 2017. *The Apparel Industry's Environmental Impact in 6 Graphics*. [Online]

Available at: <https://www.wri.org/insights/apparel-industrys-environmental-impact-6-graphics>

[Accessed 20 December 2025].

9. Appendix

9.1 Software Requirement Specifications

Software Requirements Specification (SRS) is a document that contains software's functions and constraints and acts as a blueprint for the system, developers, and stakeholders.

9.1.1 Introduction

9.1.1.1 Purpose

This Software Requirement Specification (SRS) document defines the functional and non-functional requirements of the Closet Buddy system Which is on air powered virtual storage management and outfit recommendation system. The purpose of this document is to provide a clear and comprehensive overview of the system's behavior and its constraints. This document is intended for developers, supervisors, evaluators, and other stakeholders of who are involved in this project

9.1.1.2 Scope

Closet Buddy is an AI powered virtual closet and storage management application that helps the user to organize their storage spaces by mapping their real physical storage digitally. It also generates outfit recommendations and provides insights into the user's wardrobe. The system allows the user to upload clothing images, validate the uploaded images, segments the uploaded clothing automatically, extracts metadata, manage virtual storage units, and receive AI-based outfit recommendations. The system is here to help users overcome "Decision Fatigue" in outfit planning and save their time and energy for the day.

9.1.1.3 Overview

There are 4 main parts to this document. The Introduction section outlines the purpose and scope of the system and document. The General Description section provides an overview of the system functionality, its intended users, and constraints. The Functional Requirement describes system behavior in detail, while the Non-Functional Requirements define attributes such as performance, usability, and security that contribute to the quality of the project. This document is made using the IEEE-style format.

9.1.2 General Description

9.1.2.1 Product perspective

Closet Buddy is a standalone mobile application supported with backend made using Django framework and AI processing services. This system is built using client-server architecture where a Flutter-based mobile client communicates with the Django REST backend. AI models handle image processing, clothing classification, and outfit recommendation, while a relational database currently PostgreSQL stores user, clothing, non-clothing, and storage unit data.

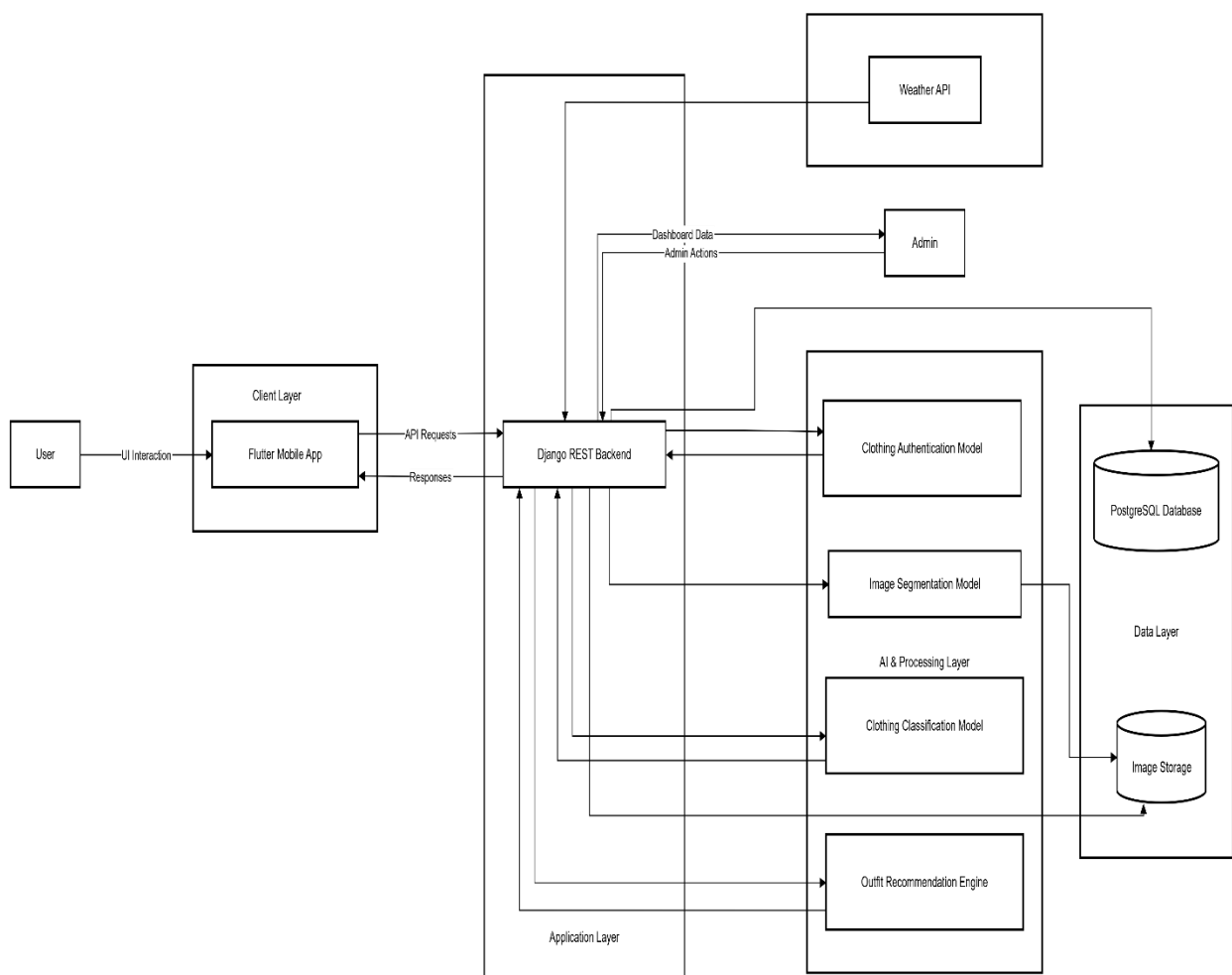


Figure 31: System Perspective Diagram

9.1.2.2 Product Functions

A high-level overview of the system functions and features are as follows:

PF1: Users can log in and register into the system

PF2: Logged in users can manage and edit their profile details

PF3: Upload and automatic processing of clothing images

PF4: Creation and management of virtual storage units

PF5: The clothing and non-clothing items in storage spaces are digitally represented.

PF6: Outfit recommendations are generated using AI and are rated by the user.

PF7: Integrates with weather APIs to provide random useful suggestions and tips.

PF8: Administrative system management

9.1.2.3 User Characteristics

User Class	Description
User	End users who create and manage storage spaces like closets, wardrobes, drawers, upload clothing images, organize the storage, generate outfits, rate outfits, and are looking for daily styling help.
Admin	System administrators responsible for managing user accounts, overseeing system data and analytics, and monitoring the performance of AI models and system.

The end users are expected to have basic smartphone proficiency and the administrators require technical knowledge of how the system operates.

9.1.2.4 Operating Environment

- Mobile Application: Android / iOS (Flutter)
- Backend Server: Django REST Framework
- Database: PostgreSQL
- AI Processing: Python-based ML models
- External Services: Weather API

9.1.2.5 Assumptions and dependencies

- The user will provide clear images of clothing items in a well-lit environment, preferably on a flat surface.
- The external APIs are available and reliable.
- The system is developed, tested and designed incrementally.
- Internet is available for online processing and storage.
- The pre-trained AI models that are used will be functional and available.

9.1.2.6 Design and implication constraints

- The AI model's accuracy depends on the quality of the training data.
- Core features require internet connectivity.
- Image processing performance varies from device to device and the quality of the provided photo.
- The system must comply with data protection standards.

9.1.3 Functional Requirements

9.1.3.1 User Account Management

FR-1: The system shall allow users to register using valid credentials.

FR-2: The system shall allow users to log in and log out securely.

FR-3: The system shall allow users to view and update profile information.

9.1.3.2 Clothing Image Processing and Uploading

FR-4: The system shall allow users to upload clothing images.

FR-5: The system shall authenticate whether an uploaded image represents a clothing item.

FR-6: The system shall segment the clothing item from the background.

FR-7: The system shall classify clothing attributes such as category and color.

FR-8: The system shall store processed clothing items in a selected storage unit.

9.1.3.3 Storage Management

FR-9: The system shall allow users to create, edit, and delete storage units.

FR-10: The system shall support hierarchical storage structures.

FR-11: The system shall allow users to assign clothing and non-clothing items to storage units.

FR-12: The system shall allow users to mark storage units as “put away.”

FR-13: The system shall display a virtual closet dashboard.

9.1.3.4 Outfit Generation

FR-14: The system shall generate outfit recommendations using available clothing items.

FR-15: The system shall use weather data to provide tips.

FR-16: The system shall allow users to rate generated outfits.

FR-17: The system shall maintain a history of generated outfits.

9.1.3.5 Non-Clothing Item Management

FR-18: The system shall allow users to add non-clothing items.

FR-19: The system shall store non-clothing items within storage units.

FR-20: The system shall exclude non-clothing items from outfit generation.

9.1.3.6 Administrative Functions

FR-21: The system shall allow admins to manage user accounts.

FR-22: The system shall allow admins to manage AI models.

FR-23: The system shall provide a system dashboard for admins.

9.1.4 Non-Functional Requirements

9.1.4.1 Performance Requirements

- The system shall process clothing image uploads within an acceptable response time.
- Outfit generation shall complete within a few seconds under normal conditions.
- All Clothing items shall display an image for viewing.

9.1.4.2 Usability Requirements

- The user interface shall be intuitive and mobile-friendly.
- The system shall minimize manual data entry for users.
- The users shall be able to edit clothing information.

9.1.4.3 Reliability Requirements

- The system shall handle unexpected inputs properly.
- User data shall be preserved in case of system failure.

9.1.4.4 Security Requirements

- User authentication data shall be securely stored.
- Access to administrative functions shall be restricted.
- The system shall protect user-uploaded images and personal data.

9.1.4.5 Scalability Requirements

- The system shall support increasing numbers of users and clothing items.
- The architecture shall allow future integration of additional AI models.

9.1.4.6 Maintainability Requirements

- The system shall follow modular design principles.
- Code and models shall be documented for future updates.

9.1.5 Conclusion

In conclusion, this SRS document defines the functional and non-functional requirements of the Closet Buddy system and serves as a foundation for system design, development, testing, and evaluation ensuring that final implementation is according to user needs and project objectives.

9.2 Details of Considered methodology

The reasons for consideration and reasons for not choosing the considered methodologies are discussed below:

9.2.1 Iterative Waterfall Methodology

This approach follows the traditional sequential phases like requirement analysis, design, development, testing, and maintenance but allows for feedback loops between each stage. It combines the step-by-step concept of the traditional waterfall model with the flexibility of iterative design, allowing for changes to be made at each step (GeeksforGeeks, 2025).

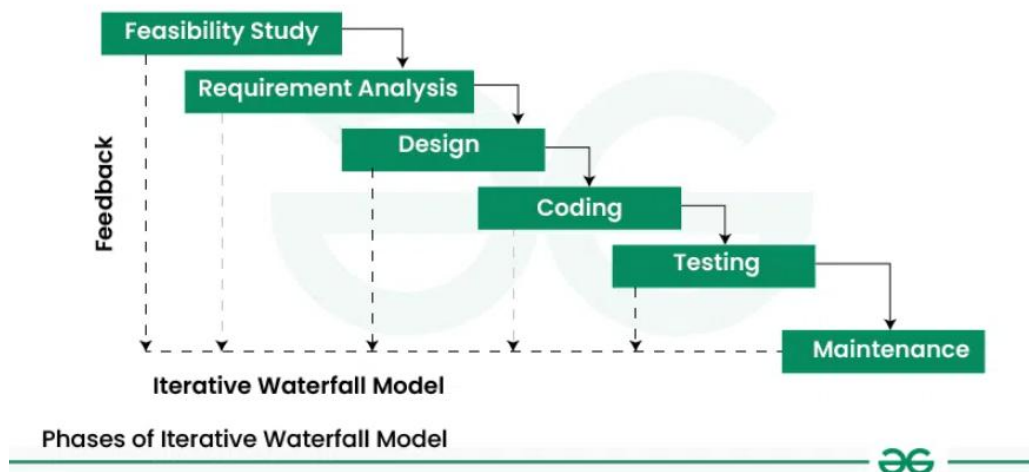


Figure 32: Iterative Waterfall (GeeksforGeeks, 2025)

Table 7: Reasoning for Iterative Waterfall

S.N.	Reasons for Considering	Reasons for Not Choosing
1.	It has clear and structured sequential phases which are easy to follow.	It is too rigid for AI development.
2.	This methodology is easy to understand and document.	Lacks flexibility and accommodation for changing requirements.
3.	It is suitable for projects that have fixed requirements.	The testing phase may be late, and thus critical issues may be discovered too late in development.
4.	Each phase has cleared and specific outcome that helps to guide the deployment process.	If some changes must be made it is difficult to cycle back to previous steps without breaking the project timeline.

9.2.2 Scrum Methodology

Scrum is a type of Agile framework that focuses on delivering functional software in short, time bounded cycles called “Sprints” and is used by teams to organize tasks and work for developing complex products (GeeksforGeeks, 2025). In this methodology flexibility, continuous feedback, and the ability to adapt to rapidly changing requirements are highly emphasized.

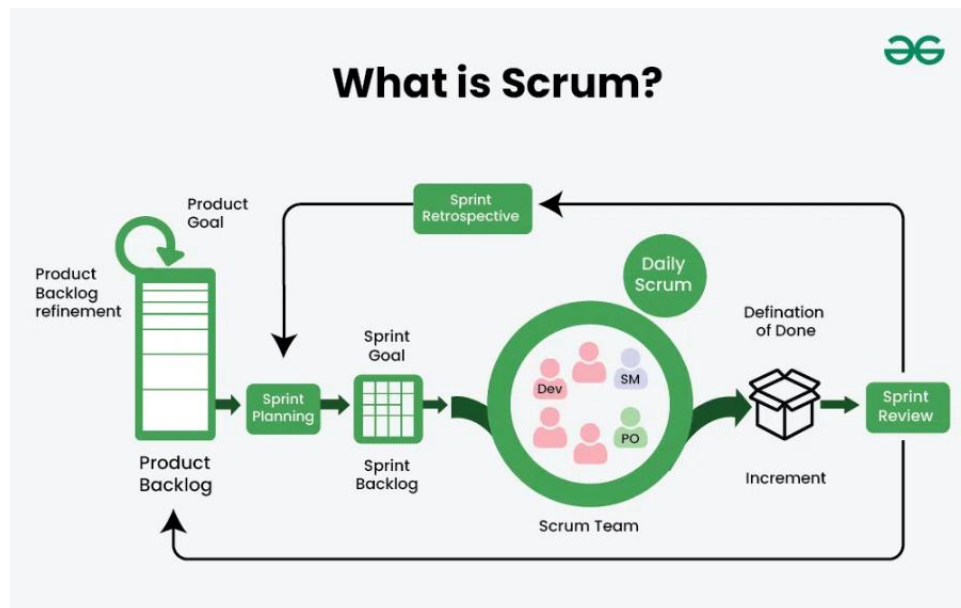


Figure 33: Scrum Cycle (GeeksforGeeks, 2025)

Table 8: Reasoning for Scrum

S.N.	Reasons for Considering	Reasons for Not Choosing
1.	This methodology is suitable for evolving software products like this project.	This methodology is not suitable for solo or small team projects.
2.	It supports frequent iteration and feedback from stakeholders, which is great for UI/UX testing.	It requires constant and continuous stakeholder involvement, which is tedious.
3.	Rapid development and adaptability while developing the project is highly encouraged.	Difficult to follow all the scrum roles and ceremonies as a single developer.
4.	Risks and problems can be identified early on and mitigated.	The lack of formal end point and two week sprint cycle is difficult for an academic project

9.3.3 Spiral Methodology

Spiral methodology is also a combination of waterfall methodology and iterative model with an added emphasis on risk management and handling along with prototyping. The four major quadrants of the spiral methodology are planning and objective identification, risk analysis which is the main part, product or prototype development, and evaluation (GeeksforGeeks, 2025). Spider methodology is mainly used in complex, large, and high-risk software projects.

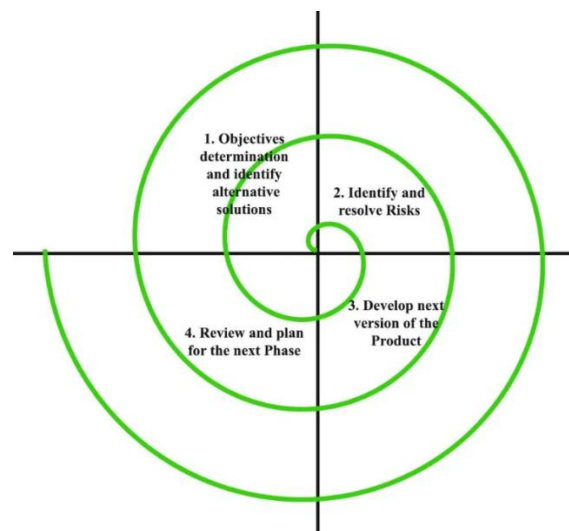


Figure 34: Spiral Methodology Cycle (GeeksforGeeks, 2025)

Table 9: Reasoning for Spiral Model

S.N.	Reasons for Considering	Reasons for Not Choosing
1.	This model gives a strong focus on risk analysis for the project.	This matter is complex to manage and document.
2.	It is suitable for complex and high-risk systems and since the AI features are high risk this model's focus on risk was appealing.	Using this methodology would have been excessive for a small-scale academic project like this.
3.	Allows for iterative refinement of the features in the project.	The iterative risk analysis loop creates significant documentation overhead.
4.	Allows for strong risk identification and mitigation with prototyping	The disadvantages significantly outweigh the advantages of using this model.

[Back to Considered Methodology.](#)

9.4 Phases of Iterative Incremental Methodology

This project will be developed using the Iterative Incremental methodology following the given phases:

1) Requirement Analysis and Planning

- Collect functional requirements of the project: segmentation, classification, outfit recommendation, storage units.
- Define the non-functional requirements.
- Prepare datasets for classification.
- Identifying external dependencies.

2) System and Architecture Design

- Architecture Design: Flutter frontend, Django Backend, AI Layer, PostgreSQL database.
- Design Dataflow.
- Design recommendation pipeline.

3) Incremental Development Cycles

The development will be carried out in different incremental development cycles to complete feature on time and for review.

- Increment 1: Authentication module
- Increment 2: Segmentation module
- Increment 3: Classification module
- Increment 4: Storage system development
- Increment 5: Recommendation AI with weather feature integration
- Increment 6: Final system development and integration

4) Testing and Evaluation

- Unit tests for each feature.
- UI testing of Flutter.
- Integration testing for backend and frontend
- Model performance testing.
- Performance testing
- Final system testing

5) Deployment (Not within the project Scope)

- Deploy backend to cloud (AWS)
- Connect Flutter app to deployed API

6) Maintenance and Iteration

- Fixing bugs and refining UI
- Perform optimization based on feedback
- Finalize report and documentation

Back to selected methodology.

9.5 Gantt Chart Task Table

Table 10: Gantt Chart Task Table

Task Name	Start	Finish	Duration
1. Requirement Analysis and Planning	11/2/2025	11/16/2025	14 day
1.1 Identify needs and requirements	11/2/2025	11/4/2025	2 day
M1 Topic Finalization	11/4/2025	11/4/2025	Milestone
1.2 Define Functional and Non-functional requirements	11/5/2025	11/9/2025	4 day
1.3 Identify External Dependencies	11/10/2025	11/13/2025	4 day
1.4 Prepare Datasets	11/14/2025	11/16/2025	4 day
2. System and Architecture Design	11/16/2025	12/3/2025	18 day
2.1 Establish architecture	11/17/2025	11/20/2025	4 day
2.2 Design Database Schema	11/21/2025	11/26/2025	6 day
2.3 Data Flow Diagrams and Use Cases	11/16/2025	11/22/2025	7 day
M2 Proposal Submission	12/3/2025	12/3/2025	Milestone
Increment Development	12/4/2025	3/20/2026	74 day
3. Increment 1 (Authentication)	12/4/2025	12/17/2025	9 day
3.1 Planning and Research	12/4/2025	12/5/2025	1 day
3.2 Determine Objectives	12/6/2025	12/7/2025	1 day
3.3 Analysis and Design	12/8/2025	12/9/2025	1 day
3.4 Implement Authentication	12/10/2025	12/13/2025	3 day
3.5 Testing	12/14/2025	12/15/2025	2 day
3.6 Evaluation	12/16/2025	12/17/2025	1 day
4. Increment 2 (Segmentation and Color Extraction)	12/17/2025	12/30/2025	8 day
4.1 Planning and Research	12/17/2025	12/17/2025	1 day
4.2 Determine Objectives	12/18/2025	12/19/2025	1 day
4.3 Analysis and Design	12/19/2025	12/20/2025	1 day
4.4 Implement Segmentation	12/20/2025	12/22/2025	2 day
4.5 Segmentation Preview + Fallback	12/22/2025	12/23/2025	1 day
4.6 Color Extraction	12/24/2025	12/25/2025	1 day
4.7 Testing	12/26/2025	12/27/2025	1 day

4.8 Evaluation	12/28/2025	12/29/2025	1 day
M3 Authentication and Segmentation functional with Interim Report Submission	12/30/2026	12/30/2026	Milestone
5 Increment 3 (Classification)	1/2/2026	1/15/2026	10 day
5.1 Planning and Research	1/2/2026	1/2/2026	1 day
5.2 Determine Objectives	1/5/2026	1/5/2026	1 day
5.3 Analysis and Design	1/6/2026	1/6/2026	1 day
5.4 Implement Clothing Type Categorization	1/7/2026	1/9/2026	3 day
5.5 Testing	1/12/2026	1/13/2026	2 day
5.6 Evaluation	1/14/2026	1/14/2026	1 day
M4 Classification and meta data extraction successful	1/15/2026	1/15/2026	Milestone
6. Increment 4 (Storage System)	1/16/2026	1/29/2026	10 day
6.1 Planning and Research	1/16/2026	1/16/2026	1 day
6.2 Analysis and Design	1/19/2026	1/19/2026	1 day
6.3 Implement Virtual Closet Model	1/20/2026	1/22/2026	3 day
6.4 Flutter UI for Storage	1/23/2026	1/27/2026	3 day
6.5 Testing	1/28/2026	1/28/2026	1 day
6.6 Evaluation	1/28/2026	1/28/2026	1 day
M5 Completed the storage interface	1/29/2026	1/29/2026	Milestone
7. Increment 5 (Recommendation + Weather)	1/30/2026	2/19/2026	15 day
7.1 Planning and Research	1/30/2026	1/30/2026	1 day
7.2 Analysis and Design	2/2/2026	2/2/2026	1 day
7.3 Develop Hybrid Recommendation Engine	2/3/2026	2/9/2026	5 day
M6 Recommendation engine functional	2/10/2026	2/10/2026	Milestone
7.4 Weather Integration	2/11/2026	2/12/2026	2 day
7.5 Develop Rating AI	2/13/2026	2/16/2026	2 day
7.6 Testing	2/17/2026	2/18/2026	2 day
7.7 Evaluation	2/19/2026	2/19/2026	1 day
M7 Weather Integration and AI rating completed	2/19/2026	2/19/2026	Milestone

8. Increment 6 (Final Integration)	2/20/2026	3/20/2026	21 day
8.1 Planning and System Design	2/20/2026	2/24/2026	3 day
8.2 Integrate Flutter with Backend + AI	2/25/2026	3/12/2026	12 day
8.3 Testing	3/13/2026	3/18/2026	4 day
8.4 Evaluation	3/19/2026	3/20/2026	2 day
M8 Backend Frontend Integration Completed	3/20/2026	3/20/2026	Milestone
Buffer	2/16/2026	3/16/2026	21 day
9. Testing and Evaluation (Full System)	3/9/2026	3/31/2026	17 day
9.1 Final System Testing	3/9/2026	3/17/2026	7 day
9.2 Integration Testing	3/18/2026	3/25/2026	6 day
9.3 Performance Testing	3/26/2026	3/31/2026	4 day
M9 Testing Phase Done	3/31/2026	3/31/2026	Milestone
10. Maintenance & Iteration	4/1/2026	4/21/2026	27 day
10.1 Fix Bugs and Improve Accuracy	4/1/2026	4/6/2026	4 day
10.2 Optimize based on Feedback	4/7/2026	4/10/2026	4 day
10.3 Finalize Documentation	4/13/2026	4/20/2026	6 day
M11 Final Report Submission	4/21/2026	4/21/2026	Milestone

[Back to Gantt Chart.](#)

9.6 Resource Requirement Specifications

1. Hardware Requirements

- Good quality laptop with the following specifications:
 - 32 GB RAM
 - Multi-core processor (Intel i9 or equivalent)
 - SSD storage (512GB or higher)
 - GPU (NVIDIA RTX 4050 or equivalent)
- A standard mobile phone

2. Software Requirements

- IDE and Testing: Visual Studio Code, Postman, Google Colab/ Jupyter Notebook
- Frontend: Flutter with Dart
- Backend:
 - Python
 - Django
 - TensorFlow/PyTorch/Scikit-learn
 - Hugging face models
- Cloud Storage: AWS
- GitHub for version control
- Database: PostgreSQL
- Diagrams: Miro, Draw.io
- Wireframing: Miro
- Surveying: Google Forms

3. Cloud/External Compute Resources

If local hardware is insufficient for heavy AI workload:

- Colab T4 GPU/ v5e-1 TPU

4. Datasets

- <https://github.com/rainalexotl/deepfashion-image-classification>

[Go Back](#)

9.7 Expanded Use-Case Description

I. Use Case: Manage Account

Actors: User

Description: Allows user to manage their information and access within the Closet Buddy system. This allows the user to register a new account or log into an existing account. After logging in, the user can view or update their profile information. Access to the system is given only after logging in and authentication is done using Django's authentication framework

Typical Course of Events:

Actor Action	System Response
1. User starts the application.	2. System shows option for login and registration.
3. The user chooses to log in or register.	4. Asks for Necessary information.
5. User enters valid credentials.	6. System authenticates the user using Django authentication framework.
7. User logs in successfully.	8. Grants access to the system.
9. Selects "View Profile" icon.	10. System displays user profile information.
11. Updates profile details (optional)	12. Validates and saves updated information

Alternate Courses:

Line 7: If the login credentials are invalid system prompts the user to reenter the information.

Line 6: If the account is already registered, prompts the user to log in.

Line 9: If the user logs out, the system terminates the session and redirects to the login screen.

II. Use Case: Upload Clothing Item

Actors: User

Description: The users can add a new clothing item to their virtual wardrobe by selecting a storage unit and uploading images. Open uploading an image the system automatically verifies if the image is a valid clothing item, segments the clothing item from the background to make a clean cut out, extracts relevant metadata such as category and color, and stores it in the selected virtual storage unit.

Typical Course of Events:

Actor Action	System Response
1. User selects "upload clothing item" option.	2. Ask the user to select a storage unit.
3. Selects a storage unit.	4. Opens image upload interface.
5. Uploads clothing images.	6. Authenticates the image as a clothing item.
	7. Perform segmentation to remove image background.
8. Prompts the user to confirm or redo segmentation.	9. System extracts metadata from the clothing item and shows the data to the user.
10. Reviews the extracted data and confirms.	11. Stores the clothing item in database

Alternate Courses:

Line 6: If the image is not recognized as a valid clothing item rejects the upload and notifies the user.

Line 8: If the user redoes segmentation asks the user to upload another image.

Line 10: if the user cancels the process, no data is stored.

III. Use Case: Add Non-Clothing Item

Actors: User

Description: This use-case allows adding non-clothing items such as accessories or miscellaneous items into the virtual storage spaces. Users can select a virtual space and simply add the name and description for the non-clothing item to store it. This ensures that the user's physical storage spaces are digitally represented.

Typical Course of Events:

Actor Action	System Response
1. User selects "Add Non-Clothing Item" option.	2. Asks to select storage unit.
3. Selects storage unit.	4. Displays input form.
5. Enters Name and Description.	
6. Submits form.	7. Stores the item in selected storage unit.

Alternate Courses:

Line 7: If the required fields are empty prompts the user to complete them.

Line 6: if the user cancels the operation no data is saved.

IV. Use Case: Manage Storage

Actors: User

Description: This use case allows the users to organize their virtual storage spaces and the items in it. Users can create, edit, and view virtual storage units such as closets, drawers, or boxes. The users can mark storage spaces as “put away” to exclude the clothing items from outfit generation and search and filtered items within the storage units.

Typical Course of Events:

Actor Action	System Response
1. Open storage interface	2. Displays the virtual storage dashboard.
3. Creates or edits storage unit.	4. Validates and saves storage unit details.
5. Moves or assigns items to storage units.	6. Updates item-storage relationship.
7. Marks a storage as “put away” (optional).	8. Excludes the item in the storage from outfit generation.
9. Searches and filters item.	10. Displays the filtered results.
11. Deletes a storage unit (optional)	12. The storage unit is deleted

Alternate Courses:

Line 9: If no matching items are found displays an empty result.

Line 11: If a storage unit contains items, the system prevents deletion.

V. Use Case: Generate Outfit

Actors: User

Description: Enables the system to generate outfit recommendations based on user's available clothing items and their requirements. The users can give ratings as feedback to the generated outfits and view their outfit history.

Typical Course of Events:

Actor Action	System Response
1. User selects "Generate Outfit"	2. Shows outfit generation interface.
3. Enters requirements.	4. Retrieves all available clothing items.
	5. Runs the recommendation engine to display the outfit combination.
6. Views the generated outfit.	
7. Rates and saves the outfit	8. Saves the outfit to database.
9. Views outfits (optional)	10. Displays all the outfits

Alternate Courses:

Line 4: If insufficient clothing items are available notifies the user.

VI. Use Case: View Admin Dashboard

Actors: Admin

Description: Provides the system administrators with the tools to manage user accounts, monitor system activity and analytics, and manage AI models used for clothing recognition and recommendation. This ensures system stability and model maintenance.

Typical Course of Events:

Actor Action	System Response
1. Admin logs into the system	2. Authenticates admin credentials.
	3. Displays admin dashboard.
4. Manages user accounts (optional)	5. Updates user records
6. Review system activity	7. Displays logs and metrics

Alternate Courses:

Line 2: If not admin, access is denied

[Back to use case diagram.](#)

9.8 Online Survey Form

A survey was carried out with the aim of collecting data and understanding user requirements. The question ranged from general user details to their clothing and storage habits along with their interest in AI features and their feedback. The survey helped to understand how people store their clothes and their daily struggles with selecting their outfits and storage management. The survey was conducted using Google forms.

Closet Buddy – User Survey

This survey is conducted for an academic Final Year Project to understand user behavior related to clothing management, virtual storage, and interest in AI-assisted outfit recommendations. It will help to improve the system and develop a better application. The responses are anonymous and will be used for academic purposes. Thank you for your time.

Age Group *

☐ Under 18

☐ 18-24

☐ 25-35

☐ 35+

Gender *

☐ Male

☐ Female

☐ Non-Binary

☐ Other

Current Occupation *

☐ Student

☐ Working Professional

☐ House Wife/Husband

☐ Other: _____

Figure 35:Survey Form 1

Section 2 of 3

Section 2: Clothing & Storage Habits

Description (optional)

How do you currently store your clothes? *

☐ Wardrobe / Closet

☐ Drawers

☐ Storage boxes

☐ Cupboards

☐ Shelves

☐ Other

How often do you forget about clothes you already own? *

☐ Very Often

☐ Often

☐ Sometimes

☐ Rarely

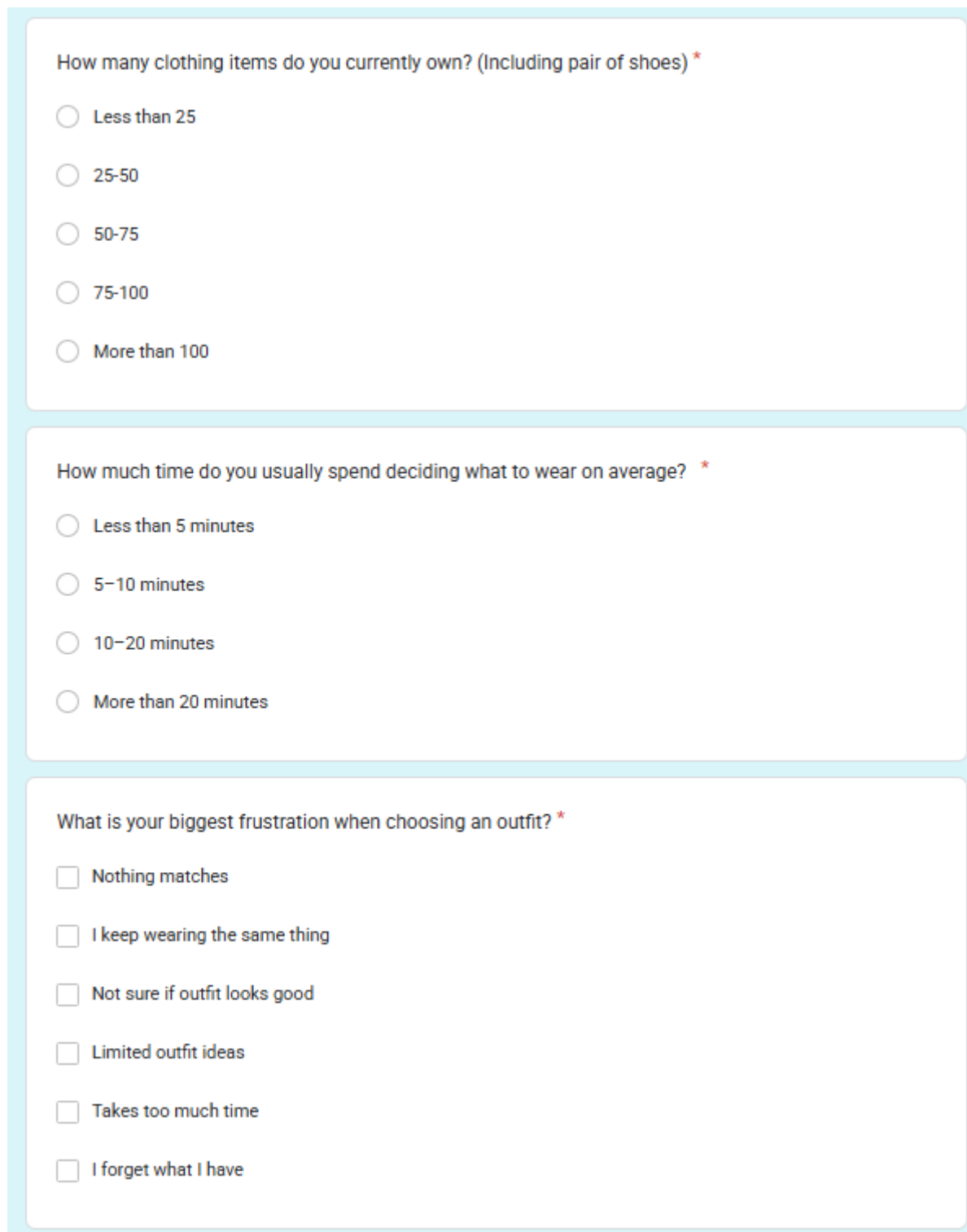
☐ Never

How often do you struggle to find a specific item because you don't remember which drawer/box it's in? *

12345

Never☐☐☐☐☐Very Often

Figure 36:Survey Form 2

The image shows a survey form with three distinct sections, each enclosed in a light blue rounded rectangular border. The first section contains a question about the number of clothing items owned, followed by five radio button options. The second section contains a question about the time spent deciding what to wear, followed by four radio button options. The third section contains a question about the biggest frustration when choosing an outfit, followed by six checkbox options.

How many clothing items do you currently own? (Including pair of shoes) *

☐ Less than 25

☐ 25-50

☐ 50-75

☐ 75-100

☐ More than 100

How much time do you usually spend deciding what to wear on average? *

☐ Less than 5 minutes

☐ 5-10 minutes

☐ 10-20 minutes

☐ More than 20 minutes

What is your biggest frustration when choosing an outfit? *

☐ Nothing matches

☐ I keep wearing the same thing

☐ Not sure if outfit looks good

☐ Limited outfit ideas

☐ Takes too much time

☐ I forget what I have

Figure 37:Survey Form 3

Section 3 of 3

Interest in Smart/AI Features

▼

⋮

Description (optional)

How important is it that your digital closet displays 'clean' cutouts of your clothes (no background) rather than raw photos? *



Raw Image **Segmented Image**

1 2 3 4 5

Not Important ☐ ☐ ☐ ☐ ☐ Very Important

Would you find it helpful to see a digital icon of your actual storage unit (e.g., 'Blue Box 1' or 'Top Drawer') to know exactly where an item is physically located? *

☐ Yes

☐ No

☐ Maybe

Would you use an app like Closet Buddy that digitally represents your storage spaces (closets, drawers, boxes), displays your clothing items and helps to generate good outfits? *

☐ Definitely yes

☐ Probably yes

☐ Maybe

☐ Probably no

Figure 38: Survey Form 4

Would you prefer the app to automatically detect the color and category of your clothing from a photo rather than typing it manually? *

☐ Yes

☐ No

How useful would AI-based outfit recommendations be for you? *

1 2 3 4 5

Not Useful ☐ ☐ ☐ ☐ ☐ Useful

Would you like to create your own outfits and receive an AI rating? *

☐ Yes

☐ No

☐ Maybe

Do you think AI feedback could help improve your dressing sense? *

☐ Yes

☐ No

☐ Maybe

What feature would make you most likely to use this app?

Short-answer text

Any suggestions or concerns regarding a virtual storage and outfit recommendation app?

Long-answer text

Figure 39: Survey Form 5

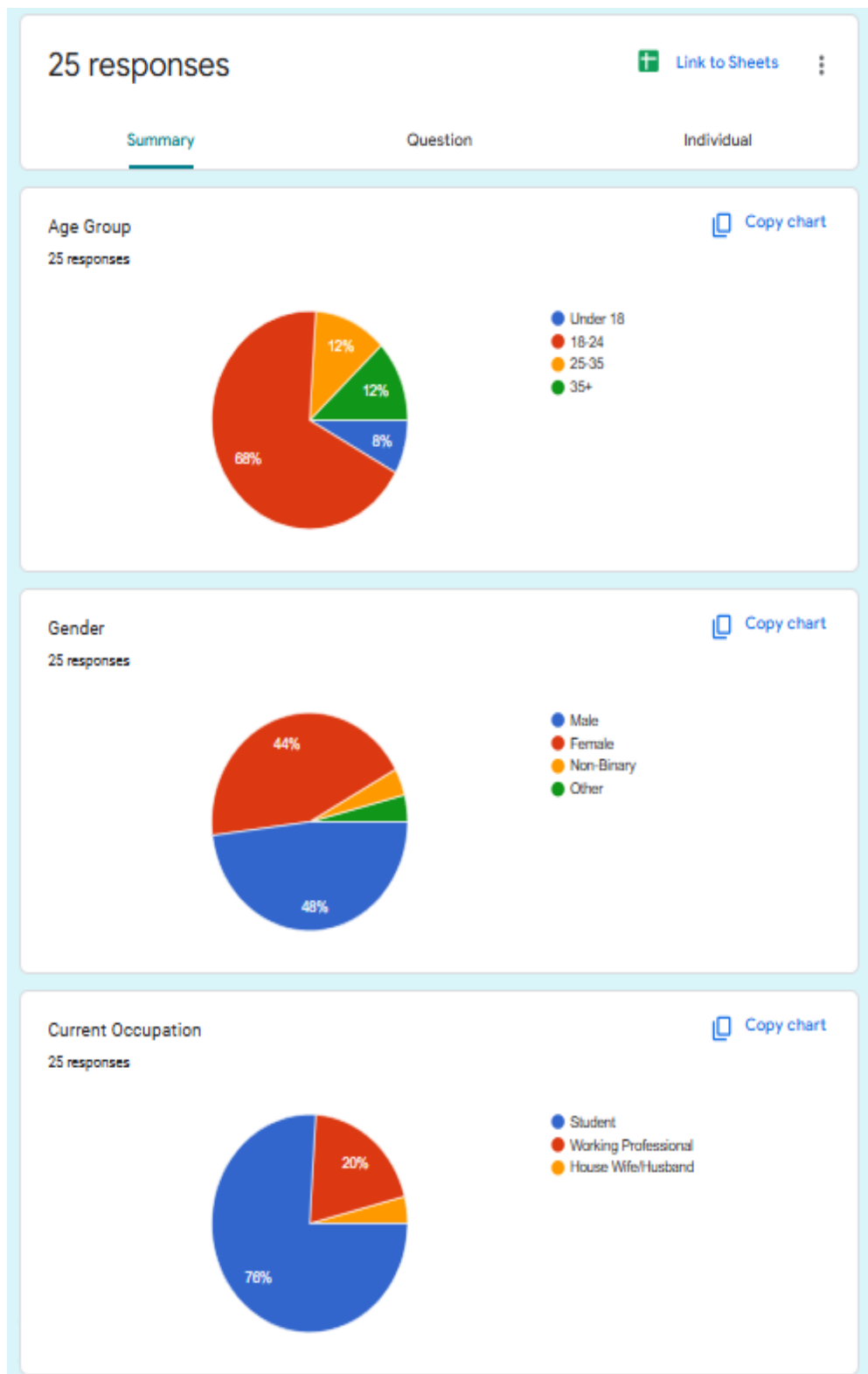


Figure 40: Survey Form Response 1

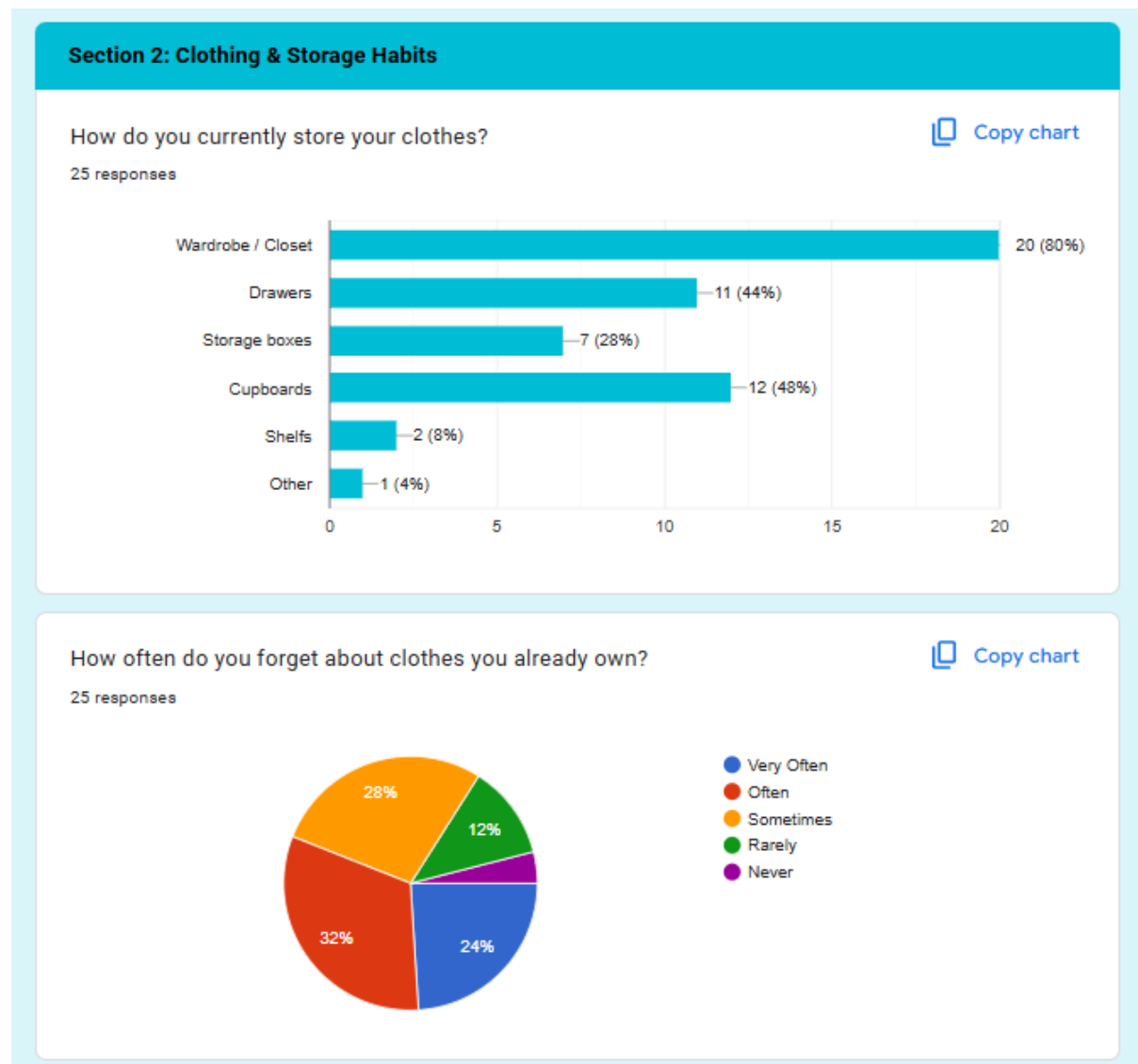


Figure 41: Survey Form Response 2

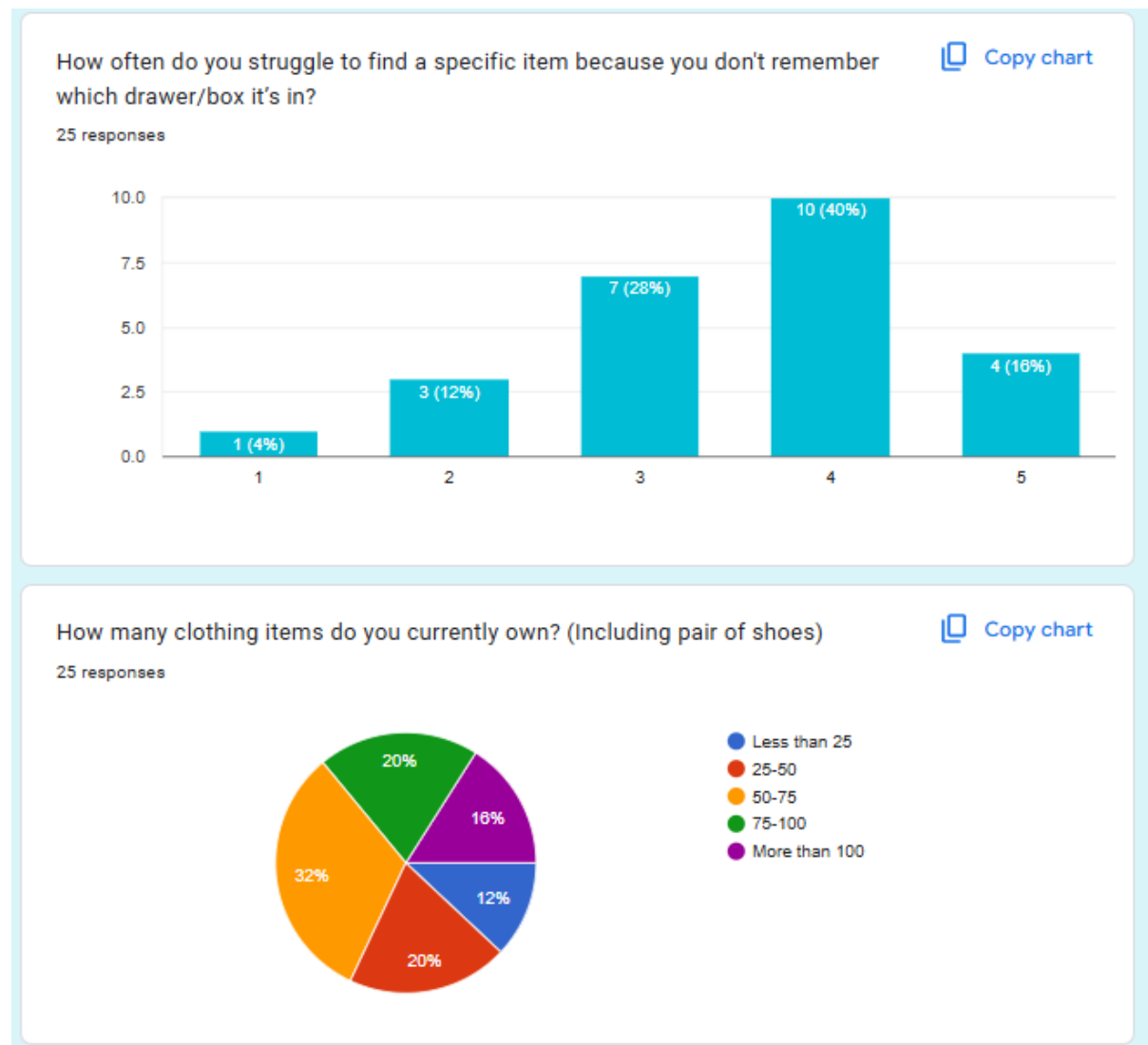


Figure 42: Survey Form Response 3

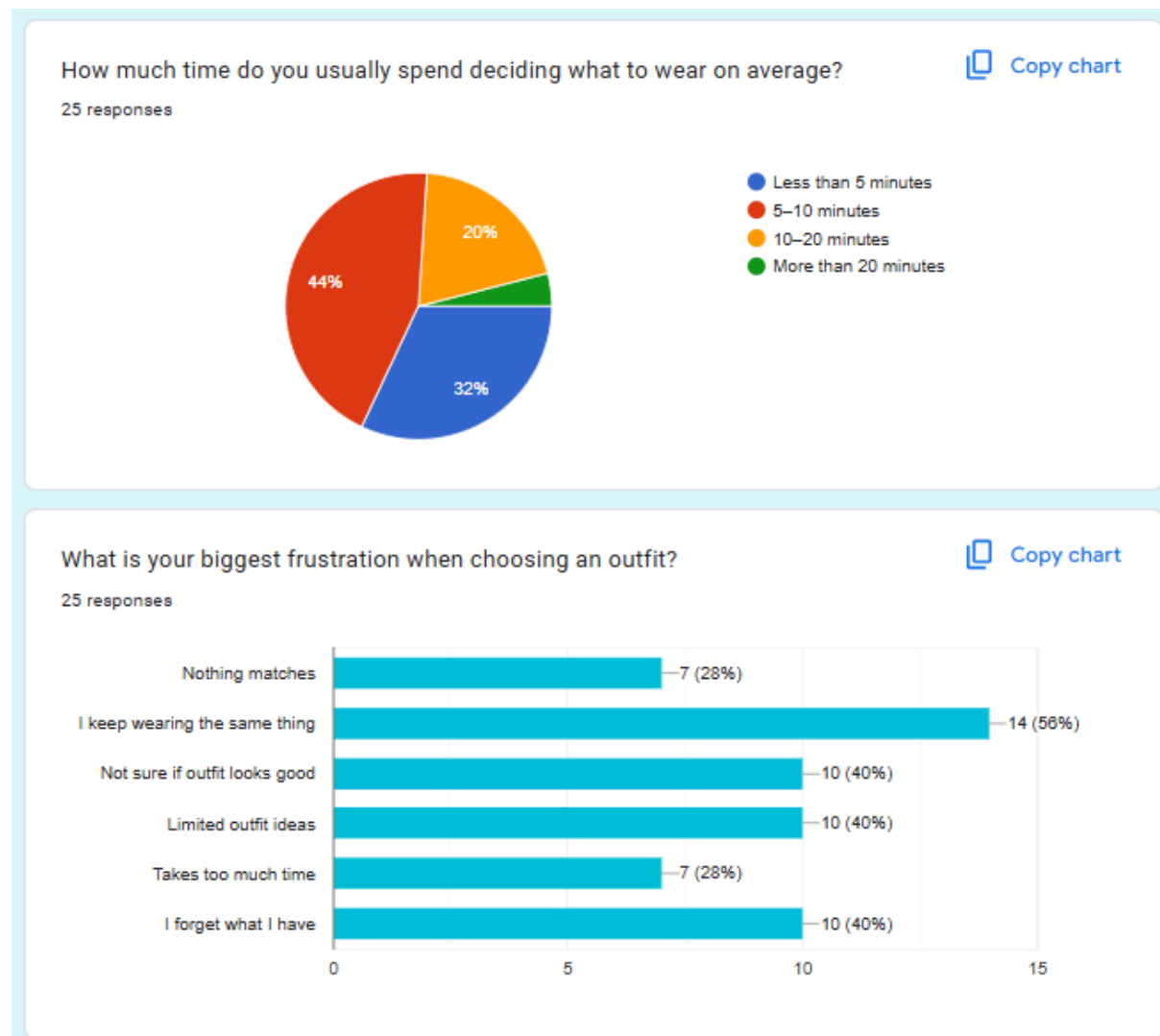


Figure 43: Survey Form Response 4

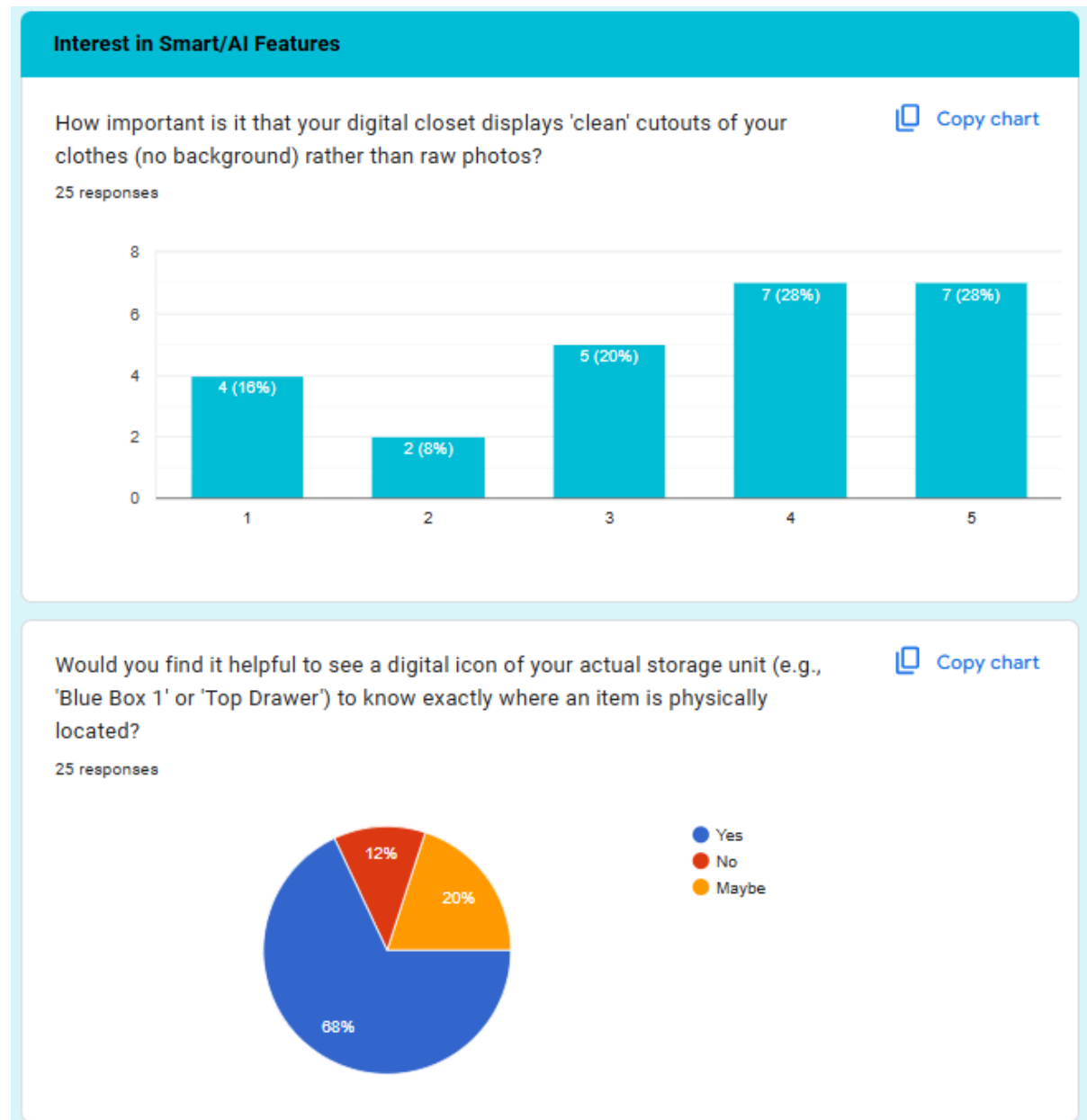


Figure 44: Survey Form Response 5

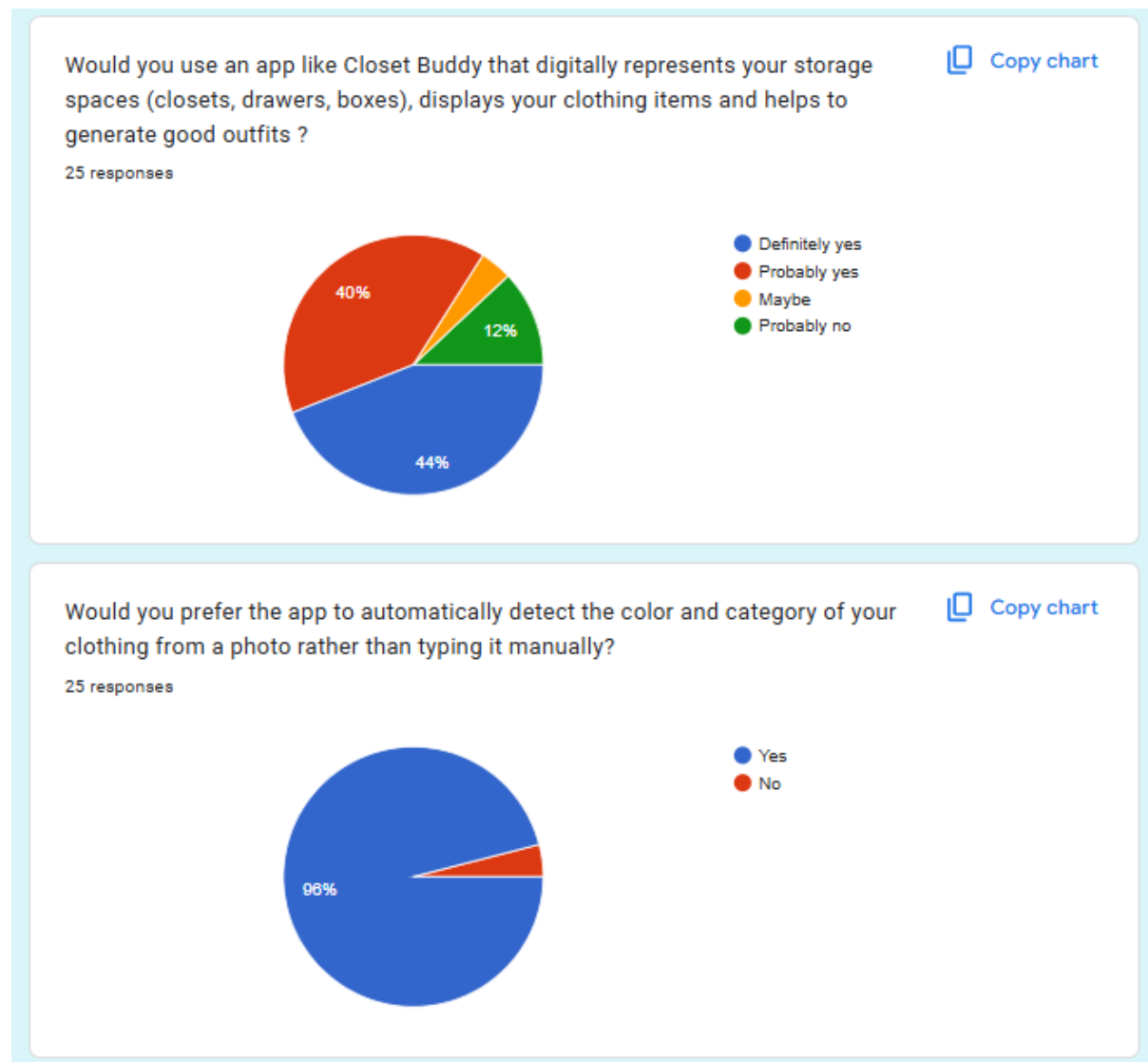


Figure 45: Survey Form Response 6

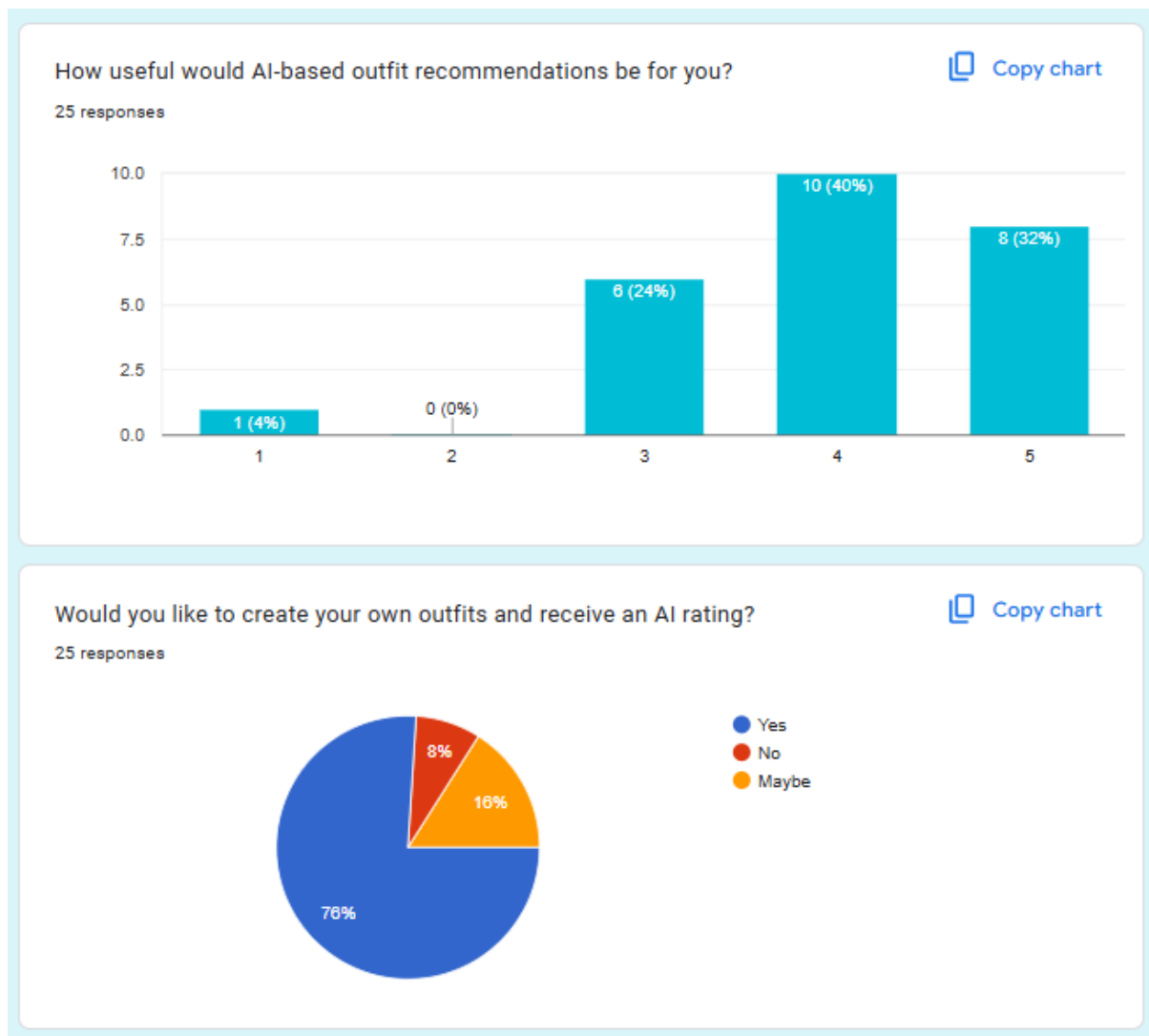


Figure 46: Survey Form Response 7

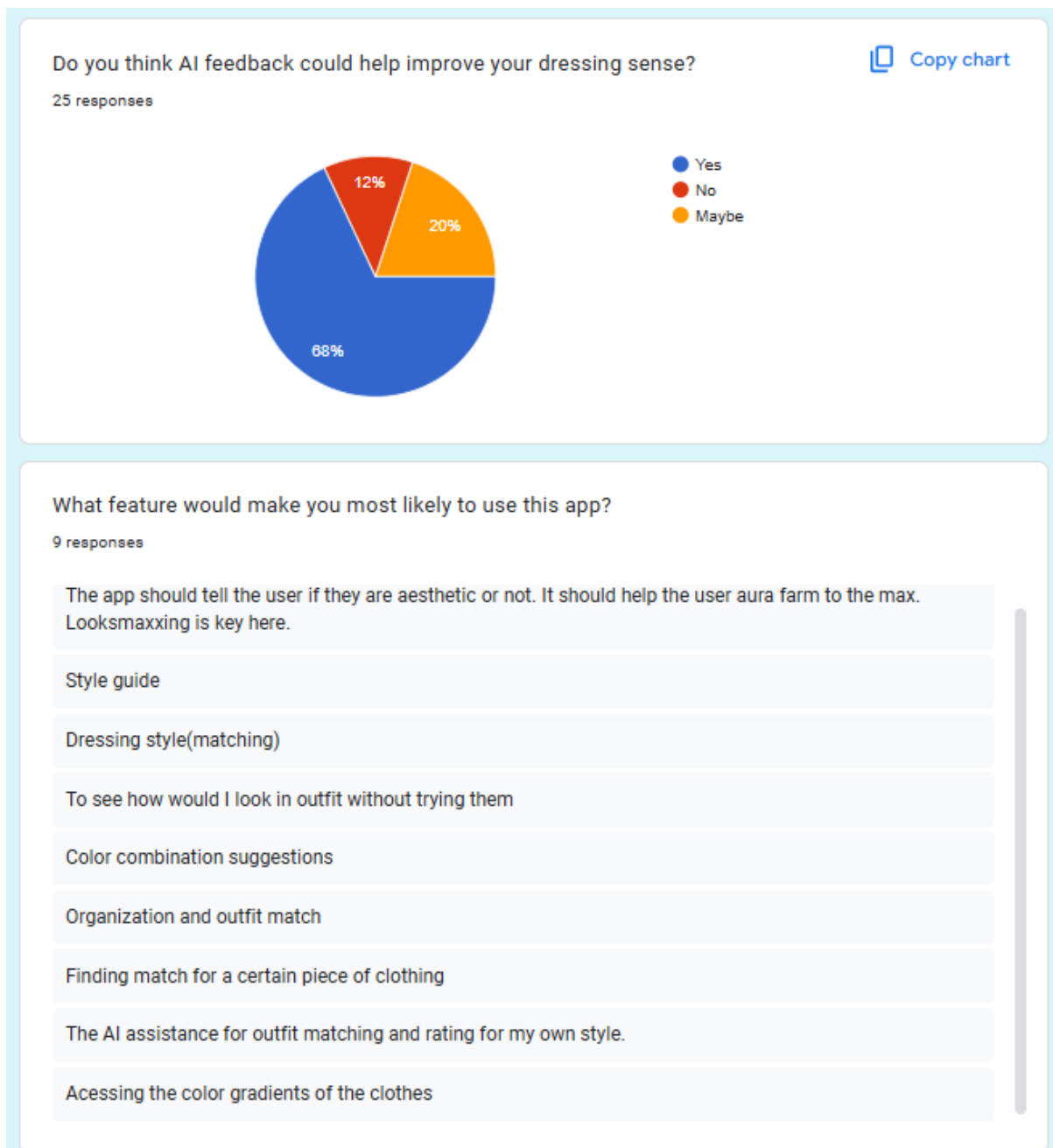


Figure 47: Survey Form Response 8

Any suggestions or concerns regarding a virtual storage and outfit recommendation app?

8 responses

Nope

Have you checked the app that shows up in the first scenes of Clueless? Closet Buddy should do exactly that.

No

It would be helpful for you to co ordinate with a fashion designer for better ideas.

Outfit repetition : feature that tells me how many times i have worn the clothing piece in the same week

Seems like a great and useful idea for a person like me who keeps on repeating clothes despite having lots, but a prototype demonstration would be easier to visualise how the app really functions especially with storage boxes and drawers

Make the user experience familair and uniform.....

Figure 48: Survey Form Response 9

[Go Back.](#)

9.9 Wireframes

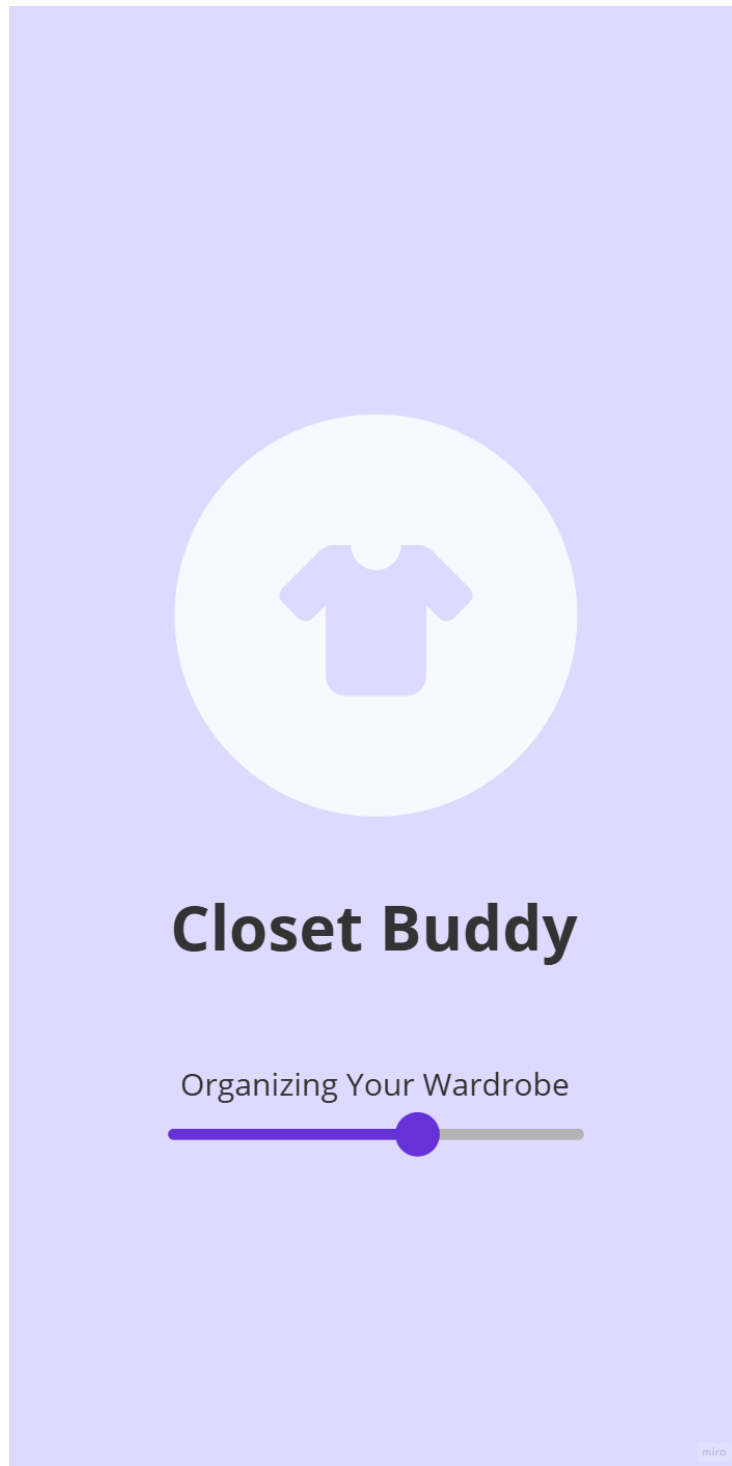
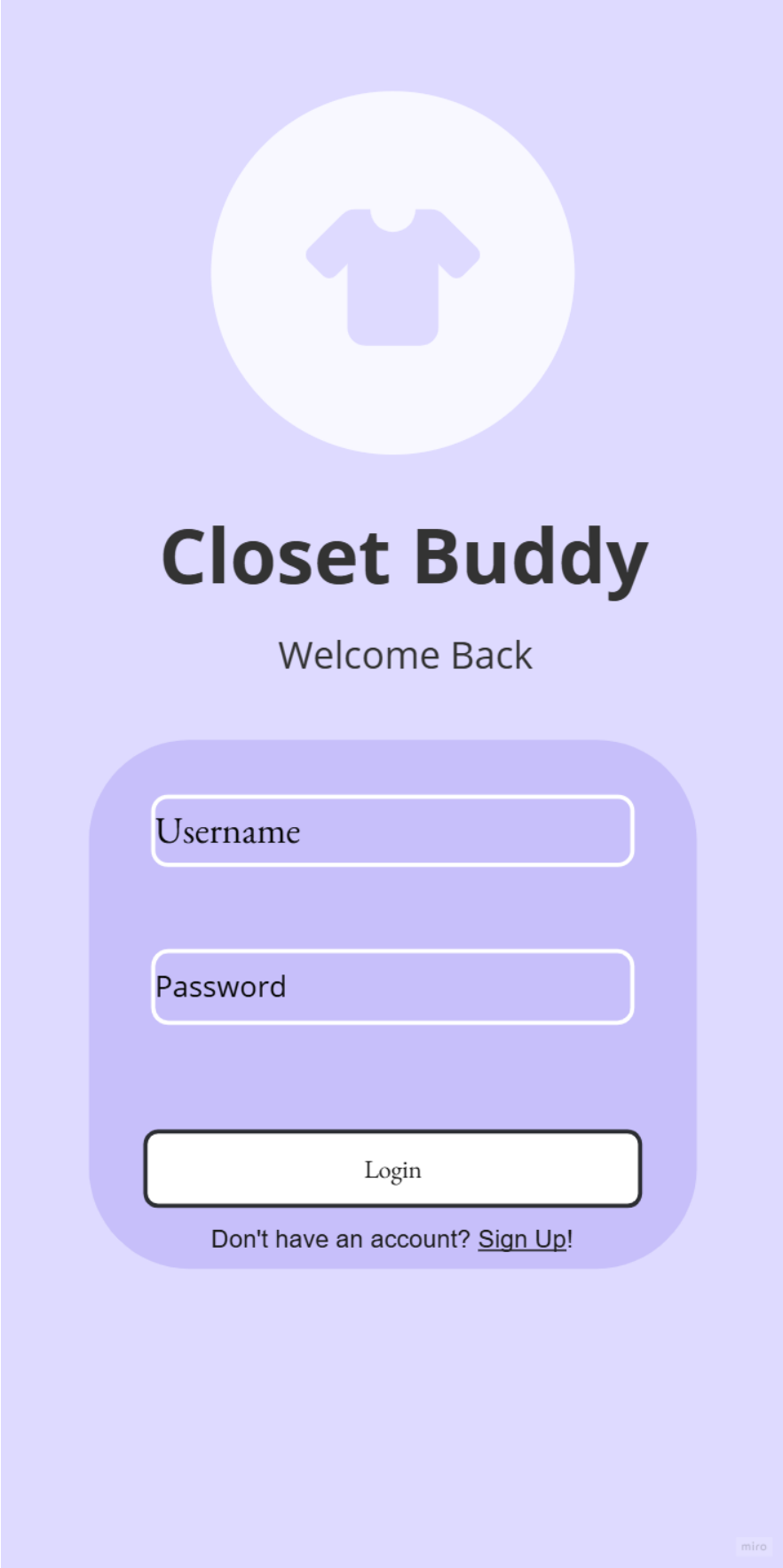


Figure 49: Loading Page



The image shows a login page for an application called "Closet Buddy". At the top, there is a circular icon containing a t-shirt. Below the icon, the title "Closet Buddy" is displayed in a large, bold, black font. Underneath the title, the text "Welcome Back" is centered. The login form is contained within a light purple rounded rectangle. It features two input fields: "Username" and "Password", both with white text on a light purple background. Below these fields is a "Login" button with a black border and white text. At the bottom of the form, there is a link that says "Don't have an account? [Sign Up!](#)". A small "miro" logo is visible in the bottom right corner of the page.



Closet Buddy

Welcome Back

Username

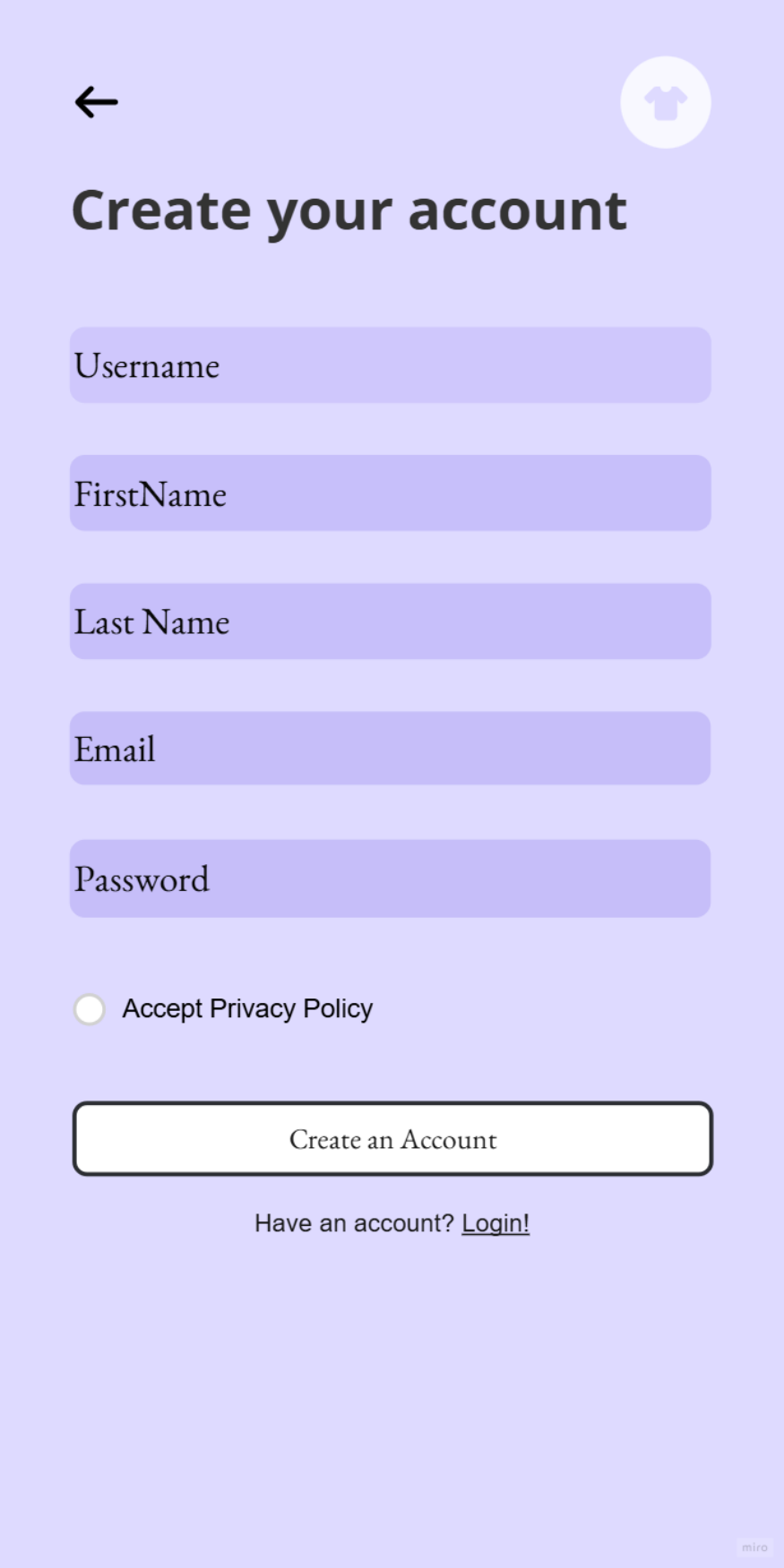
Password

Login

Don't have an account? [Sign Up!](#)

miro

Figure 50: Login Page



The image shows a mobile app register page with a light purple background. At the top left is a back arrow icon, and at the top right is a circular profile icon with a t-shirt. The main heading is 'Create your account'. Below it are five input fields: 'Username', 'FirstName', 'Last Name', 'Email', and 'Password'. Under the 'Password' field is a checkbox labeled 'Accept Privacy Policy'. A large 'Create an Account' button is centered below the checkbox. At the bottom, it says 'Have an account? [Login!](#)'. A 'miro' watermark is in the bottom right corner.

←

Create your account

Username

FirstName

Last Name

Email

Password

☐ Accept Privacy Policy

Create an Account

Have an account? [Login!](#)

miro

Figure 51: Register Page



Figure 52: Homepage Wireframe

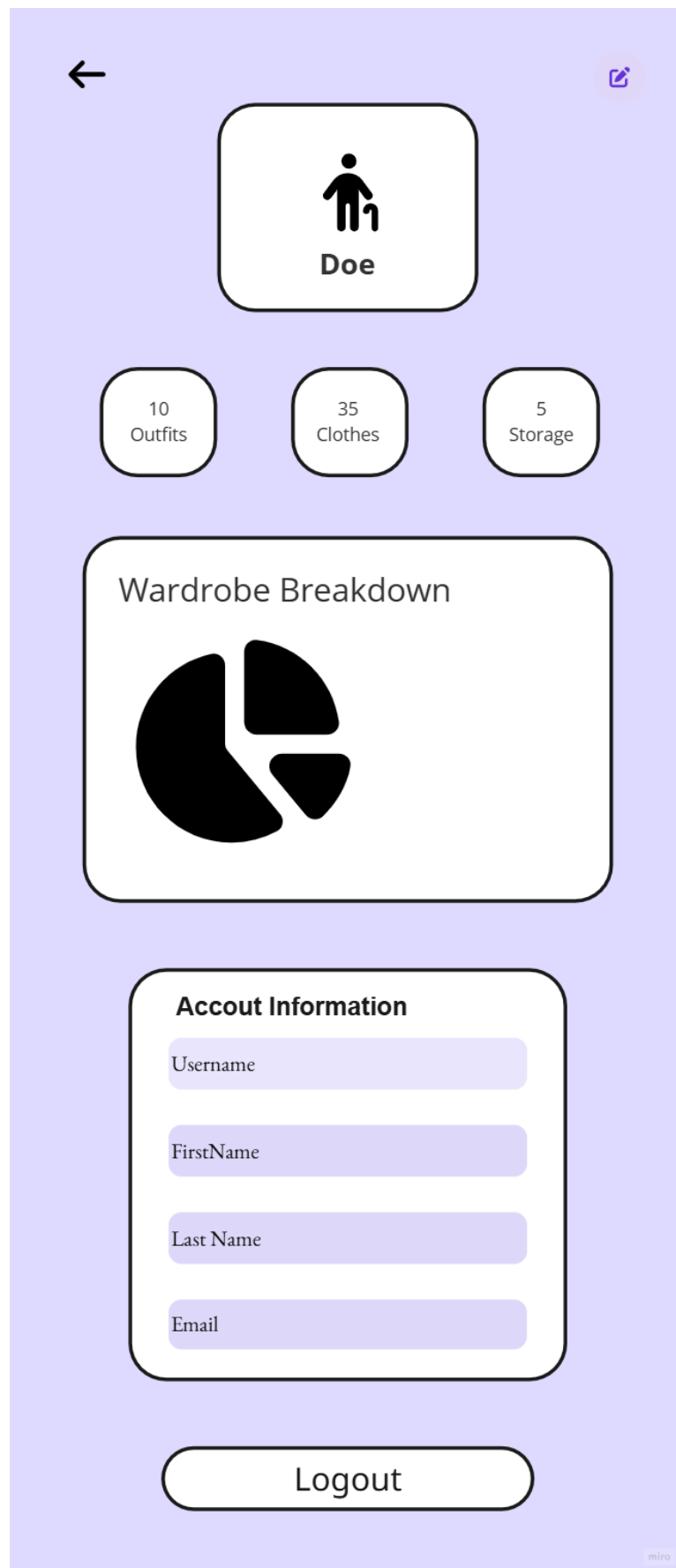
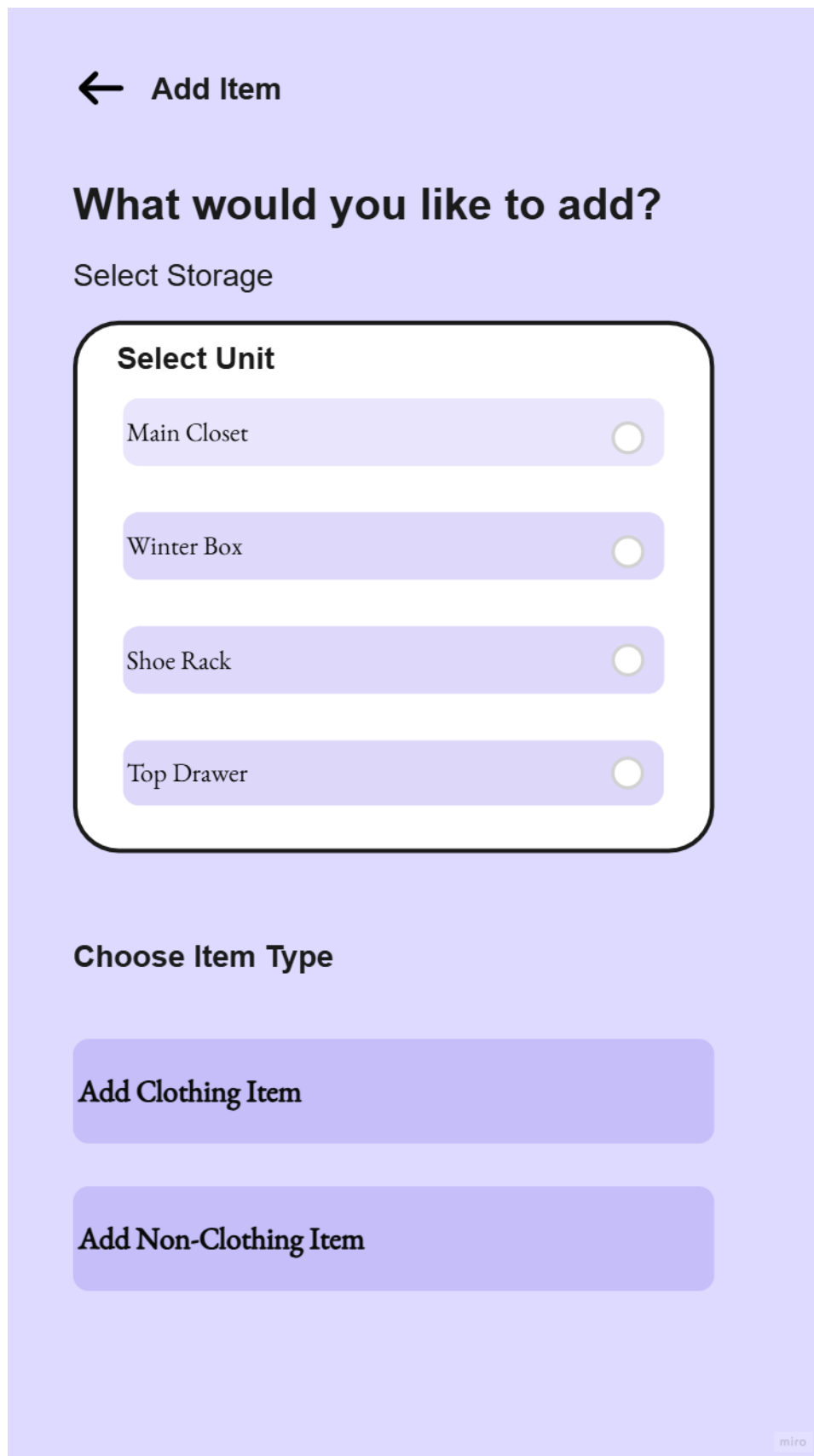


Figure 53: Profile Page Wireframe



The wireframe is set against a light purple background. At the top left, there is a back arrow icon followed by the text 'Add Item'. Below this is a large heading 'What would you like to add?'. Under the heading is the text 'Select Storage'. A white rounded rectangle with a black border contains the 'Select Unit' section. This section has a title 'Select Unit' and four radio button options: 'Main Closet', 'Winter Box', 'Shoe Rack', and 'Top Drawer'. Below the white box is the 'Choose Item Type' section, which contains two purple rounded rectangular buttons: 'Add Clothing Item' and 'Add Non-Clothing Item'. A small 'miro' logo is in the bottom right corner of the wireframe.

← Add Item

What would you like to add?

Select Storage

Select Unit

Main Closet ☐

Winter Box ☐

Shoe Rack ☐

Top Drawer ☐

Choose Item Type

Add Clothing Item

Add Non-Clothing Item

miro

Figure 54: Add Item Wireframe

← Add Non-Clothing Item

Add Item

Store Items

 Main Closet

Item Details

Item Name

Details

miro

Figure 55: Add Non-Clothing Item Wireframe

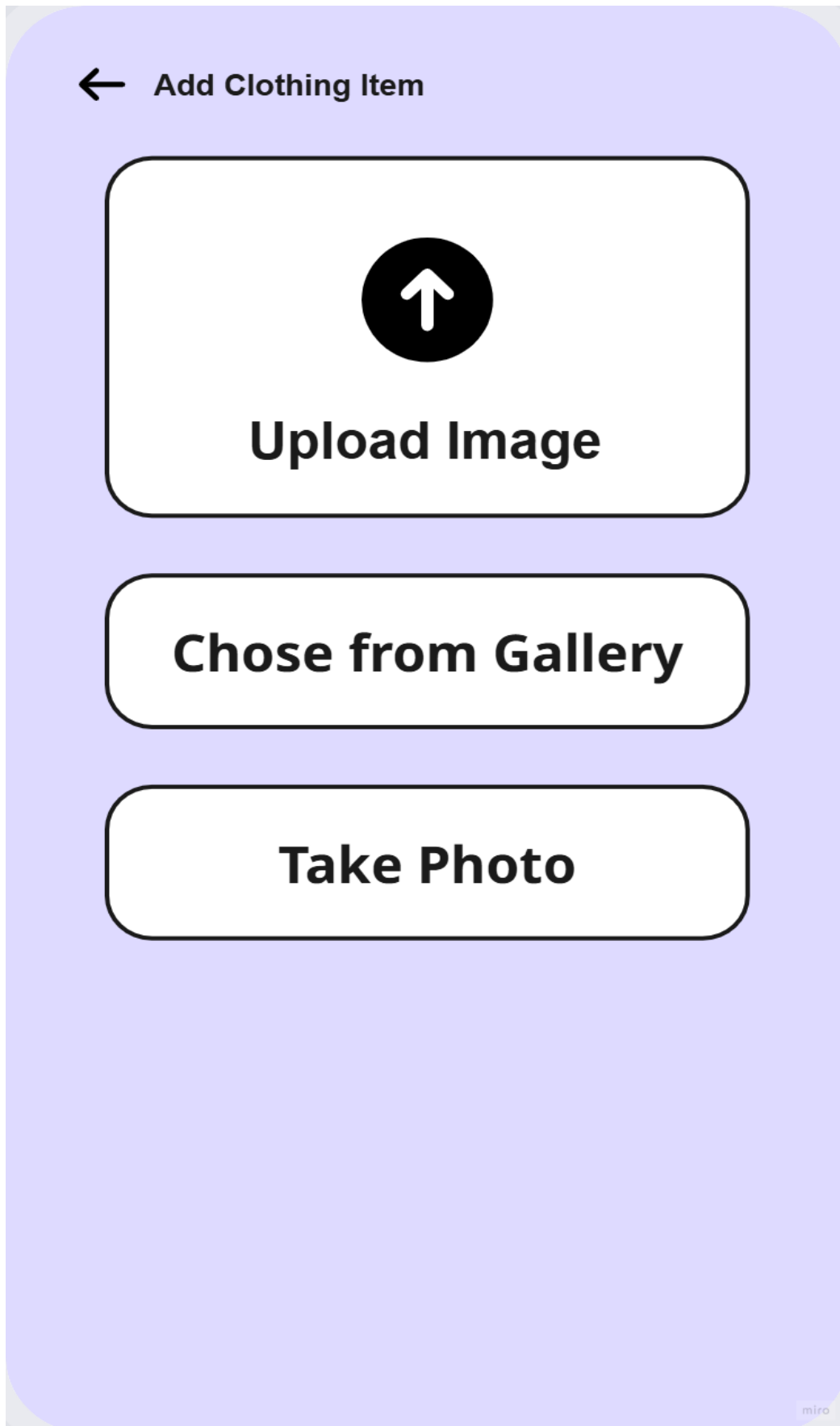


Figure 56: Upload Clothing Wireframe



Figure 57: Segmentation Screen Wireframe

The wireframe illustrates the 'Add Clothing Item' interface. At the top, a back arrow and the title 'Add Clothing Item' are positioned. Below this is a large white rounded rectangle containing a black icon of a vest. Underneath the icon is a section titled 'Extracted Metadata' in bold. This section contains five input fields: 'Category', 'Sub-Category', 'Dominant Color', 'Secondary Color', and 'Occasion'. The 'Dominant Color' and 'Secondary Color' fields are placed side-by-side. At the bottom of the screen are two buttons: 'Cancel' and 'Save'. A small 'miro' logo is visible in the bottom right corner of the purple background.

Figure 58: Metadata display Wireframe

[Go Back](#)

9.10 UI Design Screenshots

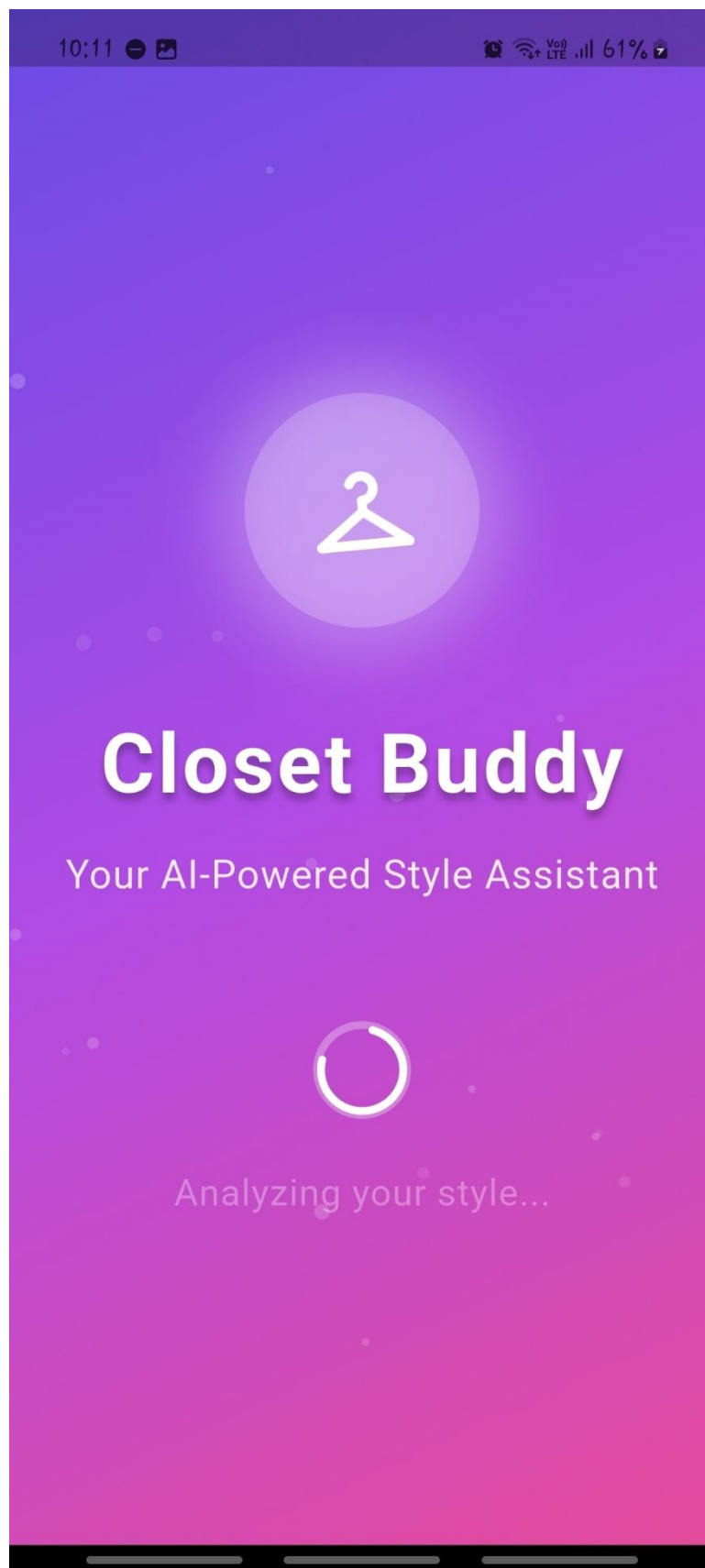


Figure 59: Loading Screen UI

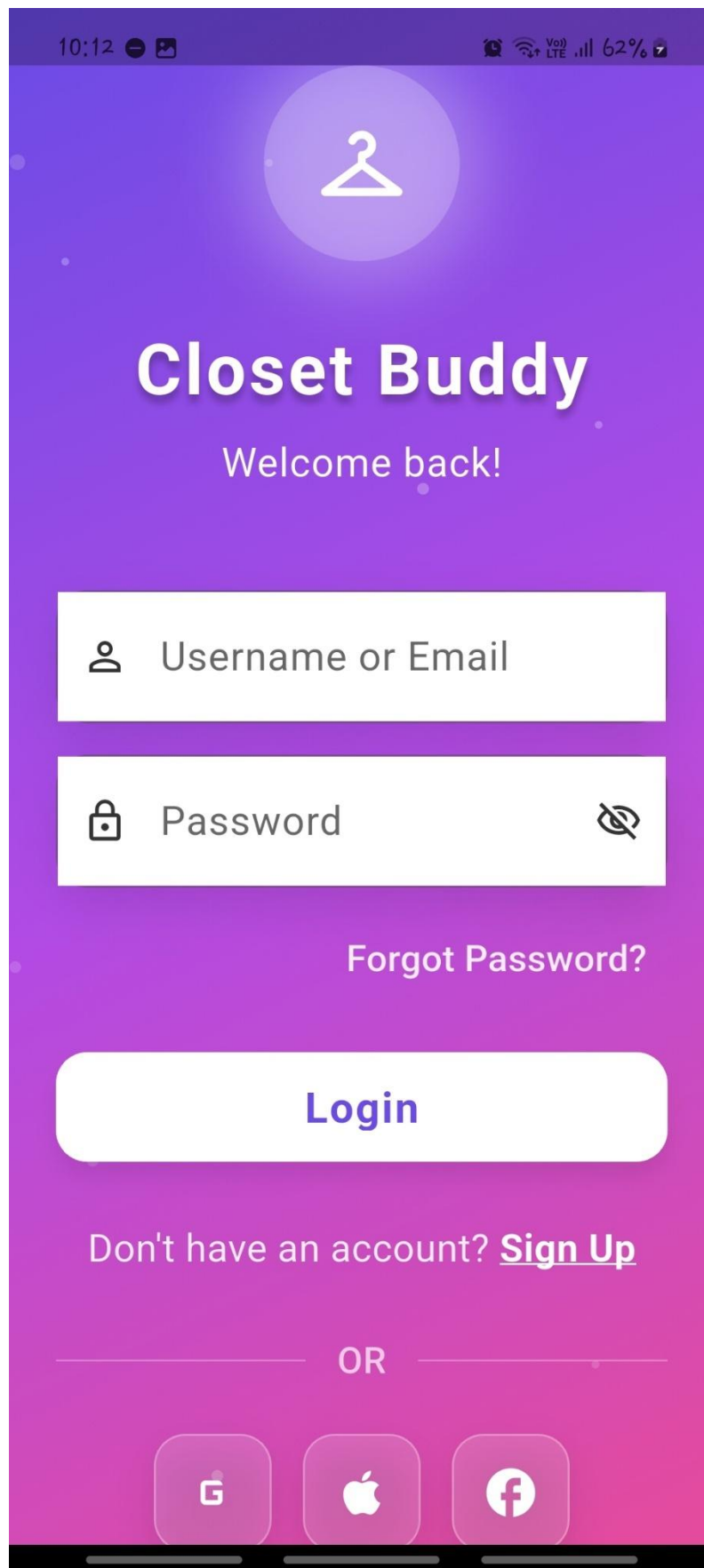
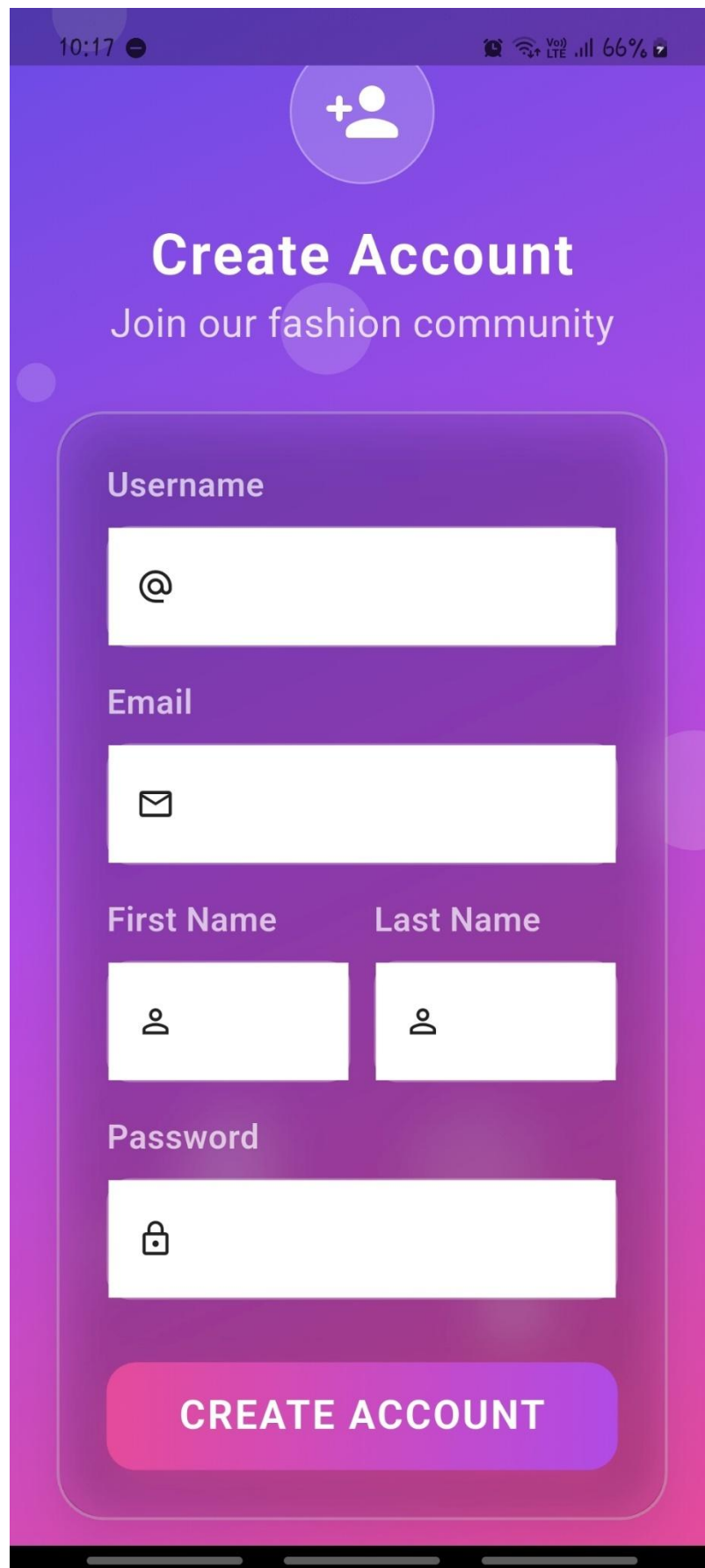


Figure 60: Login Screen UI



10:17 100% VoLTE 66%

+ person

Create Account

Join our fashion community

Username

@

Email

✉

First Name Last Name

person person

Password

🔒

CREATE ACCOUNT

Figure 61: Registration Page

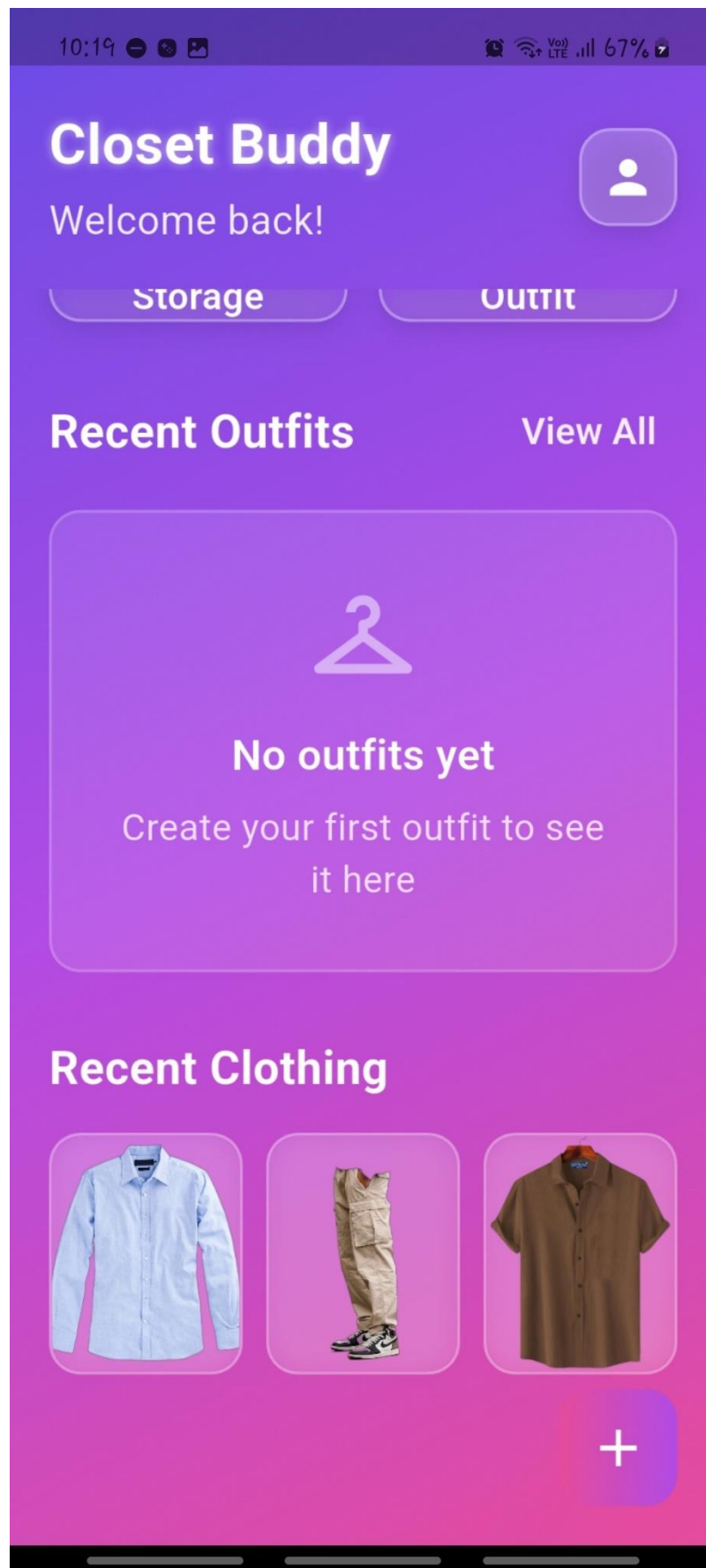


Figure 62: Homepage

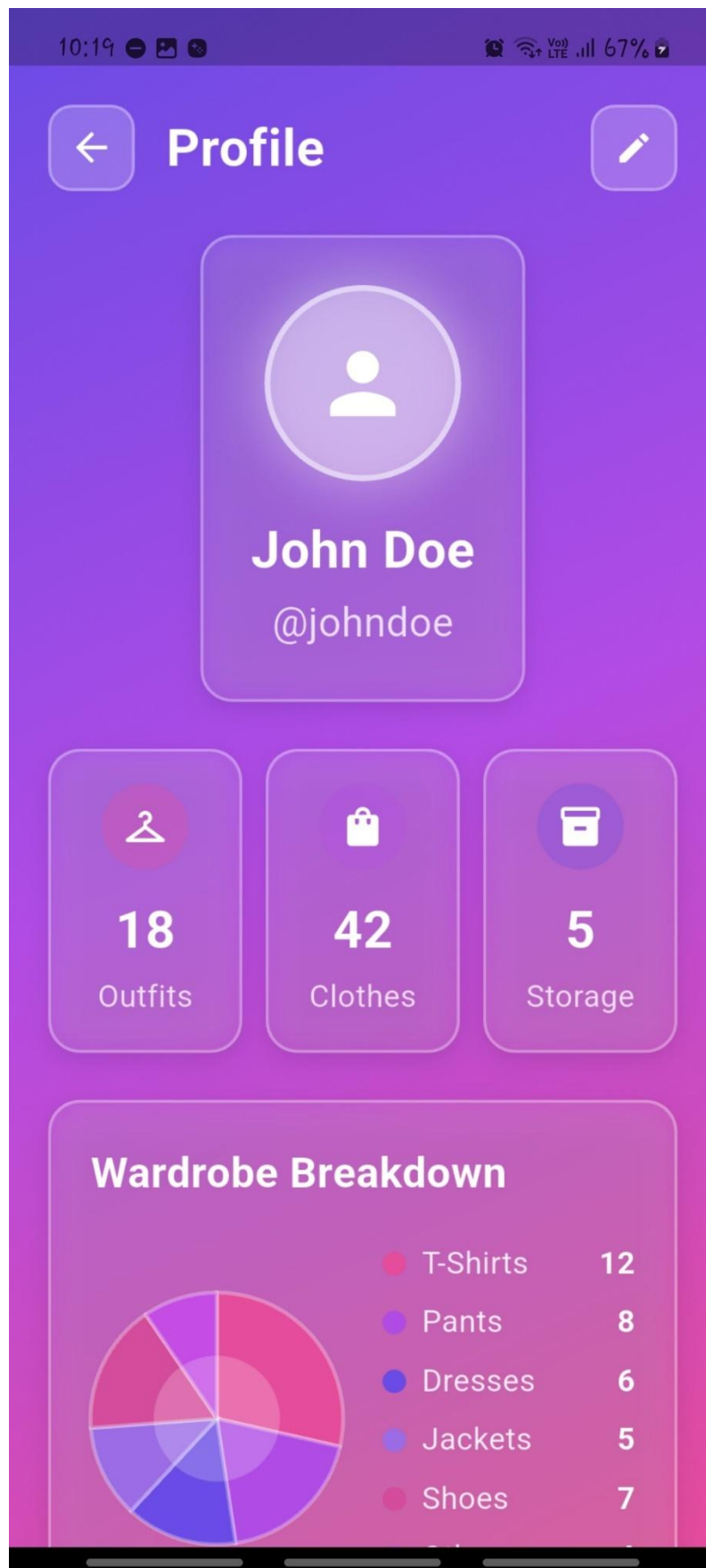


Figure 63: Profile Page 1

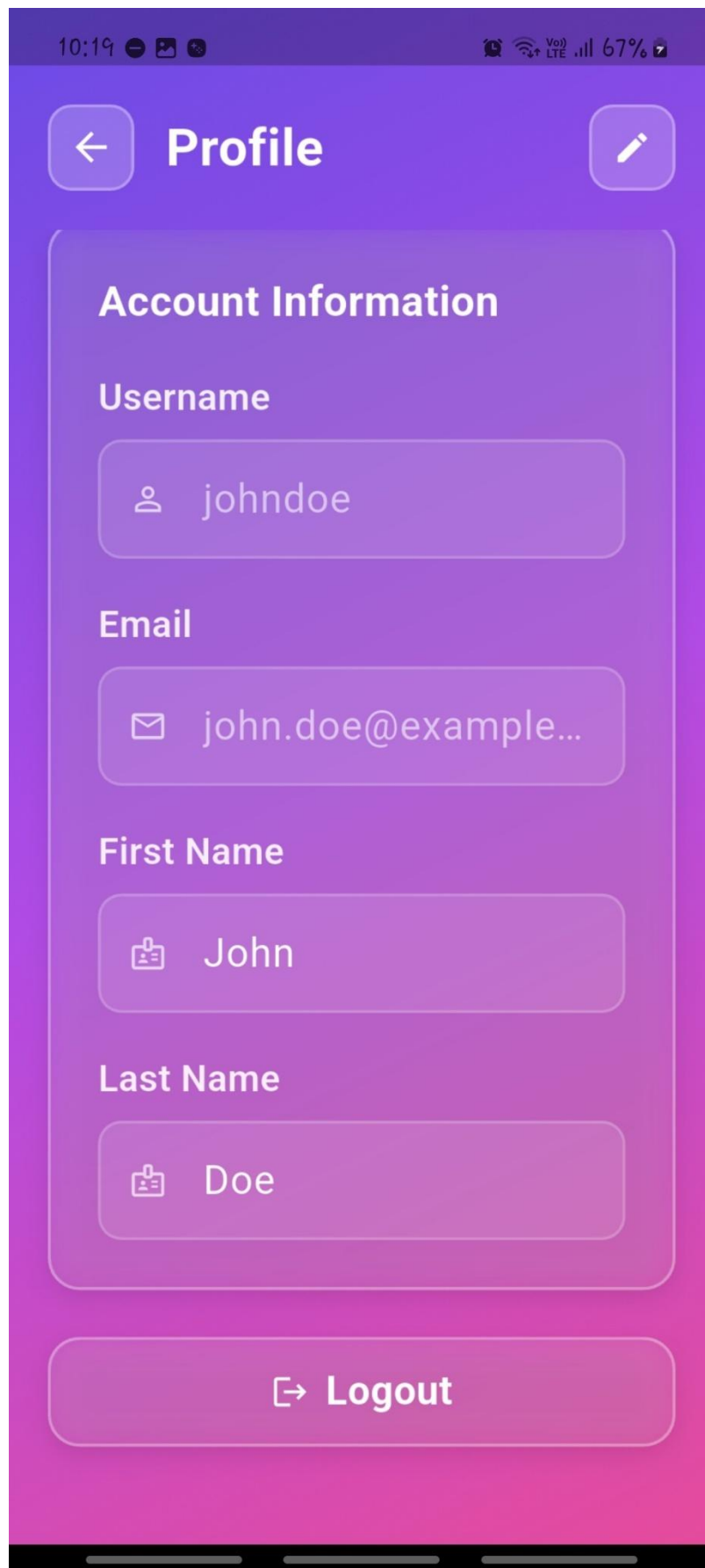
A mobile application interface for a user profile. The background is a gradient of purple and pink. At the top, a status bar shows the time 10:19, signal strength, VoLTE, and 67% battery. Below the status bar is a header with a back arrow icon on the left, the title 'Profile' in the center, and an edit icon (pencil) on the right. The main content area is a rounded rectangle with a light purple background. It contains the title 'Account Information' followed by four input fields: 'Username' with the value 'johndoe', 'Email' with the value 'john.doe@example...', 'First Name' with the value 'John', and 'Last Name' with the value 'Doe'. Each input field has a corresponding icon (person, envelope, and name tags) to its left. At the bottom of the main content area is a large button with an exit icon and the text 'Logout'.

Figure 64: Profile Page 2

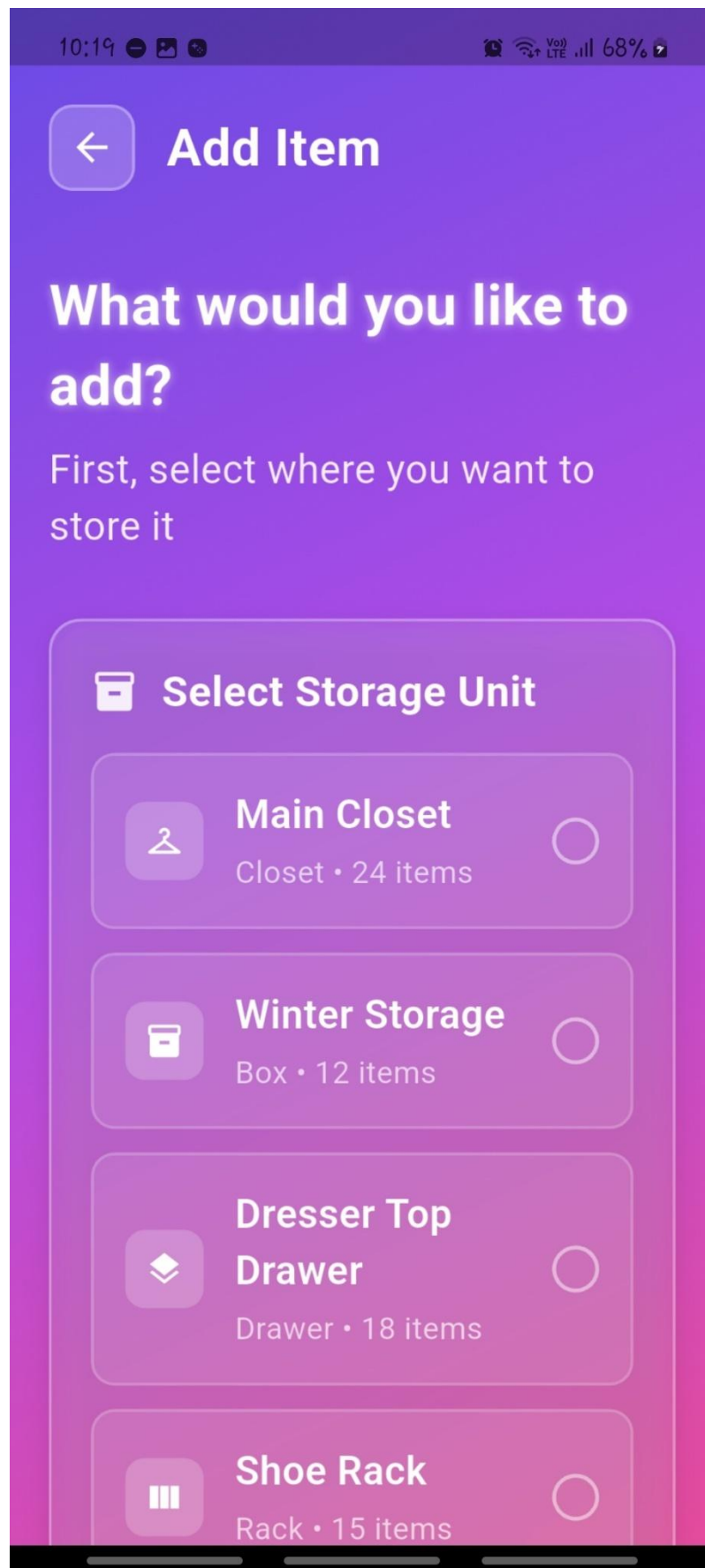


Figure 65: Add Item Page1

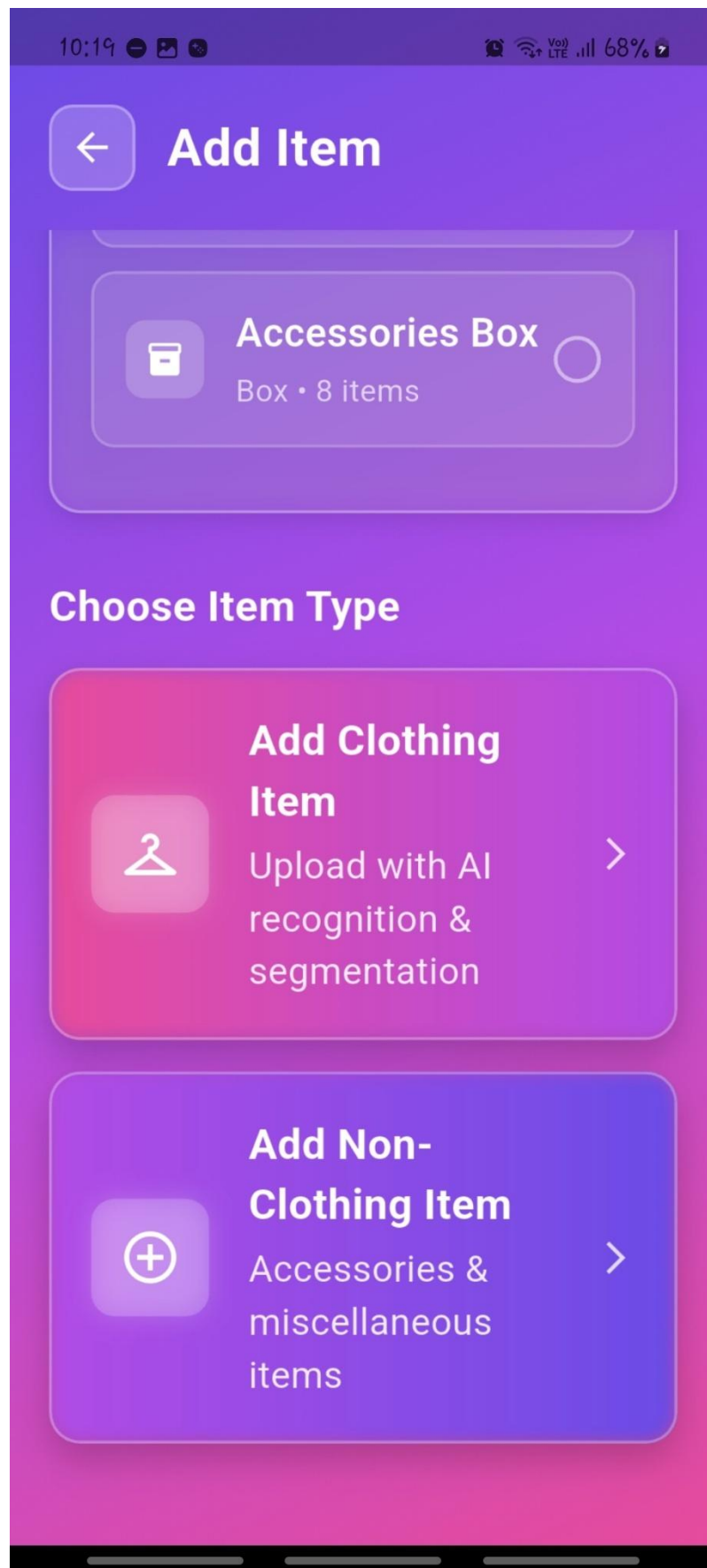


Figure 66: Add Item Page 2

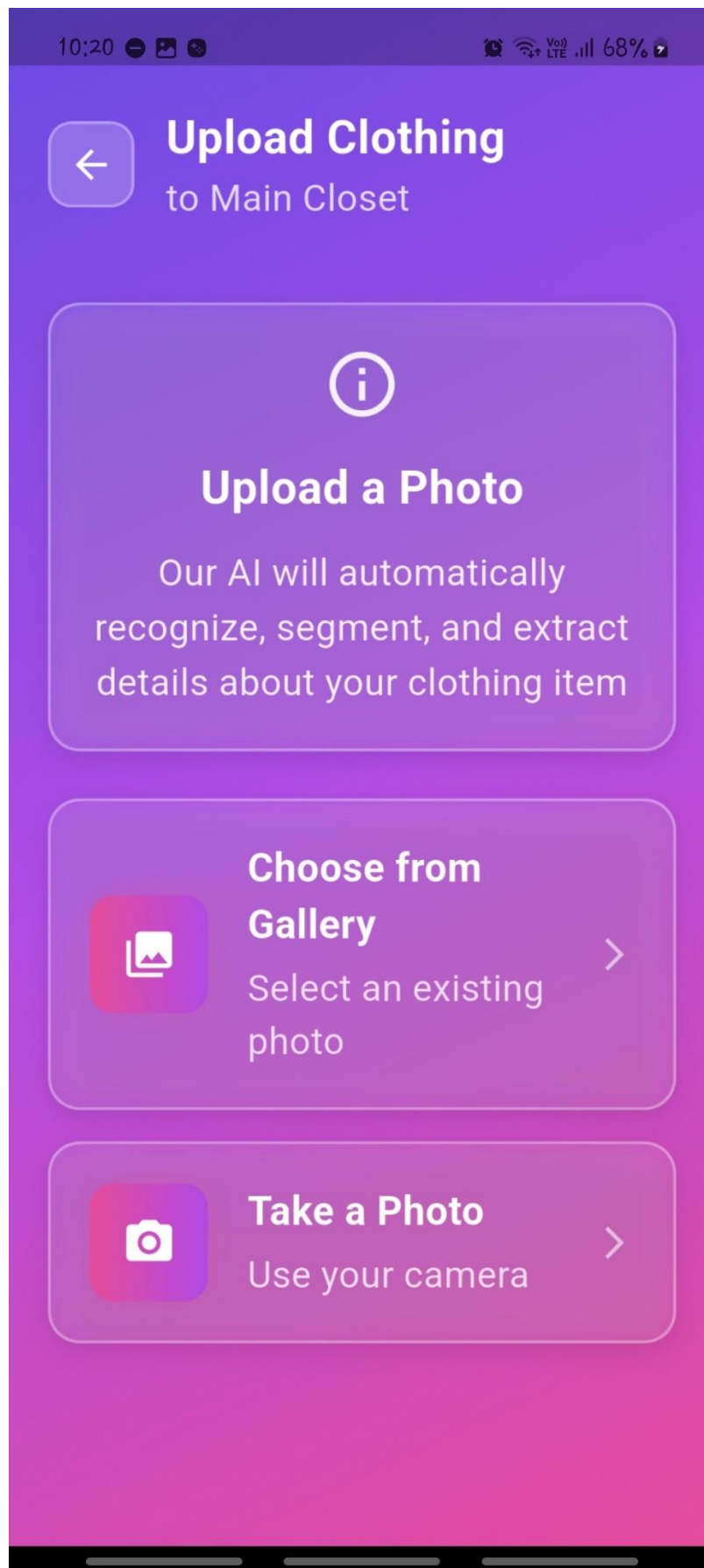


Figure 67: Upload Clothing page

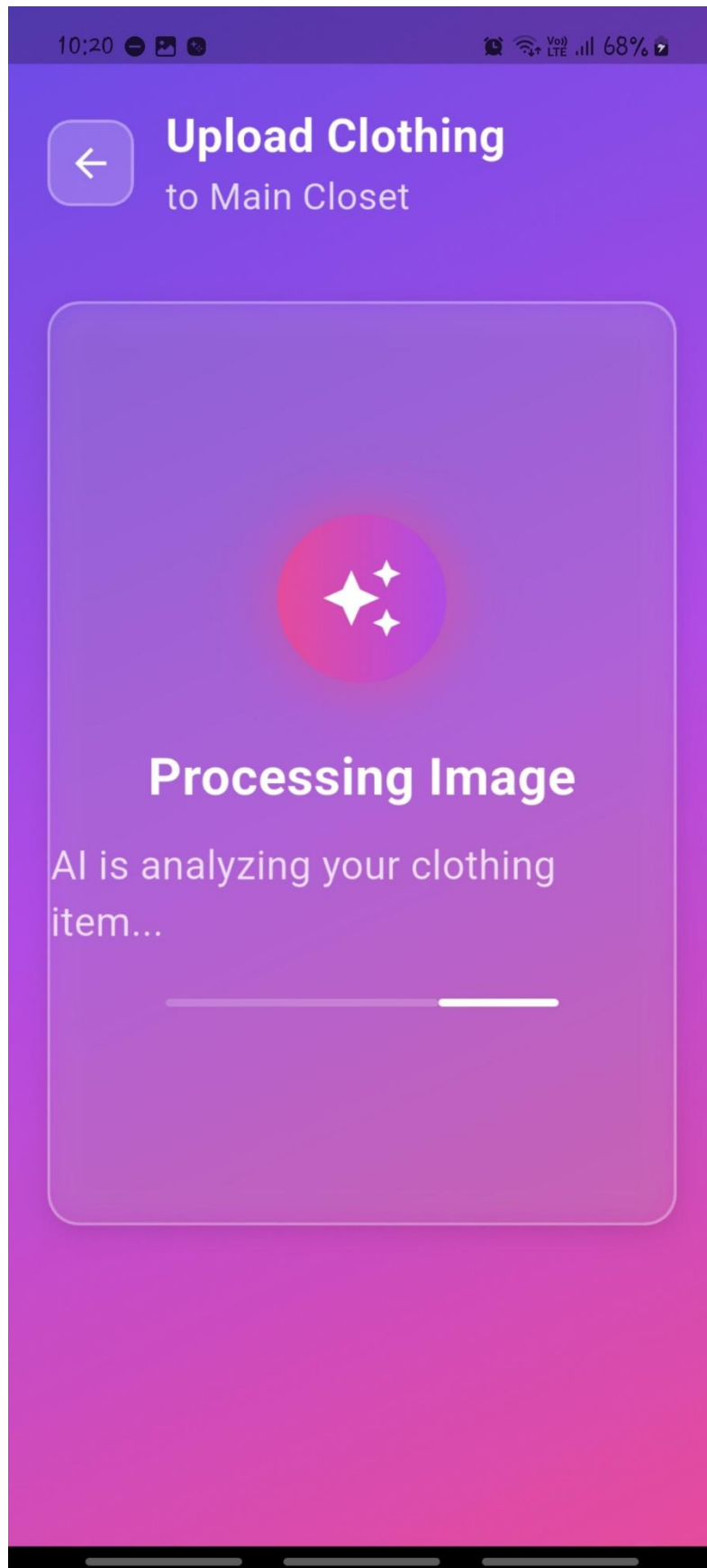


Figure 68: Processing Image

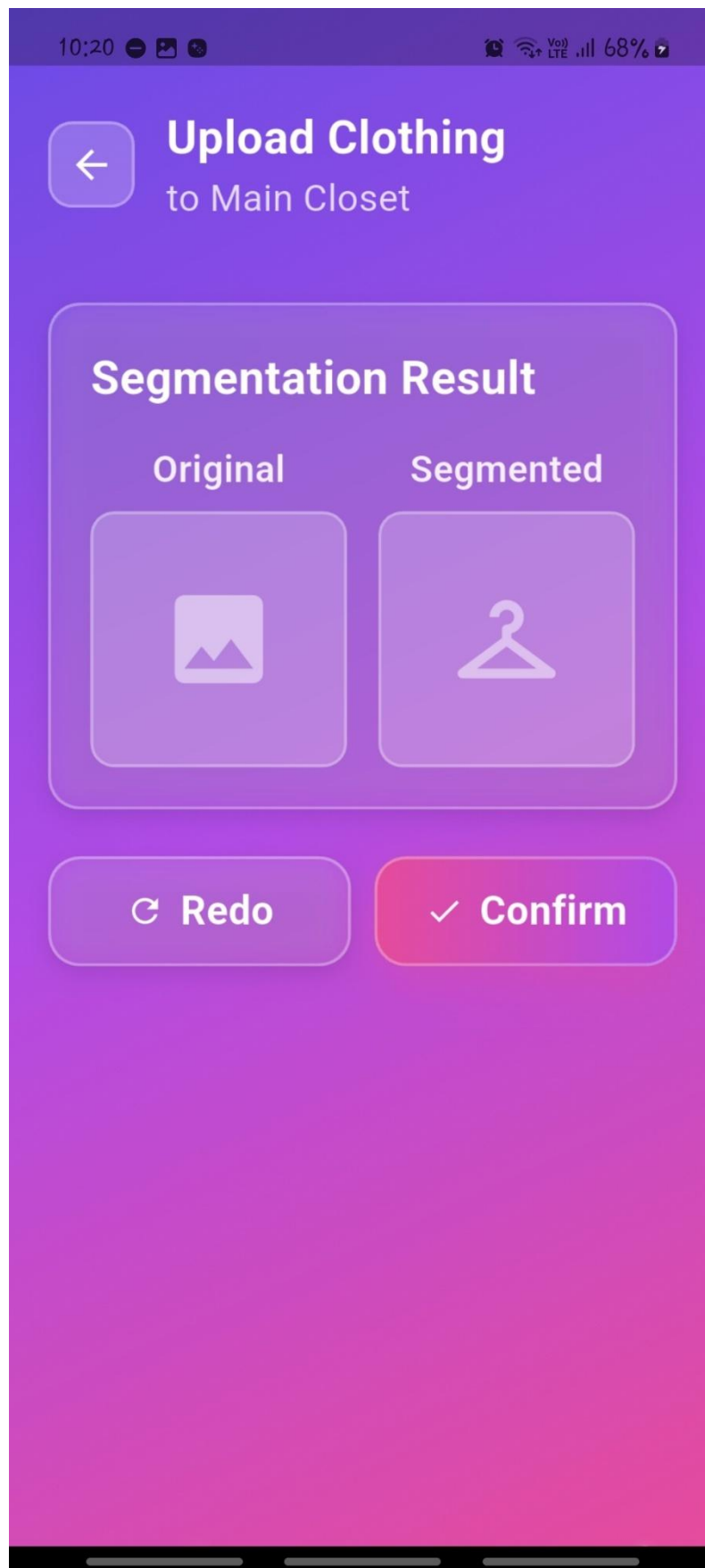


Figure 69: Confirm Segmentation Page

The screenshot shows a mobile application interface for uploading clothing. At the top, there is a status bar with the time 10:20, signal strength, VoLTE, and 68% battery. Below the status bar is a header with a back arrow icon and the text 'Upload Clothing to Main Closet'. The main content area is titled 'Extracted Metadata' with a star icon. It contains several sections: 'Category' with a topwear icon and the text 'Topwear'; 'Sub-Category' with a t-shirt icon and the text 'T-Shirt'; 'Dominant Color' with a palette icon and the text 'Blue'; 'Secondary Color' with a palette icon and the text 'White'; and 'Occasion' with a calendar icon and the text 'Casual'. At the bottom, there are two buttons: 'Cancel' and 'Save Item' with a checkmark icon.

10:20 68%

← **Upload Clothing**
to Main Closet

✦ **Extracted Metadata**

Category

▲ Topwear

Sub-Category

👕 T-Shirt

Dominant Color

🎨 Blue

Secondary Color

🎨 White

Occasion

📅 Casual

Cancel Save Item

Figure 70: Clothing Metadata Page

 **Add Non-Clothing Item**

Add Accessory or Item

Store accessories, shoes, or miscellaneous items



Storage Location

Main Closet



Item Details

Item Name 

 e.g., Leather Watch, Sunglasses

Description

Add any additional details (optional)



 **Cancel**

 **Add Item**

Figure 71: Add Non-Clothing page

9.11 Development Evidence

9.11.1 Frontend Development

Folder Structure:

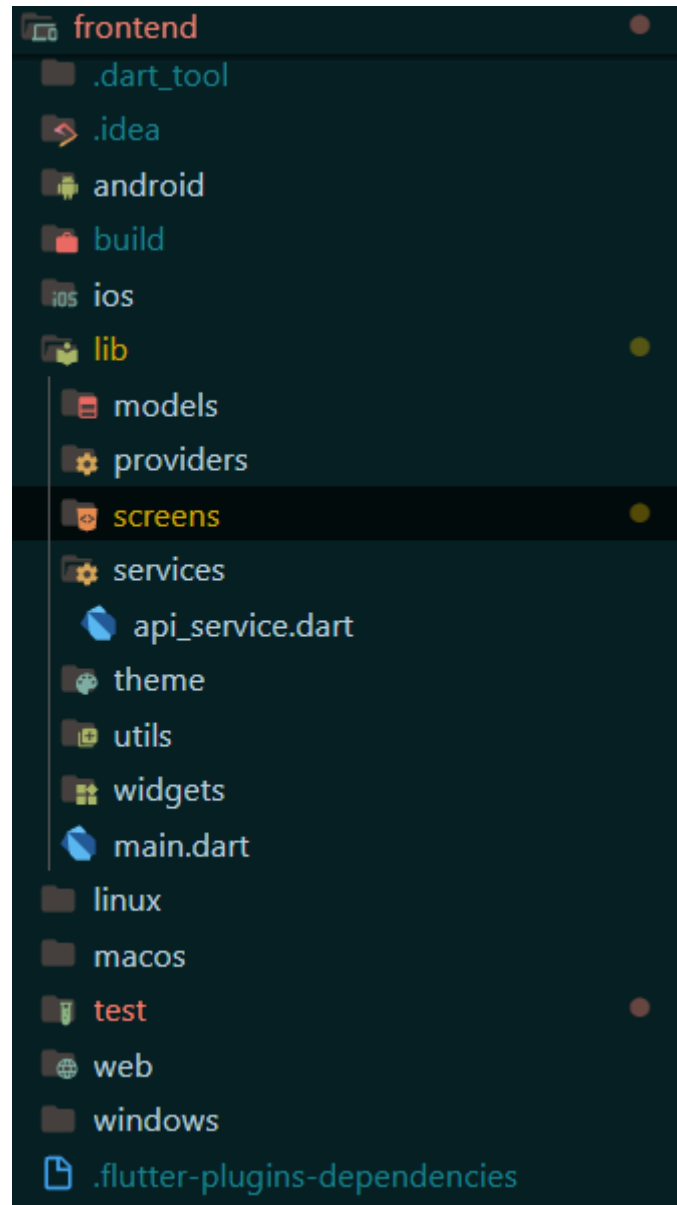


Figure 72: Frontend folder structure

Frontend logic for registration and login:

```
import 'dart:io';
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';

class ApiService {
  // BASE URLS
  static const String host = 'http://127.0.0.1:8000';
  static const String baseUrl = '$host/api/auth';

  // AUTH
  Future<bool> register(
    String username,
    String email,
    String password,
    String firstName,
    String lastName,
  ) async {
    final response = await http.post(
      Uri.parse('$baseUrl/register/'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'username': username,
        'email': email,
        'password': password,
        'first_name': firstName,
        'last_name': lastName,
      })),
    );
    return response.statusCode == 201;
  }

  Future<bool> login(String username, String password) async {
    final response = await http.post(
      Uri.parse('$baseUrl/login/'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({'username': username, 'password': password}),
    );

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body);
      await saveTokens(data['access'], data['refresh']);
      return true;
    }
    return false;
  }
}
```

Figure 73: Login registration code

Logic for session management:

```
Future<void> saveTokens(String access, String refresh) async {  
    final prefs = await SharedPreferences.getInstance();  
    await prefs.setString('access_token', access);  
    await prefs.setString('refresh_token', refresh);  
}  
  
Future<String?> getAccessToken() async {  
    final prefs = await SharedPreferences.getInstance();  
    return prefs.getString('access_token');  
}  
  
Future<String?> getRefreshToken() async {  
    final prefs = await SharedPreferences.getInstance();  
    return prefs.getString('refresh_token');  
}  
  
Future<void> logout() async {  
    final prefs = await SharedPreferences.getInstance();  
    await prefs.remove('access_token');  
    await prefs.remove('refresh_token');  
}  
  
Future<bool> refreshToken() async {  
    final refresh = await getRefreshToken();  
    if (refresh == null) return false;  
  
    final response = await http.post(  
        Uri.parse('$baseUrl/refresh/'),  
        headers: {'Content-Type': 'application/json'},  
        body: jsonEncode({'refresh': refresh}),  
    );  
  
    if (response.statusCode == 200) {  
        final data = jsonDecode(response.body);  
        final prefs = await SharedPreferences.getInstance();  
        await prefs.setString('access_token', data['access']);  
        return true;  
    }  
    return false;  
}
```

Figure 74: Session management code

```
Future<http.Response> _getWithRefresh(String url) async {
  String? token = await getAccessToken();
  var response = await http.get(
    Uri.parse(url),
    headers: {'Authorization': 'Bearer $token'},
  );

  if (response.statusCode == 401) {
    if (await refreshToken()) {
      token = await getAccessToken();
      return await http.get(
        Uri.parse(url),
        headers: {'Authorization': 'Bearer $token'},
      );
    }
  }
  return response;
}

Future<http.Response> _postWithRefresh(
  String url,
  Map<String, dynamic> body,
) async {
  String? token = await getAccessToken();
  var response = await http.post(
    Uri.parse(url),
    headers: {
      'Authorization': 'Bearer $token',
      'Content-Type': 'application/json',
    },
    body: jsonEncode(body),
  );

  if (response.statusCode == 401) {
    if (await refreshToken()) {
      token = await getAccessToken();
      return await http.post(
        Uri.parse(url),
        headers: {
          'Authorization': 'Bearer $token',
          'Content-Type': 'application/json',
        },
        body: jsonEncode(body),
      );
    }
  }
  return response;
}
```

Figure 75: Session management code 2

Logic for fetching user and admin data:

```
Future<Map<String, dynamic>>> fetchProfile() async {
  String? token = await getAccessToken();
  if (token == null) return null;

  var response = await http.get(
    Uri.parse('$baseUrl/profile/'),
    headers: {'Authorization': 'Bearer $token'},
  );

  if (response.statusCode == 401) {
    final refreshed = await refreshToken();
    if (!refreshed) return null;

    token = await getAccessToken();
    response = await http.get(
      Uri.parse('$baseUrl/profile/'),
      headers: {'Authorization': 'Bearer $token'},
    );
  }

  if (response.statusCode == 200) {
    return jsonDecode(response.body);
  }
  return null;
}

Future<Map<String, dynamic>>> fetchAdminData() async {
  final token = await getAccessToken();
  if (token == null) return null;

  final response = await http.get(
    Uri.parse('$baseUrl/admin-dashboard/'),
    headers: {'Authorization': 'Bearer $token'},
  );

  if (response.statusCode == 200) {
    return jsonDecode(response.body);
  }
  return null;
}
```

Figure 76: Code to Fetch user information

Logic for Segmentation:

```
// SEGMENTATION
Future<String?> segmentImage(File image) async {
  String? accessToken = await getAccessToken();
  if (accessToken == null) return null;

  Future<http.StreamedResponse> _sendRequest(String token) async {
    final request = http.MultipartRequest(
      'POST',
      Uri.parse('$baseUrl/segment/'),
    );
    request.headers['Authorization'] = 'Bearer $token';
    request.files.add(await http.MultipartFile.fromPath('image', image.path));
    return request.send();
  }

  http.StreamedResponse response = await _sendRequest(accessToken);
  if (response.statusCode == 401) {
    final refreshed = await refreshToken();
    if (!refreshed) {
      await logout();
      return null;
    }
    accessToken = await getAccessToken();
    if (accessToken == null) return null;
    response = await _sendRequest(accessToken);
  }

  final body = await response.stream.bytesToString();
  if (response.statusCode == 200) {
    final data = jsonDecode(body);
    final String relativeUrl = data['segmented_image'];
    return '$host$relativeUrl';
  } else {
    print(
      'Segmentation API failed. Status: ${response.statusCode}, Body: $body',
    );
    return null;
  }
}
```

Figure 77: Image segmentation backend

Deleting segmented image logic for “redo”:

```
Future<bool> deleteSegmentedImage(String segmentedUrl) async {
  final token = await getAccessToken();
  if (token == null) return false;

  try {
    final response = await http.post(
      Uri.parse('$baseUrl/delete-segmented/'),
      headers: {
        'Content-Type': 'application/json',
        'Authorization': 'Bearer $token',
      },
      body: jsonEncode({'url': segmentedUrl}),
    );
    return response.statusCode == 200;
  } catch (e) {
    print('Error deleting segmented image: $e');
    return false;
  }
}
```

Figure 78: Delete segmented image code

Logic for extracting metadata:

```
Future<Map<String, dynamic>> extractMetadata(String segmentedUrl) async {
  // Use the helper you wrote to handle 401 errors automatically
  final response = await _postWithRefresh('$baseUrl/ai/extract-metadata/', {
    'segmented_image': segmentedUrl,
  });

  if (response.statusCode == 200) {
    return jsonDecode(response.body);
  } else {
    print(
      'Failed to extract metadata: ${response.statusCode} ${response.body}',
    );
    return {};
  }
}
```

Figure 79: metadata extraction code

Loading page code:

```
frontend > lib > screens > loading_screen.dart > _LoadingScreenState > initState > checkAuth
1  import 'package:flutter/material.dart';
2  import '../services/api_service.dart';
3  import 'home_screen.dart';
4  import 'login_screen.dart';
5  import 'dart:math' as math;
6
7  class LoadingScreen extends StatefulWidget {
8    const LoadingScreen({super.key});
9
10   @override
11   State<LoadingScreen> createState() => _LoadingScreenState();
12 }
13
14 class _LoadingScreenState extends State<LoadingScreen>
15   with TickerProviderStateMixin {
16   final ApiService api = ApiService();
17   late AnimationController _rotationController;
18   late AnimationController _fadeController;
19   late AnimationController _scaleController;
20   late Animation<double> _fadeAnimation;
21   late Animation<double> _scaleAnimation;
22
23   @override
24   void initState() {
25     super.initState();
26
27     void goTo(Widget page) {
28       Navigator.pushReplacement(
29         context,
30         MaterialPageRoute(builder: (_) => page),
31       );
32     }
33
34     Future<void> checkAuth() async {
35       await Future.delayed(const Duration(seconds: 4));
36     }
37   }
38 }
```

Figure 80: Loading page code

Login page code:

```
import 'package:flutter/material.dart';
import '../services/api_service.dart';
import 'home_screen.dart';
import 'register_screen.dart';
import 'dart:math' as math;

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen>
    with TickerProviderStateMixin {
  final _formKey = GlobalKey<FormState>();
  final _username = TextEditingController();
  final _password = TextEditingController();
  final api = ApiService();

  bool _obscurePassword = true;
  bool _isLoading = false;

  late AnimationController _fadeController;
  late AnimationController _slideController;
  late Animation<double> _fadeAnimation;
  late Animation<Offset> _slideAnimation;

  @override
  void initState() {
    super.initState();

    // Initialize animations
    _fadeController = AnimationController(
      duration: const Duration(milliseconds: 1000),
      vsync: this,
    ); // AnimationController
```

Figure 81: Login page code

Register page code

```
frontend > lib > screens > register_screen.dart > _RegisterScreenState
1  import 'package:flutter/material.dart';
2  import 'dart:ui';
3  import 'dart:math' as math;
4  import '../services/api_service.dart';
5
6  class RegisterScreen extends StatefulWidget {
7    const RegisterScreen({super.key});
8
9    @override
10   State<RegisterScreen> createState() => _RegisterScreenState()
11 }
12
13 class _RegisterScreenState extends State<RegisterScreen>
14   with TickerProviderStateMixin {
15   final api = ApiService();
16   final _username = TextEditingController();
17   final _email = TextEditingController();
18   final _password = TextEditingController();
19   final _first = TextEditingController();
20   final _last = TextEditingController();
21   late AnimationController _floatingController;
22
23   @override
24   void initState() {
25     super.initState();
26     _floatingController = AnimationController(
27       duration: const Duration(seconds: 10),
28       vsync: this,
29     )..repeat(); // AnimationController
30   }
31
32   @override
33   void dispose() {
34     _floatingController.dispose();
35     _username.dispose();
36     _email.dispose();
37     _password.dispose();
```

Figure 82: Registration page code

Profile page code (incomplete):

```
import 'package:flutter/material.dart';
import '../services/api_service.dart';
import 'login_screen.dart';
import '../widgets/primary_buttons.dart';

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});

  Widget infoTile(String label, String value) {
    return Container(
      margin: const EdgeInsets.only(bottom: 12),
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(14),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.05),
            blurRadius: 10,
            offset: const Offset(0, 4),
          ), // BoxShadow
        ],
      ), // BoxDecoration
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Text(label, style: const TextStyle(fontWeight: FontWeight.w500)),
          Text(value, style: TextStyle(color: Colors.grey.shade600)),
        ],
      ), // Row
    ); // Container
  }

  Future<void> _logout(BuildContext context) async {
    final api = ApiService();
    await api.logout();

    if (!context.mounted) return;

    Navigator.of(context).pushAndRemoveUntil(
      MaterialPageRoute(builder: (_) => const LoginScreen()),
      (route) => false,
    );
  }
}
```

Figure 83: Profile page code

Back.

9.11.2 Backend Development

Folder Structure:

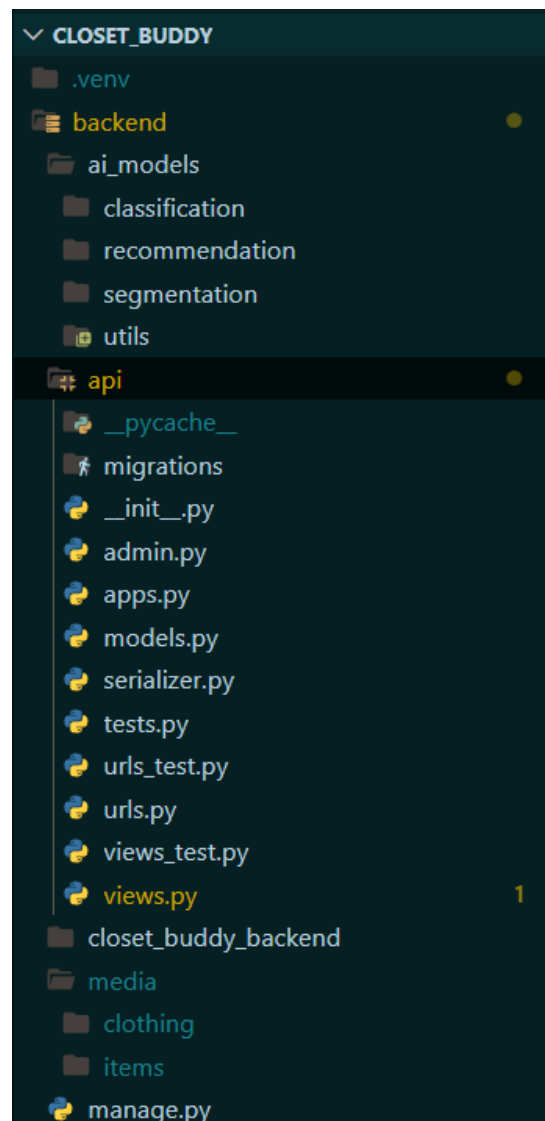


Figure 84: Backend folder structure

For user authentication, uses the jwt tokens in url.py:

```
urlpatterns = [
    # Auth
    path('register/', RegisterView.as_view(), name='register'),
    path('login/', TokenObtainPairView.as_view(), name='login'),
    path('refresh/', TokenRefreshView.as_view(), name='token_refresh'),
    path('logout/', logout, name='logout'),|
```

Figure 85: User authentication with jwt

Serializer code for registration and clothing items:

```
from rest_framework import serializers
from .models import ClothingItem, NonClothingItem, StorageUnit
from django.contrib.auth.models import User

class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, min_length=6)

    class Meta:
        model = User
        fields = ('username', 'email', 'password', 'first_name', 'last_name')

    def create(self, validated_data):
        user = User.objects.create_user(
            username=validated_data['username'],
            email=validated_data['email'],
            password=validated_data['password'],
            first_name=validated_data.get('first_name', ''),
            last_name=validated_data.get('last_name', '')
        )
        return user

class ClothingItemCreateSerializer(serializers.ModelSerializer):
    storage_id = serializers.PrimaryKeyRelatedField(
        queryset=StorageUnit.objects.all(),
        source="storage_unit",
        write_only=True
    )

    class Meta:
        model = ClothingItem
        fields = [
            "storage_id",
            "image",
            "category",
            "subcategory",
            "occasion",
            "dominant_color",
            "secondary_color",
        ]

    def create(self, validated_data):
        user = self.context["request"].user
        return ClothingItem.objects.create(user=user, **validated_data)
```

Figure 86: Serializer code 1

Serializer code for storage units and non-clothing item:

```
class StorageUnitSerializer(serializers.ModelSerializer):
    parent_storage = serializers.SerializerMethodField()

    class Meta:
        model = StorageUnit
        fields = ["id", "name", "description", "type", "parent_storage", "is_put_away"]

    def get_parent_storage(self, obj):
        if obj.parent_storage:
            return {
                "id": obj.parent_storage.id,
                "name": obj.parent_storage.name,
                "type": obj.parent_storage.type,
            }
        return None

class NonClothingItemSerializer(serializers.ModelSerializer):
    storage_unit = serializers.SerializerMethodField()

    class Meta:
        model = NonClothingItem
        fields = ['id', 'name', 'description', 'storage_unit', 'created_at']

    def get_storage_unit(self, obj):
        """Return minimal storage info"""
        return {
            "id": obj.storage_unit.id,
            "name": obj.storage_unit.name,
            "type": obj.storage_unit.type
        }
```

Figure 87: Serializer code 2

For registration, user information, and admin authentication:

```
class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    serializer_class = RegisterSerializer

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def profile(request):
    """Return user's profile info"""
    user = request.user
    role = "admin" if user.is_staff else "user"
    return Response({
        "username": user.username,
        "email": user.email,
        "first_name": user.first_name,
        "last_name": user.last_name,
        "role": role,
    })

@api_view(['POST'])
@permission_classes([IsAuthenticated])
def logout(request):
    """Blacklist JWT refresh token"""
    try:
        refresh_token = request.data.get("refresh")
        token = RefreshToken(refresh_token)
        token.blacklist()
    except Exception:
        pass
    return Response({"detail": "Logged out"})

# ----- ADMIN -----

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def admin_dashboard(request):
    """Admin-only dashboard data"""
    user = request.user
    if not user.is_staff:
        return Response({"detail": "Permission denied."}, status=403)

    return Response({
        "message": f"Welcome Admin {user.username}",
        "total_users": User.objects.count(),
    })
```

Figure 88: Backend code

Backend code for segmenting image and deleting the image upon pressing “redo” button:

```
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def segment_clothing(request):
    """Segment clothing image"""
    image = request.FILES.get('image')
    if not image:
        return Response({"error": "No image provided"}, status=400)

    segmented_url = segment_image(image)
    return Response({"segmented_image": segmented_url})

@api_view(['POST'])
@permission_classes([IsAuthenticated])
def delete_segmented_image(request):
    segmented_url = request.data.get('url') or request.data.get('segmented_image')

    if not segmented_url:
        return Response({"detail": "URL not provided."}, status=400)

    try:
        media_url = settings.MEDIA_URL
        if media_url in segmented_url:
            relative_path = segmented_url.split(media_url)[-1]
        else:
            relative_path = segmented_url

        file_path = os.path.join(settings.MEDIA_ROOT, relative_path)
        print(f"DEBUG: Attempting to delete file at: {file_path}")

        if os.path.exists(file_path):
            os.remove(file_path)
            return Response({"detail": "File deleted successfully."}, status=200)
        else:
            print(f"DEBUG: File not found at {file_path}")
            return Response({"detail": "File not found on server."}, status=404)

    except Exception as e:
        print(f"DEBUG: Deletion Error: {str(e)}")
        return Response({"detail": f"Error: {str(e)}"}, status=500)
```

Figure 89: Segmentation backend

Code for color extraction with dummy values for other fields:

```
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def extract_metadata(request):
    print(f"DEBUG: Incoming request data: {request.data}")
    segmented_url = request.data.get("segmented_image") or request.data.get("url")

    if not segmented_url:
        return Response({"error": "Key 'segmented_image' or 'url' is missing in request"}, status=400)

    try:
        media_url = settings.MEDIA_URL
        relative_path = segmented_url.split(media_url)[-1] if media_url in segmented_url else segmented_url
        image_path = Path(settings.MEDIA_ROOT) / relative_path

        if not image_path.exists():
            return Response({"error": f"Image not found at {image_path}"}, status=404)
        # color extraction logic
        colors = extract_colors_with_names(str(image_path))

        return Response({
            "dominant_color": colors.get("dominant_color", "Unknown"),
            "secondary_color": colors.get("secondary_color", "Unknown"),
            "category": "Topwear",
            "subcategory": "Shirt",
            "occasion": "Casual",
        }, status=200)
    except Exception as e:
        print(f"DEBUG: Extraction Error: {e}")
        return Response({"error": str(e)}, status=500)
```

Figure 90: Colour extraction backend

9.11.3 AI Development

Segmentation code:

```
from rembg import remove
from PIL import Image
import uuid
from django.conf import settings
from pathlib import Path

def segment_image(uploaded_image):
    input_image = Image.open(uploaded_image)

    output_image = remove(input_image)

    filename = f"{uuid.uuid4()}.png"
    output_path = Path(settings.MEDIA_ROOT) / "clothing" / filename

    output_image.save(output_path)

    return f"{settings.MEDIA_URL}clothing/{filename}"
```

Figure 91: Segmentation code

Color Extraction code:

```
import cv2
import numpy as np
from sklearn.cluster import KMeans
import webcolors

import webcolors

def get_color_name(rgb):
    """Finds the closest human-readable name for an RGB value."""
    try:
        return webcolors.rgb_to_name(rgb)
    except ValueError:
        min_colors = {}
        for name in webcolors.names():
            color_rgb = webcolors.name_to_rgb(name)
            rd = (color_rgb.red - rgb[0]) ** 2
            gd = (color_rgb.green - rgb[1]) ** 2
            bd = (color_rgb.blue - rgb[2]) ** 2
            min_colors[(rd + gd + bd)] = name
        return min_colors[min(min_colors.keys())]

def extract_colors_with_names(image_path):
    img = cv2.imread(image_path, cv2.IMREAD_UNCHANGED)
    if img is None:
        print(f"CRITICAL: OpenCV could not read file at {image_path}")
        return {"dominant_color": "Unknown", "secondary_color": "Unknown"}
    if img.shape[2] == 4:
        mask = img[:, :, 3] > 0
        pixels = img[mask][:, :3]
    else:
        pixels = img.reshape(-1, 3)
    if len(pixels) < 10:
        return {"dominant_color": "Empty", "secondary_color": "Empty"}

    # K-Means clustering
    pixels = cv2.cvtColor(np.array([pixels]), cv2.COLOR_BGR2RGB)[0]
    kmeans = KMeans(n_clusters=2, n_init=10)
    kmeans.fit(pixels)

    colors = kmeans.cluster_centers_.astype(int)

    return {
        "dominant_color": get_color_name(colors[0]),
        "secondary_color": get_color_name(colors[1]) if len(colors) > 1 else None
    }
```

Figure 92: Code for colour extraction

9.11.4 Migration and Model Creation

Model Creation:

```
from django.db import models
from django.contrib.auth.models import User
from django.core.exceptions import ValidationError
from django.core.validators import MinValueValidator, MaxValueValidator

class StorageUnit(models.Model):
    STORAGE_TYPE_CHOICES = [
        ("closet", "Closet"),
        ("wardrobe", "Wardrobe"),
        ("cupboard", "Cupboard"),
        ("drawer", "Drawer"),
        ("box", "Box"),
        ("shelf", "Shelf"),
        ("other", "Other"),
    ]
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name="storage_units")
    name = models.CharField(max_length=100)
    description = models.TextField(null=True, blank=True)
    type = models.CharField(
        max_length=30,
        choices=STORAGE_TYPE_CHOICES
    )
    parent_storage = models.ForeignKey(
        "self",
        null=True,
        blank=True,
        on_delete=models.SET_NULL,
        related_name="sub_units"
    )
    is_put_away = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.name} [{self.type}] - {self.user.username}"
```

Figure 93: Storage Unit Model

```

class ClothingItem(models.Model):
    user = models.ForeignKey(
        User,
        on_delete=models.CASCADE,
        related_name="clothing_items"
    )
    storage_unit = models.ForeignKey(
        StorageUnit,
        on_delete=models.PROTECT,
        related_name="clothes"
    )
    image = models.ImageField(upload_to="clothing/")
    category = models.CharField(max_length=50) # topwear, footwear
    subcategory = models.CharField(max_length=50) # shirt, jeans
    occasion = models.CharField(max_length=50, null=True, blank=True)
    dominant_color = models.CharField(max_length=30)
    secondary_color = models.CharField(max_length=30, null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.category} ({self.user.username})"
    def clean(self):
        if self.storage_unit.user != self.user:
            raise ValidationError("Storage unit must belong to the same user.")

```

Figure 94: Clothing Item Model

```

class NonClothingItem(models.Model):
    user = models.ForeignKey(
        User,
        on_delete=models.CASCADE,
        related_name="non_clothing_items"
    )
    storage_unit = models.ForeignKey(
        StorageUnit,
        on_delete=models.PROTECT,
        related_name="misc_items"
    )
    name = models.CharField(max_length=100)
    description = models.TextField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return self.name
    def clean(self):
        if self.storage_unit.user != self.user:
            raise ValidationError("Storage unit must belong to the same user.")

```

Figure 95: Non-Clothing Item Model

```

class Outfit(models.Model):
    user = models.ForeignKey(
        User,
        on_delete=models.CASCADE,
        related_name="outfits"
    )
    name = models.CharField(max_length=100)
    clothing_items = models.ManyToManyField(
        ClothingItem,
        related_name="outfits"
    )
    occasion = models.CharField(max_length=50, null=True, blank=True)
    rating = models.IntegerField(null=True, blank=True, validators=[MinValueValidator(1), MaxValueValidator(5)])
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return f"{self.name} ({self.user.username})"

```

Figure 96: Outfit Model

Making Python migrations:

```

(closet-buddy) PS C:\Users\acer\Desktop\Personal\closet_buddy\backend> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, api, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying api.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK

```

Figure 97: Successfully Migrating the database

Databases being made in PostgreSQL:

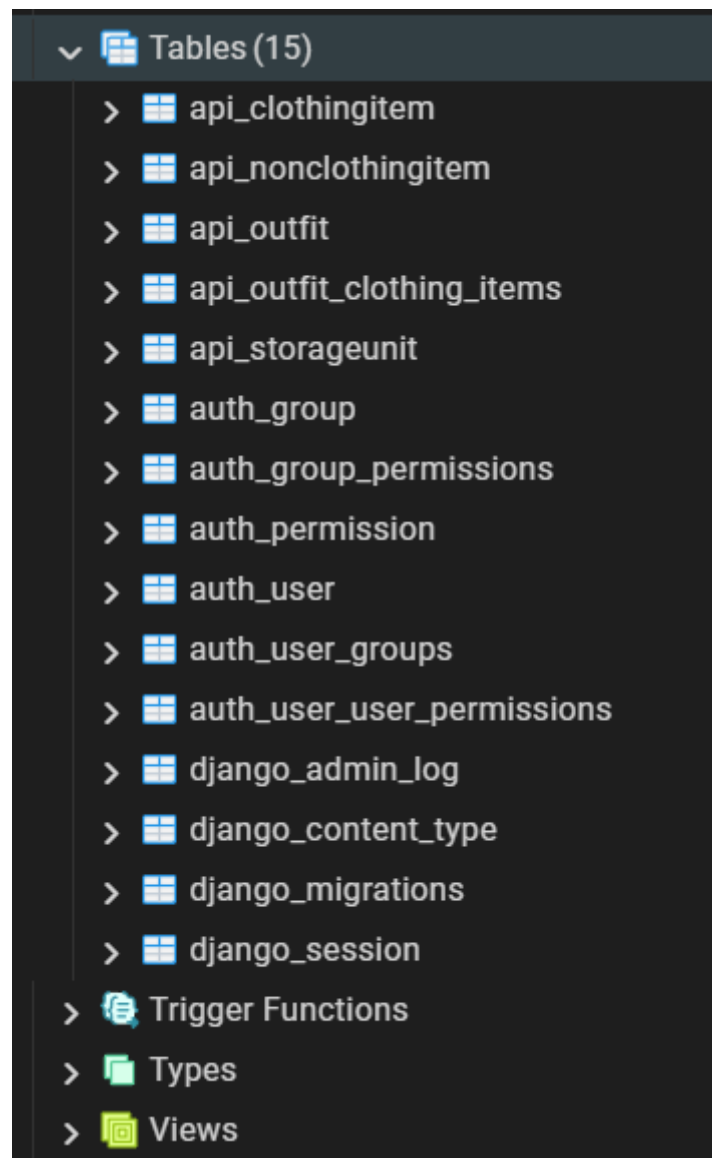


Figure 98: Database view in PostgreSQL

Back.

9.12 Project Management and Versioning

For the purpose of managing the project progress, tracking the development, and version control of the project, systems like Trello and GitHub are used. Keeping track of project progress against the outlined Gantt chart timeline is crucial for ensuring that the project gets completed on time. Version control is necessary for keeping track of development logs and backing up progress on cloud for safeguard.

9.12.1 Trello

Trello is a project management and tracking tool that provides a kanban-style interface, complete with boards, cards, timeline tracker, Todo list, doing list, done list, and various other features for work management (Trello, 2025). It also allows for the supervisors to keep track of our project progress.

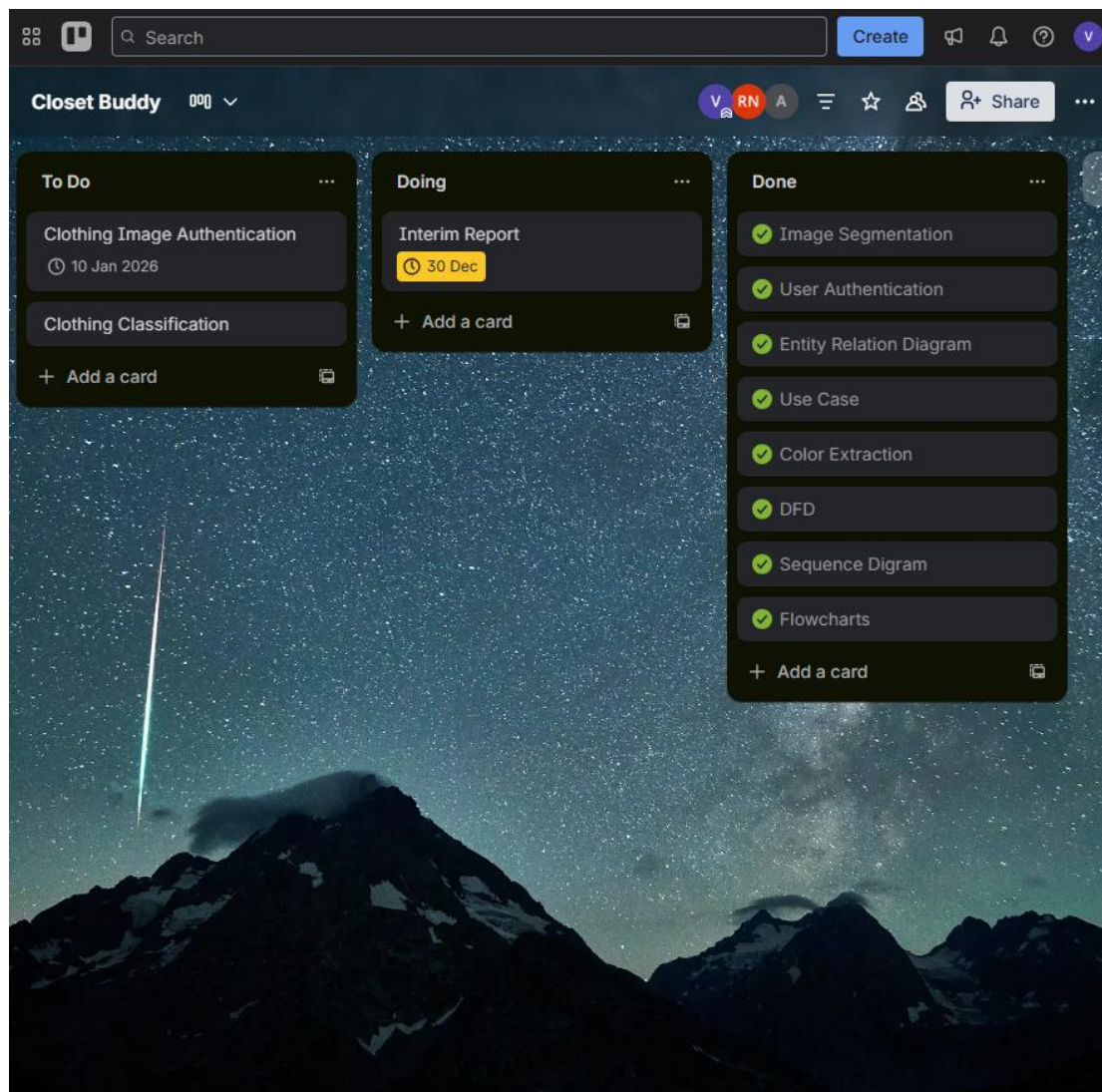


Figure 99: Trello Board

9.12.2 GitHub

GitHub is an excellent tool for version control of the project, which is built on Git. We can store, manage, and track changes in the code making it easier to review and backup the project in cloud (GeeksforGeeks, 2025). It is an online community of developers collaborating on different projects, sharing their works, and socializing with each other. It provides various tools for keeping track of the code and managing tasks, making it perfect for this project.

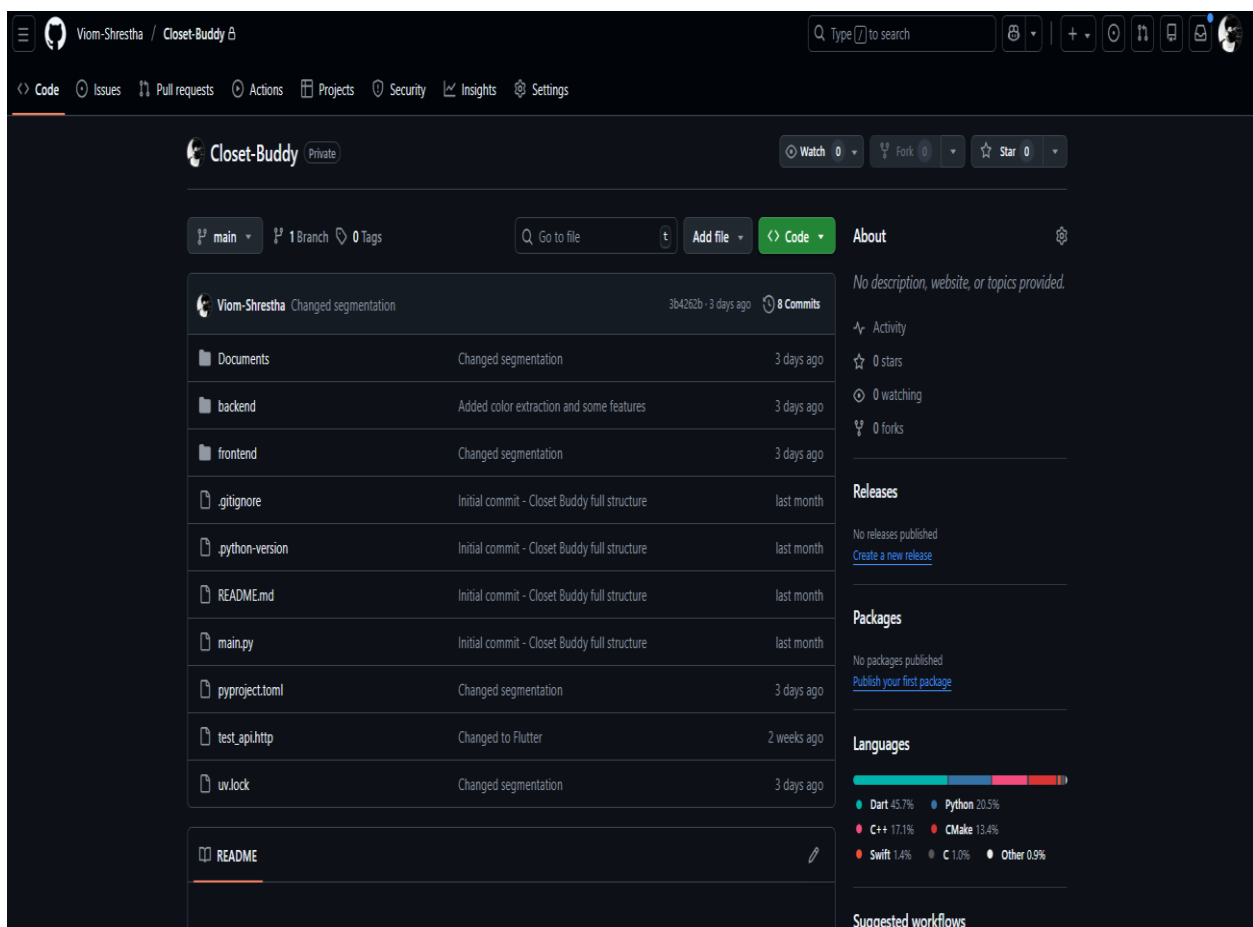


Figure 100: GitHub Repository for the project